

# 1. Что такое "вычислительная платформа" с точки зрения пользователя? Каков её состав?

## Что такое "вычислительная платформа" с точки зрения пользователя?

**Вычислительная платформа** - набор инструментов, позволяющих пользователю описать определённый вычислительный процесс и затем запустить его.

- **C, Python и другие языки программирования** - современные примеры инструментария вычислительной платформы.
- **Система команд процессора** - более низкоуровневый и традиционный пример такого инструментария.
- На каждой платформе пользователю предоставляются средства, с помощью которых он формулирует **вычислительную модель**: какие данные есть, какие действия над ними выполняются и в каком виде описывается алгоритм.

### состав:

#### 4 компонента вычислительной платформы:

- **Язык** - средство, в рамках которого формулируется вычислительный процесс.
  - Пример: синтаксис C.
- **Наборы типов** - данные, которыми можно оперировать в рамках языка.
  - Примеры: машинное слово, `int`, `char`, `pointer`.
- **Инстансы** - примитивные элементы, из которых составляется вычислительный процесс.
  - Примеры: операция `+`, вызов функции.
- **Паттерны** - практика использования языка, типов и инстансов вместе.
  - Они определяют, **как именно** пользоваться платформой для решения реальной задачи.
  - недостаточно иметь отвёртку, нужно ещё понимать, как ей пользоваться.
  - Паттерны переводят инструмент с уровня чисто технического ресурса на уровень человеческой деятельности.

---

## 2. Какие уровни абстракций относятся к дисциплине "Архитектура компьютера"? Почему границы определены

## именно так?

### Какие уровни абстракций относятся к дисциплине "Архитектура компьютера"?

Дисциплина рассматривает уровни **от середины операционных систем до цифровых схем**. нужно держать в голове следующую цепочку уровней:

- **Операционная система** - верхняя граница курса; уровень, где виртуальная память, процессы и прерывания переводятся из команд софта в реальные действия процессора.  
рассматриваются только те механизмы ОС, которые связаны с архитектурой.
  - память, процессы, прерывания.
- **Система команд процессора, ISA** - контракт между программой и процессором:
  - инструкции, регистры, типы данных, способы работы с памятью.
- **Архитектура процессора** - что процессор предоставляет программисту как вычислительную платформу. логическая модель системы: как устроена память, как работают прерывания и как процессор выполняет вычисления в целом
- **Микроархитектура** - внутренняя реализация архитектуры:
  - Control Unit, Data Path, конвейер, кэши и другие блоки.
- **Цифровая схемотехника**:
  - логические элементы, комбинационные схемы, триггеры и физическая организация цифровых узлов.

### Почему именно так?

- **ниже цифровых схем** - подробное изучения аналоговой электроники и физики.
- **Операционные системы частично** находятся выше рассматриваемой области и изучаются отдельно.
- Поэтому курс связывает программно видимый уровень компьютера с тем, как он реализуется вплоть до цифровых схем.

---

## 3. В чём заключается отличие Platform-Based Design от проектирования для конкретной платформы?

**В чём заключается отличие Platform-Based Design от проектирования для конкретной платформы?**

**Проектирование для конкретной готовой платформы** - традиционный подход: разработчик получает уже готовый выбранный язык, библиотеки, framework, ISA и аппаратную платформу, после чего реализует программу в рамках их возможностей.

- **Плюс** - можно переиспользовать готовые инструменты и быстро разрабатывать ПО.
- **Минус** - появляются уровни абстракции и внутренние механизмы, которые разработчик не контролирует.
- Это особенно плохо в низкоуровневых и встроенных системах, где есть жёсткие ограничения по энергии, теплу, времени реакции и условиям эксплуатации.

**Platform-Based Design, PBD** - двунаправленное проектирование.

- на основе сырых микросхем собираем системную платформу, закрывающую пространство задач, путем добавления необходимых именно для них системных блоков (контроллеры/ускорители нс/тд) .
- Одновременно с этим разрабатывается ПО, которое использует созданные механизмы.
- Граница между hardware и software определяется не сразу и может изменяться по мере проектирования.

железо подстраивается под требования софта, а софт переписывается под ограничения железа. **одновременно**

---

## **4. Почему большинство современных компьютерных систем считаются системами с преобладающей программной составляющей? Приведите примеры.**

**с преобладающей программной составляющей:**

- Если в процессе разработки компьютерной системы огромное влияние оказывает разработка программной составляющей то она - программная (с преобладающей программной составляющей)
- Причина: тотально снижаются затраты на "железную" составляющую.
  - за счет того что ее очень редко разрабатывают под определенную систему,
  - за счет модернизации производства
  - за счет роста затрат на программную составляющую.
- За редчайшими исключениями любая компьютерная система на самом деле программная система.

## примеры.

- производство и покупка процессоров: от 50 до 75% - затраты на ПО.
  - ноутбуки Apple: железная составляющая значительно меньше ОС, которая стоит над этим железом и позволяет так классно им пользоваться.
- 

## 5. Что такое информационная и управляющая система? Каковы их отличия (функции, ПО, аппаратура)?

### Что такое информационная система?

**Информационная система** - система, основная задача которой - преобразование или накопление полученных данных.

- **Критерий эффективности** - в общем случае «чем быстрее, тем лучше»; уменьшение среднего времени ответа обычно полезно.
- **Примеры подходов** - batch computing и distributed computing.
- **Примеры систем** - база данных, поисковая система и другие системы обработки информации.

### Что такое управляющая система?

**Управляющая система** - система, ориентированная на взаимодействие с реальным физическим миром с целью его контроля или управления.

характерно:

- **Встроенное исполнение** - система интегрирована в физический объект.
- **Ограниченные ресурсы** - энергия, тепло, память, вычислительная мощность.
- **Специализация** - функции, платформа и аппаратура часто проектируются под конкретную задачу.
- **Автономная эксплуатация** - система должна долго работать без постоянного участия человека.
- **Режим реального времени** - реакция должна быть предсказуемой и укладываться в установленный срок.

### Каковы их отличия: функции, ПО и аппаратура?

- **Информационная система** в первую очередь преобразует информацию и обычно оптимизируется по средней производительности.

- **Управляющая система** должна ещё физически взаимодействовать с объектом, поэтому сильнее зависит от датчиков, исполнительных устройств, специализированного hardware и требований реального времени.
  - Один продукт иногда может выполнять обе роли: `ffmpeg`, который может просто конвертировать файл либо участвовать в обработке и выводе данных в реальном времени.
- 

## **6. Что такое "требование реального времени"? Примеры систем. Что противоречит РВ в современных компьютерах?**

### **Что такое "требование реального времени"?**

- Если система работает в режиме реального времени - значит, что система работает предсказуемо и в срок
- Есть продукты, которые могут выступать как вычислительная платформа и как управляющая система: `ffmpeg`, которая может как
- конвертировать видео из одного формата в другой, так и читать файл с диска, выводя на экран в реальном времени.

### **Примеры систем.**

- система управления водосброса гЭС - фиксируем превышение уровня, следим n-ное время, если не уменьшается, считаем, что реально переполнение и в течение n-ного времени постепенно сливаем.
- система управления тормозами авто

### **Что противоречит РВ в современных компьютерах?**

- В современных компьютерах - очень многое: все оптимизации, улучшения и так далее, которые направлены на увеличение производительности в среднем, могут вносить неопределенные задержки в реакции системы, система теряет предсказуемость времени выполнения из-за спекулятивности
- 

## **7. Каковы этапы эволюции информационных систем: от пакетной обработки до облачных систем?**

## Каковы этапы эволюции информационных систем: от пакетной обработки до облачных систем?

Эволюция шла в сторону распределения вычислений и превращения вычислительных ресурсов в сервис.

1. **Пакетная обработка** - задачи формировались пакетами, передавались системе и последовательно обрабатывались. типа перфокарты. выдавался результат долго.
2. **Распределённые вычисления** - вместо единственного большого сервера появились несколько меньших независимых узлов, способных совместно или отдельно обрабатывать задачи.
3. **Персональные компьютеры и компьютерные сети** - появляется ПК, связанные локальной сетью - проводами в офисе.
4. **Интернет** - большое количество сетей разных офисов и вычислительных систем объединяется глобально. стало возможным использовать информационные системы не только в корпоративном пространстве но и удаленно
5. **Мобильные сети** - Доступ к информационным системам перестал быть привязан к конкретному месту или зданию, работать с ними стало возможно «на ходу».
6. **IoT и облачная инфраструктура** - массовое подключение к сети умных устройств и датчиков (Интернет вещей), а также переход на облачные серверы. Физическое компьютерное оборудование перестает быть ценностью и превращается в услугу, которую бизнес арендует под текущую нагрузку

---

## 8. Каковы этапы эволюции управляющих систем: от ИУС до КФС?

### Каковы этапы эволюции управляющих систем: от ИУС до КФС?

1. **ИУС - информационно-управляющие системы.**
  - Объект управления и управляющая система физически разделены и могут существовать как два отдельных объекта.
  - Управляющая система получает данные об объекте и выдаёт управляющее воздействие.
2. **ВсС - встроенные системы.**
  - Когда компьютеры стали достаточно маленькими, вычислитель стало возможно конструктивно встроить непосредственно в объект управления.
  - Принцип управления остаётся похожим на ИУС, но контроллер теперь физически находится внутри устройства.

### 3. РВсС - распределённые встроенные системы.

- Система управления перестаёт быть одним монолитным контроллером и распределяется между несколькими узлами.
- Узлы располагаются максимально близко к управляемым объектам, чтобы уменьшать задержки.
- **Пример - современный автомобиль:** по всему кузову раскиданы десятки мелких контроллеров, соединенных шиной: один контроллер сидит прямо в двери и управляет стеклоподъемником, второй — в фаре, третий — в тормозной колодке. Они распределены, чтобы провода не тянуть через всю машину и снизить задержки.

### 4. КФС - киберфизические системы.

- Физическая система уже не может нормально существовать без вычислительного управления.
- **Примеры:** гироскутер. без управления его смысла нет, тк не выполняются требования по его работе. он буквально не работает без управления.
- вычислительное управление - неотъемлемая часть объекта управления.

---

## 9. Каковы задачи и предмет дисциплины "Системная инженерия"? Какова роль системных инженеров?

### Каковы задачи и предмет дисциплины "Системная инженерия"?

- **Системная инженерия** - подход, отвечающий на то как делать успешные системы?
- Системная инженерия занимается разработкой больших систем.
- Большая система - это система, которую невозможно собрать у кого-то в голове целиком без системного инженера в силу ее масштабности.
- **Предмет системной инженерии** - архитектура и жц Audi, Boeing 787, нефтедобывающая платформа. Словом - предмет системной инженерии это большие системы

### Какова роль системных инженеров?

- **Роль системного инженера** - технический лидер, который
  - отвечает за целостность системы и соответствие требованиям.
  - согласует и управляет решениями на стыке подсистем
  - согласовывает работу разработчиков разного профиля.
  - синхронизирует тех процессы и стандарты между командами.

## 10. Что такое успешная система? Какие точки зрения (заинтересованные стороны) необходимы для построения успешной системы?

### успешная система?

**Успешная система** - система, при реализации которой удовлетворены все значимые заинтересованные стороны.

**Заинтересованные стороны, stakeholders** - физ лица, группы или организации, которые способны влиять на проект либо испытывают влияние от результатов проекта.

- К ним относятся не только заказчик и разработчики.
  - **государство, сертифицирующие органы**, субъекты, имеющие власть или влияние на проект.
  - Даже активисты, выступающие против проекта, могут быть stakeholder и иногда должны учитываться.

### точки зрения для построения успешной системы?

нужно комплексно рассмотреть её с **пяти точек зрения**:

- **Структура** — рассмотрение внутренних элементов системы и связей между ними.
    - При их объединении *новое качество* — новые свойства всей системы, которых не было у её частей по отдельности.
  - **Функциональное место** — где именно в реальном мире мы можем применить систему, чтобы она выполняла свою роль и приносила пользу.
  - **Операционное окружение** — окружение, в рамках которого функционирует система
  - **Жизненный цикл** — понимание: от того, откуда она появляется (концепт), до этапа эксплуатации и того, как выводить из эксплуатации.
  - **Обеспечивающие системы** — внешние системы, которые помогают продукту существовать на разных этапах его жизни, но не вносящие непосредственный вклад в его функционирование.
- 

## 11. Как можно рассматривать систему с точки зрения структуры? Каковы причины множественности структуры?

Как можно рассматривать систему с точки зрения структуры?

**Система с точки зрения структуры** - совокупность взаимосвязанных частей, которые совместно создают новое качество, отсутствующее у отдельных компонентов.

- Систему можно **декомпозировать** - разбить на подсистемы и составные части.
  - с помощью «диаграммы деления»: строим иерархию или дерево подсистем.
- Внутри одной физической системы можно выделять разные наборы связей в зависимости от решаемой задачи.

## **причины множественности?**

потому, что различным заинтересованным сторонам важны разные части системы и разные отношения между ними.

структура выбирается под конкретную точку рассмотрения и конкретного stakeholder.

- строитель рассматривает несущие конструкции; электрик - электропроводку;
    - сантехник - воду; дизайнер - размещение мебели.
  - Теоретически можно построить одну «гиперструктуру», содержащую все связи сразу, но она будет настолько сложной, что потеряет практический смысл.
- 

## **12. Как можно рассматривать систему с точки зрения функционального места? Кто такие заинтересованные стороны (Stakeholders)? Что такое операционное окружение?**

### **Как можно рассматривать систему с точки зрения функционального места?**

рассматривать систему с точки зрения функционального места значит:

- Идентифицировать систему от её назначения и внешнего интерфейса, а не от внутренней конструкции
- **Пример: насосная станция.** если сломался «насос 2», нас интересует его функциональная роль, а не серийный номер. Можно заменить насос другой моделью, сохранив тот же интерфейс и функцию; компонент изменился, но функциональное место осталось тем же.

### **Кто такие заинтересованные стороны (Stakeholders)?**

**Stakeholders** - физические лица, группы или организации, которые влияют или испытывают влияние от деятельности проекта и его результатов.

- Конкретная архитектурная точка зрения во многом определяется тем, какие проблемы соответствующего stakeholder нужно решить.

## Что такое операционное окружение?

**Операционное окружение** - окружение, в рамках которого система реально функционирует и с которым вынуждена взаимодействовать.

- **Пример для процессора:** система питания, память и периферийные устройства.
  - При проектировании важно учитывать не только саму систему, но и интерфейсы с этим окружением.
- 

## 13. Как можно рассматривать систему с точки зрения жизненного цикла? Что такое обеспечивающая система?

### рассматривать с точки зрения жизненного цикла?

**Жизненный цикл системы** - путь системы от возникновения идеи до прекращения её использования и вывода из эксплуатации.

Типовые этапы:

1. **Концептуальный этап** - зарождение идеи и определение того, какую систему вообще нужно создать.
2. **Разработка** - проектирование и реализация.
3. **Производство** - физическое изготовление и подготовка экземпляров системы.
4. **Использование** - непосредственная эксплуатация.
5. **Поддержка** - обслуживание, ремонт, обновление и сопровождение.
6. **Вывод из эксплуатации** - прекращение использования и корректное завершение жизненного цикла.

Для конкретной системы могут существовать дополнительные специфические этапы; их нужно продумывать заранее.

### обеспечивающая система?

**Обеспечивающая система** - система, дополняющая основную систему на определённом этапе её жизненного цикла, но может не вносить прямой вклад в выполнение основной функции продукта.

- **Примеры** - типа команда разработчиков на разработке и промышленное производство на производстве.
  - У каждой обеспечивающей системы также существует собственный жизненный цикл, который тоже необходимо учитывать.
- 

## 14. Каковы цели архитектурного проектирования компьютерных систем?

- Главная цель - **снижение рисков** - обнаружение ошибок на ранних этапах, обеспечение целостности системы и поддержка процессов системы на всех этапах жц.
  - архитектурный подход обеспечивает предсказуемость сроков и бюджета, повышая вероятность, что эти предсказания будут верны.
  - В цели не входит "сделать дешевле", "сделать быстрее"
- 

## 15. Что такое архитектура по Гради Бучу? Что представляют собой логическая и физическая структура?

### Что такое архитектура по Гради Бучу?

- **Архитектура** - логическая и физическая **структура компонентов системы и их взаимосвязи**, сформированная всеми стратегическими (важно в долгосрок) и тактическими (сейчас так удобно) проектными решениями, применяемыми во время разработки.

### Что представляют собой логическая и физическая структура?

- **Логическая структура** - устанавливает существование и роль ключевых абстракций и механизмов, которые будут определять архитектуру и общий дизайн системы
- **Физическая структура** - описывает конкретный программный и аппаратный состав реализации системы. Зависит от конкретной технологии

интернет-магазин, то классы `Customer`, `Order`, `Product` и связи между ними (например, «У одного *Order* может быть много *Products*») — это **логическая структура**.

интернет-магазин в физической структуре — это файл `server.jar`, запущенный на сервере `Ubuntu Base` в облаке `Yandex Cloud`, который обращается к базе данных

## 16. Что такое архитектура согласно ISO 42010? Что такое архитектурное описание?

### Что такое архитектура согласно ISO 42010?

- **Архитектура** - фундаментальные концепции или свойства системы в её окружении, воплощенные в её элементах, взаимосвязях, а также в принципах её проектирования и развития.
- Немного шуточных, но сущностных определений:
- **Архитектура** - все важное
- **Архитектура** - это такие проектные решения, ошибка в которых скорее всего приведет к тому, что проект будет закрыт, отменен или провален

### Что такое архитектурное описание?

- **Архитектурное описание** - документ, в котором описана архитектура для какого-то стейкхолдера. рабочий продукт (артефакт), используемый для выражения (фиксации) архитектуры.
- 

## 17. Как архитектурные решения влияют на проектные метрики? Что такое V-диаграмма и какую особенность разработки она демонстрирует?

### Как архитектурные решения влияют на проектные метрики?

Архитектурные решения определяют,

- сколько проект будет стоить
- и
- как быстро он будет реализован

Если же на старте архитектурная ошибка, то по **V-модели**, мы найдем её слишком поздно. В итоге исправление обойдется оч дорого, что проект мб придется просто закрыть.

### V-диаграмма и особенность разработки?

- **По вертикали** - уровень детализации (чем ниже тем более конкретно, чем выше тем детализация абстрактнее)  
каждому этапу проектирования слева соответствует свой этап проверки справа (например, архитектура и требования проверяются системными тестами).
  - **слева сверху вниз:**
    - Концептуальный дизайн -
    - требования и архитектура -
    - детальный дизайн
  - **снизу:**
    - реализация
  - **справа снизу вверх:**
    - интеграция, тестирование и верификация -
    - системная верификация и валидация -
    - эксплуатация и обслуживание.
  - **Особенность** - чем выше уровень, на котором мы находим ошибку, тем дороже нам влетит ее исправление. Также для левой части, чем выше уровень, на котором мы совершим ошибку, тем дороже будет ее исправление.
- 

## 18. Почему плохой менеджмент может увеличить бюджет проекта быстрее, чем другие факторы? Проблема передачи информации.

- **Менеджмент** - коммуникация между разными элементами системы.
  - **круг важных вопросов** менеджера и разработчика очень отличаются
  - поэтому когда менеджер формулирует задачу разработчику часть задачи разработчиками не понимается
  - Плохая коммуникация между системами могут привести к большим проблемам, неэффективным решениям, дорогим ошибкам и так далее.
- 

## 19. Сравните подходы к реализации вычислений в арифмометре и на логарифмической линейке. Аналогии с современными компьютерами.

- **Арифмометр** - цифровое устройство - вычисление строится как последовательность воспроизводимых операций; результат получается

непосредственно в ходе вычислительного процесса.

- Аналогия: CPU или ALU
  - область определения и значений: медленный рост сложности, средний рост длительности вычислений
  - **Логарифмическая линейка - аналоговое:** значения функций не вычисляются а находят в заранее подготовленной таблице функция-значение.
    - Аналогия: кэширование заранее вычисленных значений ускоряют вычисления ценой памяти
    - область определения и значений: быстрый рост сложности, минимальный рост длительности вычислений
- 

## **20. Каков был подход к расчёту артиллерийских таблиц группой людей? Аналогии с современными компьютерами (параллелизм уровня инструкций, надёжность)**

- подход реконфигурируемого потокового вычислителя.
    - Для вычисления берем математика, который формирует граф вычислений.
    - В каждую вершину графа ставим человека, который совершает только одну операцию, передает результат дальше.
      - Можно проверят - надежно: вычислять дважды, собрав два графа, дать им одни и те же значения, на выходе сравнить и если что пересчитать.
  - Плюсы в параллелизме и конвейнеризации:
    - независимые этапы могут выполняться одновременно
    - как только на одном этапе закончилось вычисление - запускаем новые
  - На практике это встречается в современных компьютерных системах:
    - Процессорах общего назначения (RISC)
    - В специализированных процессорах (CGRA процессора)
    - В распределенных системах
- 

## **21. Каково устройство электрического реле? Какие виды реле существуют и каковы области их применения?**

- **Электрическое реле** - автоматизированная кнопка, может быть нормально разомкнутое, нормально замкнутое. Если подан ток - катушка создает магнитное поле

и одни контакты замыкаются, другие размыкаются.

- использовались:
    - первые поколения компьютеров (ЭВМ),
    - там, где транзисторы не выдерживают,
    - в силовых шкафах управления,
    - в неблагоприятной среде и в ПЛК
  - виды:
    - Механические (концевой выключатель дверей в лифте)
    - Тепловые (в чайнике, пластина изгибается и отключает нагрев)
    - Оптические (реагируют на освещение)
    - временные (отключают по прошествии времени)
- 

## 22. Как реализуется булев базис на электрических реле?

- **И** - просто последовательно два реле (нормально разомкнутых)
  - **ИЛИ** - параллельно два реле (нормально разомкнутых)
  - **НЕ** - одно нормально замкнутое реле.
    - Когда тока на входе нет 0, контакты сомкнуты — на выходе есть сигнал (1).
    - Когда мы подаем ток на вход 1, магнитное поле притягивает контакт и **размыкает** цепь — на выходе ноль (0).
  - **RS-триггер** - простейшая ячейка памяти.
    - Строится на основе механизма обратной связи.
    - Выходной ток замкнувшегося реле направляется обратно в его же удерживающую катушку
- 

## 23. Опишите классическую схему включения электродвигателя с аварийной остановкой. Объясните, как работает обратная связь.

- есть две параллельные цепи между силовыми линиями:
  - первая - упр: последовательно нз аварийная остановка, нз остановка, блок включения(старт с параллельной ран), и ран
  - вторая - силовая: ран и мотор
- Сначала ток через нажатый **Start** на катушку **Run**. Катушка создает магнитное поле и **одновременно** замыкает оба **контакта Run** — один подхватывает питание в обход

кнопки Start , а второй включает Motor

- убираем нажатие с старт но все работает из-за **захвата тока в обход старта** через параллельный ран - обратная связь.
  - если эррор стоп или стоп то выключается и надо заново нажимать
  - эррор стоп специально изначально нормально замкнутые чтоб при случае чего все обесточить
- 

## **24. В чём заключается принцип развития иерархических систем Седова? Иллюстрируйте применительно к вычислительным платформам.**

### **В чём заключается принцип развития иерархических систем Седова?**

- **Принцип Седова** - чтобы получить большое разнообразие на следующем уровне иерархии, нужно ограничить разнообразие элементов на нижнем уровне. В вычислительных системах стандартизация нижнего уровня позволяет строить много разнообразных систем выше.

### **применительно к вычислительным платформам.**

чем больше операционных систем у нас будет, тем меньше разнообразия хорошего софта мы получим; чем больше языков программирования, тем меньше у нас будет библиотек для них

---

## **25. Что такое программируемые логические контроллеры (ПЛК)**

### **Что такое программируемые логические контроллеры (ПЛК)**

- **ПЛК** - разновидность ЭВМ, защищенная от помех, пыли и влаги, имеющая множество разъемов для подключения датчиков и исполнительных механизмов.
- **Задача:** возможность программировать логику с помощью графических языков - например, релейных схем, без изучения кода и программирования.
- Масштабирование сложности,
- нет износа,
- соответствие "документации" реальности (ибо это в контроллер и зашито)

- Применяются в системах управления в промышленности, в системах с высокими требованиями к надежности.
  - Можно применять для модернизации областей, где применялись релейные шкафы (огромные релейные схемы, вот их можно заменить куда меньшей по габаритам схемой. Кроме того эта схема будет надежнее за счет отсутствия механических частей)
- 

## **26. Какова структура программируемых логических контроллеров (ПЛК)? Причины отделения инструментальной составляющей?**

**Какова структура программируемых логических контроллеров (ПЛК)?**

- Устроены ПЛК примерно так:
    - входные модули ввода ->
    - само ПЛК (микро-компьютер) ->
    - модули вывода
  - снабжены оптической изоляцией между каждым модулем.
    - Позволяет отделить контроллер от объектов, с которыми он интегрирован (если что-то пойдет не так на входе или выходе, это не выведет из строя сам контроллер).
  - Инструментальная составляющая (среда разработки и прочее) отделены от исполняющей системы ввиду тотальной ограниченности ресурсов самого контроллера.
- 

## **27. Каковы особенности аппаратного обеспечения программируемых логических контроллеров (ПЛК)? Объясните причины таких особенностей.**

**особенности аппаратного обеспечения программируемых логических контроллеров (ПЛК)**

- **Защищенное исполнение:** Предназначены для работы в агрессивных промышленных средах. Имеют пыле- и влагозащищенные корпуса, устойчивы к сильным вибрациям, ударам и широкому диапазону температур.

- Модули ввода и вывода изолированы от центрального процессора. Если на датчик или мотор прилетит высоковольтный скачок напряжения, ПЛК останется цел.
- Часто устойчивы к электромагнитным помехам, механическим повреждениям
- имеют внутренние системы, перезагружающие схему, если она (вдруг) зависнет

## причины особенностей.

они разрабатывались под конкретный контекст их применения.

ПЛК работают на заводах и фабриках в условиях грязи, вибраций и сильных помех от двигателей. отказ контроллера может привести к аварии или огромным убыткам. железо проектируют сверхнадежным и физически защищенным, а его архитектуру делают предсказуемой, чтобы гарантировать работу в жестком реальном времени без малейших задержек

## 28. Каковы особенности программного обеспечения программируемых логических контроллеров (ПЛК)? Объясните причины таких особенностей.

- 1. возможность программирования с помощью:
    - релейно-контактных схем
    - функциональных блочных схем - нечто среднее между логической схемой с простыми условиями и соединениями
    - последовательных функциональных диаграмм - Что-то типа конечных автоматов, позволяет описать систему с переключающимися состояниями
    - структурированного текста - почти программирование
  - ПРИЧИНА: Это позволяет использовать ПЛК без знания программирования.
- 2. работа в **бесконечном и очень быстром цикле**, который состоит из трех шагов (миллисекунды):
    - проверка всех датчиков и кнопок и их состояния
    - выполнение схемы или код от начала до конца.
    - подача сигналов на моторы лампочки и тд
  - ПРИЧИНА: бесконечный нужен чтобы постоянно проверять актуальное состояние.

## 29. Что такое булев базис и какова его роль в вычислительной технике? Приведите примеры.

- принципы вычислительной техники, на которых построена булева алгебра:
  - **двоичная логика** - все можно представить в виде дискретных сигналов - единиц и нулей
  - полный **набор булевых операций**, комбинируя которые можно реализовать любое преобразование:
    - И, ИЛИ, НЕ
    - штрих Шеффера - отрицание конъюнкции И-НЕ. ноль если оба входа дают 1
    - стрелка Пирса - ИЛИ-НЕ. истинно когда оба ложны.
  - комбинационные схемы - из булевых функций собираем преобразователи, которые работают за один прогон
  - триггер - хранение состояния

Из перечисленных выше пунктов (имея их) мы можем построить всю современную вычислительную технику (кроме квантовых компьютеров и аналоговых вычислителей)

---

## 30. Что такое двоичное кодирование сигналов? Каковы его достоинства и недостатки? Что такое запретная зона?

- **двоичный сигнал** - способ представления информации, при котором непрерывный электрический сигнал (напряжение) разделяется на два четких диапазона: один интервал значений считается логическим нулем, а другой — логической единицей.
  - Сигнал передается из источника по проводнику в приемник. В приемнике диапазоны декодирования намеренно делают чуть шире, чем у источника. для того, чтобы компенсировать затухание сигнала и помехи в проводе.
  - **Запретная зона** — диапазон напряжений между логическим нулем и логической единицей. В норме сигнал не должен принимать эти значения. если из-за сильной помехи сигнал попадет в запретную зону, система поймет, что произошла ошибка передачи.
- **Достоинства:**
  - **надежно и помехоустойчиво** - главное понять в какой диапазон попало значение а не определять точный сигнал.
  - **простая арифметика** - нужны только простые транзисторные ключи (вкл/выкл) и булева логика.
  - точность и диапазоны наращиваются разрядностью
  - погрешности можно рассчитать изначально в процессе проектирования.

- **Недостатки:**
    - ошибки округления, из-за которых простые десятичные дроби (вроде 0.1) превращаются в бесконечные двоичные
    - потеря точности при оцифровке реального мира
    - физическое ограничение по скорости: при слишком быстром переключении сигнала проводники начинают сглаживать импульсы, уводя их в ту самую запретную зону».
- 

## 31. Какова роль машинного слова в устройстве процессора? Что такое Big- и Little-endian?

- **машинное слово** - фиксированная совокупность бит, которую процессор может обработать как единое целое за один такт. Размер машинного слова определяет разрядность процессора: у 32-битных - 4 байта, у 64-битных — 8 байтов.
  - Не всегда машинного слова бывает достаточно для наших вычислений. Тогда, например, большое число может быть представлено как совокупность машинных слов и возникает вопрос как из отдельных слов составить полное число:
    - **big-endian** - запись слов "большим концом", т.е. сначала идут старшие машинные слова (или байты), а затем младшие: 0xABCDEF -> 0xAB 0xCD 0xEF
      - протоколы соединения TCP/IP
    - **little-endian** - запись слов "малым концом", т.е. сначала идут младшие машинные слова (или байты), а затем старшие 0xABCDEF -> 0xEF 0xCD 0xAB
      - **little-endian** - стандарт для архитектуры **x86/x64** + также в мобильных процессорах ARM, так как аппаратно процессору проще начинать арифметические операции именно с младших разрядов.
- 

## 32. Какие существуют способы кодирования целочисленных данных? Что такое позиционное кодирование, дополнительный код, код Грея, BCD?

- **Позиционная система счисления** - способ записи чисел, где вклад каждой цифры зависит от её позиции (разряда). В двоичной системе веса разрядов справа налево растут как степени двойки: 1, 2, 4, 8 и так далее.
- **Дополнительный код** - стандартный способ представления отрицательных чисел в процессорах. Чтобы его получить, мы инвертируем все биты положительного числа (0 меняем на 1, а 1 на 0) и прибавляем к результату единицу.

- Символьное хранение - число хранится как последовательность символов в текстовом формате, где на каждую цифру выделяется по байту. В вычислениях почти не используется из-за огромной медлительности, но удобно для вывода на экран.
  - Код Грея - способ двоичного кодирования, при котором любые два соседних числа (например, 3 и 4) отличаются в двоичном виде **строго в одном бите (разряде)**. При переходе от числа к числу биты изменяются одновременно, что исключает случайные аппаратные сбои при чтении данных.
  - BCD - способ кодирования, где каждая цифра десятичного числа записывается отдельно 4 битами. 123 = 0001 0010 0011
- 

### 33. Какие существуют способы кодирования бинарных данных? Что такое Base64, Base58?

- как можно бинарные данные (картинки, файлы, ключи) представлять через текст:
    - **Base64** - способ кодирования, при котором каждые три байта (24 бита) разбиваются по 6 бит и преобразуются в 4 печатных текстовых символа. Использует алфавит из 64 символов (латиница, цифры и знаки + , / ).
      - Если исходных байт не хватает для ровного деления, в конец строки добавляются символы = , чтобы длина итогового текста всегда была кратна 4
    - **Base58** - модификация Base64, созданная специально для **удобства ручного ввода человеком**. Из алфавита намеренно убрали все похожие друг на друга символы, в которых легко ошибиться: 0 (ноль) и O (заглавная o), I (заглавная и), l (строчная эль), а также спецсимволы + и / .
      - В её конец дополнительно зашивается хэш-сумма (контрольная сумма). Если человек опечатается хотя бы в одном символе при вводе (например, криптокошелька), система мгновенно обнаружит ошибку и заблокирует транзакцию.
- 

### 34. Что такое комбинационные схемы? Что такое переходный процесс? С чем связаны задержки и накопление ошибки? Как связаны комбинационные схемы с параллелизмом уровня бит?

#### Комбинационная схема

цифровая схема из логических элементов ( AND , OR , NOT , а также BUF - буфер, который

не меняет логику, а лишь электрически усиливает затухающий сигнал для передачи дальше). у комб схемы состояние выходов в любой момент времени зависит *только* от текущего состояния входов.

### ПЕРЕХОДНЫЙ ПРОЦЕСС

временной интервал после изменения входов, в течение которого электрический сигнал на выходах «устаканивается» в стабильное состояние. Пока процесс идет, по разным веткам схемы могут возникать ложные кратковременные переключения

### ЗАДЕРЖКИ

- Чтобы переключиться из 0 в 1, каждый транзистор нужно «зарядить», на что требуется время задержки вентиля.

#### НАКОПЛЕНИЕ ОШИБКИ:

- В длинных цепочках логики задержки накапливаются. Если мы подадим тактовый импульс слишком рано, приемник прочитает промежуточное не устоявшееся значение. Это приведет к аппаратной ошибке вычислений.

### ПАРАЛЛЕЛИЗМ БИТ

В схемах инструкции выполняются не последовательно для каждого бита, а одновременно. За один такт обрабатываются все биты.

---

## 35. Какие состояния существуют в комбинационных схемах (0, 1, x, z) и что они означают?

- 0 - стабильное состояние низкого напряжения.
  - 1 - стабильное состояние высокого напряжения.
  - x - неизвестно, значение может быть произвольным.
    - когда входное значение не входит в область определения функции
    - Во время переходного процесса
  - z - состояние «высокого сопротивления». Провод физически отключен от схемы внутри транзистора, ток по нему не течет, он «висит в воздухе».
    - позволяет подключать выходы множества разных элементов к одной шине, только одно устройство переходит в режим 0 или 1 и передает данные, а все остальные устройства принудительно уходят в Z-состояние, чтобы не мешать и не создавать конфликтов
-

## 36. Каковы особенности реализации "условного оператора" в комбинационных схемах?

- Глобально есть два сценария, как мы можем получать условный оператор.
    - Есть два входа и мы хотим выбрать, из какого входа получить данные
      - **Решение** - мультиплексор. у него есть информационные входы и управляющий вход (селектор). Селектор работает как переключатель: в зависимости от его значения (0 или 1), на выход транслируется сигнал либо с первого, либо со второго входа.
    - У нас есть один входной сигнал, и мы хотим выбрать, на какой из выходов (к какому потребителю) его отправить.
      - **Демультимплексор (DEMUX)**. Он принимает один сигнал и перенаправляет его на тот выход, который указан на управляющем входе (селекторе). На остальных выходах при этом будет логический ноль.
      - Если нам нужно не просто направить ток, а записать данные в конкретную ячейку памяти (D-триггер), мы используем элемент **И (AND)** на линии тактового сигнала (клока). Ток подается на все триггеры сразу, но клок «пропускается» через вентиль **И** только к тому устройству, которое должно защелкнуть (сохранить) это значение.
- 

## 37. Что такое триггеры в цифровых схемах? Каковы варианты их использования?

- **Триггер** - класс электронных устройств, обладающих способностью длительно находиться в одном из двух устойчивых состояний и уметь менять их под действием внешних факторов
  - Позволяют:
    - **зафиксировать состояние** линии на длительное время (всевозможные ячейки памяти).
    - **сократить задержку** в комбинационной схеме, что позволит одной части комбинационной схемы работать уже над другими входными данными, пока остальная схема доделывает прошлые.
    - создавать **многотактные схемы** - Триггеры меняют состояние по импульсу клона и гарантируют пошаговое синхронное выполнение.
    - **Устранениедребезга контактов**: Триггер позволяет защелкнуть самый первый импульс и игнорировать последующийдребезг.
-

## 38. Что такое D-триггер и RS-триггер? Какие существуют варианты условия изменения состояния?

**D-триггер** - триггер с одним информационным входом **D**. Он просто принимает (запоминает) состояние этого входа и передает его на выход, работая как классическая защелка одного бита данных.

**RS-триггер** - триггер с двумя входами. Вход **S (Set)** устанавливает выход в 1, а вход **R (Reset)** сбрасывает выход в 0. Если на обоих входах нули, триггер просто хранит свое прошлое состояние.

- По фронту
    - триггер реагирует на вход в момент **скачка** сигнала клона — либо в момент перехода с 0 на 1 (передний фронт), либо с 1 на 0 (задний фронт).
  - По уровню (фиксируем при определенном уровне)
    - Триггер меняет выход всё то время, пока клон равен единице. Когда клон падает в ноль, триггер защелкивается.
- 

## 39. Что такое пространственные и временные вычисления? Как они могут быть использованы для оптимизации процессоров?

### Пространственные:

подход, при котором мы жертвуем площадью и увеличиваем ее ради ускорения вычислений.

делаем один большой сумматор, на 16 бит, который сразу их сложит, но займет больше места.

### Временные:

Подход, при котором жертвуем временем ради уменьшения площади.

переиспользуем один и тот же маленький аппаратный блок, выполняя задачу пошагово (сумматор на 8 бит), тогда будет вычисляться больше времени.

- переход от временных вычислений к пространственным позволит организовать параллелизм уровня задач, инструкций, бит. И он же позволит сделать конвейерную организацию работ за счет многотактовости
- 

## 40. Что называют синхронной схемотехникой? Каковы её достоинства и недостатки?

## Синхронная схематехника

такая схематехника, в которой изменение элементов памяти происходит одновременно по блоку, т.е. возможно существование многотактовых схем

Достоинства:

- проще синхронизация - Весь компьютер работает синхронно в одном ритме.
- просто рассчитать время прохождения сигнала - измерить длину самого долгого пути тока.
- Нет гонок - Сигналы в проводах не обгоняют друг друга а приходят одновременно
- переходные процессы сведены к минимуму - все по блоку.

Недостатки:

- Тактовая частота схемы определяется самым медленным блоком - медленные тормозят быстрых.
- Дискретизация входных данных по времени - Компьютер не может реагировать на живой мир (например, на звук в АЦП) непрерывно. Он проверяет входящий сигнал строго в моменты ударов такта.

---

## 41. Почему цифровая схемотехника оперирует уровнями, а не сигналами? Какие возможности это открывает и какие проблемы создаёт?

Схемотехника оперирует уровнями, потому что **в проводе ток есть всегда**. Железо не умеет отправлять пакеты данных, как программа — провод не бывает пустым. держится какое-то напряжение — либо ноль, либо единица.

- **Возможности:**

- Все значения меняются и передаются **одновременно** на физическом уровне. Нет последовательной очереди на отправку между шинами.

- **Проблемы:**

- Требуется **большое количество логики** мультиплексоры, триггеры, чтобы просто отсекал нужные уровни от ненужных, которые давят на входы нескольких устройств всегда.
  - Если мы забыли прописать какое-то условие в схеме, на проводе всё равно выставится случайный ток, и компьютер выдаст случайный бред на выходе
-

## 42. Каковы ключевые тенденции в производстве радиоэлектронной аппаратуры и связанные с этим проблемы?

### Монтаж радиоэлектронной аппаратуры

- процесс сборки отдельных радиодеталей (транзисторов, резисторов, чипов) в единое работающее устройство на печатной плате.

### Ключевые тенденции:

- Рост уровня интеграции. Стремление поместить как можно больше элементов в минимальный объем пространства.
- наушники с шумодавом - процессор, микрофоны, Bluetooth-антенна и аккумулятор в корпусе.
- Снижение затрат на производство изделия при массовом производстве.
- Усложнение производственной цепочки (из-за роста интеграций необходимо подтягивать технологичность производства, в частности его автоматизацию, что приводит к усложнению производственной цепочки)

### проблемы:

- **Перегрев:** Миллиарды транзисторов, зажатые в маленьком корпусе, выделяют много тепла. Если устройство не охлаждать, оно просто расплавится или сгорит.
- **Нулевая ремонтпригодность:** Если в наушниках ломается один микрочип, заменить его отдельно невозможно — приходится выбрасывать и менять всю плату целиком.
- **Помехи:** Когда провода и элементы расположены слишком близко друг к другу, их электромагнитные поля начинают пересекаться, создавая взаимные помехи и искажая цифровые сигналы.

---

## 43. Что такое навесной монтаж, монтаж на печатную плату, штырьевой и поверхностный монтаж? Как они обеспечивают "гибкость"?

- **Навесной монтаж (объемный)** — ручная пайка деталей обычными проводками.
  - **Этапы:**
    1. Расставляем крупные детали на каркасе
    2. Вручную зачищаем и сгибаем провода
    3. Спаиваем их паяльником по схеме
    4. Заливаем клеем (компаундом) для прочности.

- **Плюсы и минусы:** Просто без завода, но громоздко, ненадежно.
  - **Гибкость:** Высокая **ремонтная гибкость**. Ошибку в схеме можно исправить за секунду, просто перепаяв провод прямо на коленке.
  - **Монтаж на печатную плату** — автоматизированная сборка схем на текстолитовой пластине с готовыми медными дорожками (от однослойных до многослойных).
    - **Плюсы и минусы:** Огромная плотность элементов. Появляются чипы на шариках-ножках прямо на нижней части чипа. В массе это очень дешево, но дико дорого для одного экземпляра.
    - **Гибкость:**
      - Высокая **конструкторская гибкость**. Позволяет развести сложнейшие схемы со всеми связями.
      - Но **ремонтная гибкость нулевая** — ошибку внутри слоев платы исправить невозможно, только выбрасывать.
    - **Включает два подхода:**
  - **1. Штыревой монтаж (THT):**
    - **Суть:** Ножки деталей вставляются в сквозные отверстия в плате и запаиваются с обратной стороны. Нужен для крупных деталей (трансформаторы), чтобы они держались прочно.
    - **Этапы:**
      1. Сверлим дырки в плате
      2. Вставляем ножки деталей
      3. Прокатываем плату над ванной с жидким оловом (пайка волной)
      4. Срезаем лишние хвосты ножек.
  - **2. Поверхностный монтаж (SMD):**
    - **Суть:** Крошечные детали укладываются роботом прямо на поверхность платы и запекаются. Это главный метод для современных процессоров.
    - **Этапы:**
      1. Через трафарет намазываем на плату паяльную пасту
      2. Робот-автомат с бешеной скоростью расставляет детали на пасту
      3. Запекаем плату в длинной печи, чтобы паста расплавилась и припаяла детали
      4. Моем плату и проверяем рентгеном или камерами.
- 

## 44. Какова производственная цепочка поверхностного монтажа? Каковы её этапы?

**Суть:** Это полностью автоматизированный конвейер. — всё делают роботы и печи, чтобы добиться микроскопической точности и бешеной скорости сборки.

- **Её этапы:**

1. **Нанесение паяльной пасты :** На плату накладывают металлический трафарет с дырочками. По нему раскатывают паяльную пасту (густую смесь из микро-капель олова и флюса). Паста ровным слоем остается строго на контактных площадках, куда сядут детали.
  2. **Установка компонентов (Робот-установщик / Pick and Place):** Плата едет к специальному роботу-автомату. Он с бешеной скоростью (десятками тысяч деталей в час) забирает из лент крошечные SMD-компоненты и процессоры и расставляет их на липкую пасту.
  3. **Запекание (Оплавление припоя в печи):** Плата въезжает в длинную конвейерную печь, где температура плавно растет. Паяльная паста плавится, превращается в жидкое олово, центрирует детали за счет поверхностного натяжения, а на выходе остывает и намертво припаивает их.
  4. **Контроль и отмывка (AOI и рентген):** Плату моют от остатков химии. Затем автоматическая оптическая инспекция (камеры) или рентген проверяют плату на микро-трещины и ошибки. Если всё ок — плата готова.
- 

## 45. Какова производственная цепочка кремниевого производства? Каковы особенности формирования цены изделия?

- **Суть:** Это создание самих полупроводниковых микросхем (кристаллов процессоров) на кремниевых фабриках. Процесс идет на нанометровом уровне в стерильных «чистых комнатах», где любая пылинка может уничтожить всю партию.
- **Её этапы:**
  1. Обычный кремниевый песок очищают и плавят, чтобы вырастить из него идеально чистый цилиндрический монокристалл кремния.
  2. Этот кремниевый цилиндр нарезают алмазными пилами на тончайшие круглые блины — пластины (вафли).
  3. Пластины покрывают светочувствительным лаком. Специальный лазер через трафарет (маску) «выжигает» на ней миллиарды микроскопических транзисторов и дорожек.
  4. Кремний обстреливают ионами других веществ, чтобы в нужных местах он начал проводить ток, превращаясь в транзисторы.
  5. Специальный робот иглами проверяет работу процессоров прямо на круглом блине, отсекая бракованные.
  6. Пластины разрезают на отдельные крошечные квадратики (кристаллы). Каждый рабочий кристалл упаковывают в защитный пластиковый или керамический корпус

с выводами.

Особенности формирования цены изделия в кремниевом производстве

- На одной круглой пластине получаются сотни процессоров. Если из-за микропылинки или дефекта 50% чипов окажутся бракованными, то цена оставшихся рабочих процессоров мгновенно вырастет в два раза.
- Если при производстве кремниевого кристалла (процессора) из-за пылинки ломается одно или два ядра, завод не выбрасывает чип. Бракованные ядра просто аппаратно отключают (выжигают к ним дорожки лазером). получается процессор подешевле на меньше ядер.

---

## 46. Какие трудности связаны с производством аппаратного обеспечения (подготовка производства, элементная база)?

Собственно трудности, по сравнению с производством программного продукта (или веб-сервиса), будут возникать на каждом этапе:

- **Подготовка производства**

Чтобы запустить плату в тираж, нужны трафареты для пасты и написать программы для роботов-установщиков. Если в схеме найдется ошибка, весь этот дорогой предпроизводственный этап придется оплачивать и делать заново.

- **Элементная база (Зависимость от поставщиков):**

- Код собирается из бесплатных библиотек, а для железа детали нужно покупать. Если завод в Китае задержит поставку одного копеечного конденсатора — встанет весь конвейер, и вы не сможете выпустить продукт.

- **Логистика** — доставка тонн компонентов через границы и таможню

- **Склады** — детали нужно где-то хранить, соблюдая разные требования.

- **Специалисты** — нужны инженеры-технологи, операторы станков и монтажники .

- **Производственная цепочка (Аналог GitHub Actions)** — реальный конвейер из роботов и печей, который может сломаться или застрять.

- **Тестирование** — железо нужно проверять физически: рентгеном, вольтметрами, гонять в термокамерах и трясти на вибростендах.

- **Упаковка** — коробки, пластик, пенопласт, которые стоят денег и занимают место.

- **Дистрибуция (Аналог Docker Hub)** — готовые коробки нужно физически развезти фурами и кораблями по магазинам по всему миру.

- **Гарантийный ремонт (Аналог Support)** — Если баг нашли в железе — нам пришлют коробку обратно, и инженер будет вручную перепаявать детали по гарантии, что дико дорого.
- 

## 47. Какие трудности связаны с эксплуатацией аппаратного обеспечения (амортизация, физический доступ, среда эксплуатации)?

- **Амортизация и износ:** Железо физически изнашивается: конденсаторы высыхают, вентиляторы забиваются пылью и ломаются, термопаста пересыхает, а транзисторы со временем деградируют от постоянного нагрева и токов.
  - **Необходимость физического доступа:** Если у железного контроллера на заводе отвалился провод или сгорел блок питания — никакой удаленный доступ не поможет. Нужен человек, который физически приедет, разберет корпус и поменяет деталь.
  - **Особенности среды эксплуатации:** Реальное железо (например, ПЛК) работает на жаре, морозе, под дождем, в условиях дикой вибрации станков и химически агрессивных сред, которые разъедают контакты.
  - **Долгосрочные эффекты (Физика материалов):** Со временем внутри железа начинают происходить необратимые химические и физические процессы. Бессвинцовое олово на платах со временем само по себе начинает проращивать микроскопические металлические «усы», которые могут закоротить соседние дорожки через несколько лет работы и намертво сжечь плату.
  - **Сложность обновлений (Апгрейд):** Физически обновить или заменить сломанное железо на космическом аппарате за пределами Солнечной системы или земли — технически сложно или невозможно. Проект либо должен работать на старом железе до конца, либо его придется закрыть.
  - **Поддержка, отладка и восстановление:** Найти баг в работающем железе невероятно трудно. Инженеру-схемотехнику, чтобы понять, почему плата сбоит, приходится подключать осциллографы, логические анализаторы, выпаивать детали и искать микротрещины в платах.
- 

## 48. Какие подходы к решению проблемы "устаревающей аппаратуры" существуют с точки зрения аппаратуры и ПО?

- **Проблема устаревающей аппаратуры** - Ситуация, когда важная система должна работать дальше, но её компоненты сняты с производства.
  - **Аппаратные подходы (Hardware):**
    - **Перепроектирование на новой элементной базе:** Полная переделка схемы на современных деталях.
    - **Модульная организация через стандартные интерфейсы:** Мы не меняем всю систему целиком. Благодаря модульности, мы можем вытащить одну сгоревшую старую плату и заменить её на новую, совместимую по разъемам.
      - **Имитация старых систем:** Создание современных адаптеров-переходников. имитация перфокарт или лент. Снаружи для старого компьютера это выглядит как древний накопитель, но внутри стоит микроконтроллер, который читает данные с современной быстрой SD-карты или SSD.
    - **Использование ПЛИС (FPGA):** - Специальных «программируемых» микросхем. На них можно на физическом уровне (в кремнии) воссоздать точную схему старинного процессора, которого больше нет в продаже.
  - **Программные подходы (Software):**
    - **Виртуализация и симуляторы:** Мы полностью списываем старое железо на свалку. Вместо этого мы берем современный мощный компьютер, запускаем на нем **виртуальную машину (эмулятор)**, которая программно имитирует архитектуру старой ЭВМ. На этой виртуалке без изменений запускается старый софт, написанный полвека назад.
- 

## 49. Что такое концепция 2-этапного производства? Объясните подходы к "конфигурированию": сборка, комплектация, реконфигурация, программирование.

- **2-этапное производство** - стратегия, при которой на первом этапе массово и дешево производятся одинаковые «универсальные» железки (платформы), а на втором этапе они настраиваются (конфигурируются) под конкретные задачи разных пользователей.
  - **Главный плюс:** Это позволяет создавать **кастомные и уникальные продукты** по низкой цене массового конвейерного производства. Количество промежуточных уровней между этапами может быть любым.
    - При проектировании платформы архитектор **заранее закладывает универсальные разъемы**, свободную память и возможность изменения логики.

### 4 подхода к "конфигурированию" (способы настройки на 2-м этапе):

- **На уровне сборки и комплектации:** Физическая настройка железа.
    - Здесь мы физически меняем железо: вставляем платы расширения, собираем схемы на макетных платах или переключаем **перемычки**, чтобы направить ток по нужной дорожке.
      - *Джамперы (перемычки)* — замыкание нужных контактов на плате для выбора режима работы.
      - *Платы расширения* — добавление новых блоков в готовые разъемы (например, видеокарты).
      - *Макетные платы* — сборка и соединение компонентов проводками на специальной базе.
  - **Реконфигурация:** Изменение связей между готовыми вычислительными блоками без их физической замены.
    - не меняем сами детали, но меняем связи между ними. В современном мире за это отвечают чипы ПЛИС, где связи между транзисторами перестраиваются программно.
      - *Исторический пример:* Выстраивание людей-вычислителей в разном порядке для расчета математических таблиц.
      - *Технический пример:* Перекоммутация кабелей и блоков в старых ЭВМ (или программное изменение связей в современных ПЛИС).
  - **Программирование:**
    - Самый гибкий софтверный уровень. Железо и связи неизменны, меняется только кусок инструкции в памяти, который определяет, как должен работать вычислитель прямо здесь и сейчас.
- 

## 50. Каково определение программной системы согласно OMG Essence? Каковы её части?

- **Программная система** - система, собранная из ПО, АО и данных, предназначенная для решения конкретных задач и принесения ценности заинтересованным лицам. Данные являются важнейшей частью этой системы (дают ей первичную ценность), но сами по себе не являются программным обеспечением.
  - Шрифты. БД.

### части?

АО - Физическое железо, процессоры, ПЛК, сервера, память и кабели, на которых этот код физически запускается и работает.

ПО - Сам исполняемый код, логика, алгоритмы и программы, написанные

разработчиками (то, что управляет процессом).

Данные - Цифровая информация, контент и настройки (например, шрифты, таблицы БД, словари), которые система обрабатывает. Они дают системе первичную ценность, но кодом не являются.

---

## 51. Что такое программное и аппаратное обеспечение? Что означают понятия Hardware и Software? Сопоставьте их.

**Аппаратное обеспечение** — электронные и механические части вычислительного устройства, входящие в систему или сеть, исключая ПО и данные. Проще: физические элементы, взаимодействие которых позволяет выполнять прикладную задачу.

**Программное обеспечение** — совокупность программ и программных документов, необходимых для эксплуатации системы; оно заставляет аппаратную часть выполнять вычисления или функции управления.

При этом **Hardware и Software** — не строго синонимы «железа» и «ПО». Здесь смысл скорее в изменяемости:

- **software** — то, что легко, быстро и относительно дёшево менять;
- **hardware** — то, изменение чего связано с долгими и дорогими этапами.

Например, Minix внутри процессора формально является ПО, но с точки зрения готового продукта ведёт себя как hardware, потому что пользователь практически не может его изменить.

---

## 52. Какие возможности открывает программное обеспечение (цикл разработки, гибкость, контроль, и т.д.)?

Программное обеспечение даёт системе прежде всего **гибкость и быстрый цикл изменений**.

Основные возможности:

- быстрый цикл разработки;
- ПО легко заменить уже у пользователя;
- возможно удалённое обновление, в том числе автоматически;
- создание и внедрение ПО хорошо автоматизируются;

- можно реализовывать системы очень высокой логической сложности без изменения физической аппаратуры.

Главное отличие от hardware: после выпуска продукта поведение системы можно существенно менять программно, не перестраивая производственную цепочку и не заменяя физические компоненты.

---

### **53. Чем отличается типовое проектирование Hardware/Software от совместного (CoDesign) проектирования Hardware/Software? Каковы достоинства и недостатки?**

При **типовом HW/SW-проектировании** сначала проектируют аппаратуру: систему команд, интерфейсы, схемотехнику. Затем программисты пишут ПО, используя уже зафиксированные возможности этой аппаратуры.

Проблемы такого подхода:

- **шаблонные решения** из-за жёсткого разделения аппаратчиков и программистов;
- **высокие интеграционные риски**: отдельно HW и SW могут работать, а вместе — нет;
- **последовательная разработка**, которая увеличивает сроки.

**Hardware/Software CoDesign** старается как можно дольше не фиксировать границу между аппаратурой и ПО. Сначала работают на уровне моделей, где функцию ещё можно перенести из hardware в software или наоборот, и только позднее принимают окончательное решение.

Плюс — можно лучше совместить требования задачи и возможности реализации. Минус — проектирование требует тесной координации разных специалистов и становится организационно сложнее.

---

### **54. Что такое "Модель вычислений"? В чём назначение моделей вычислений? Приведите примеры.**

**Модель вычислений** — это математическая абстракция и средство формального описания вычислительного процесса. Она задает правила выполнения операций,

организации данных и перехода системы из одного состояния в другое, не привязываясь к конкретному процессору

Она **нужна**, чтобы описывать вычисления не через конкретный процессор или программу, а через более общие правила. применяются:

- **Для теоретического доказательства свойств алгоритмов:** позволяют оценивать временную и пространственную сложность независимо от «железа».
- **Для формальной верификации:** используются для математического доказательства корректности программ и отсутствия в них критических ошибок (deadlocks).
- **При проектировании языков программирования:** определяют семантику языка (например, Си опирается на императивную).
- **В Model-Driven Engineering (MDE):** позволяют автоматически генерировать рабочий код из абстрактных логических схем и моделей.
- **Последовательные (императивные) модели:**
  - **Машина Тьюринга:** бесконечная лента и головка чтения/записи; служит для определения границ вычислимости.
- **Функциональные модели:**
  - **$\lambda$ -исчисление:** вычисление выглядит как последовательное применение функций к аргументам, где нет глобального состояния и переменных.
- **Параллельные модели:**
  - **Сети Петри:** ориентированный граф с фишками, моделирующий асинхронные процессы и конкурентный доступ к ресурсам.

---

## 55. Что такое последовательные модели вычислений? Приведите примеры. Как в них представляется вычислительный процесс?

**Последовательные модели вычислений** — это модели, в которых инструкции, события или действия выполняются **строго последовательно, одно за другим**.

Вычислительный процесс в них представляется как **пошаговое изменение глобального состояния** системы. Пока не завершится один шаг, следующий не начнется.

**Примеры:**

- **Конечный автомат (FSM):** самый простой пример. Задаётся набор состояний и правила перехода между ними. Система всегда находится строго в одном состоянии, а следующий шаг зависит от текущего состояния и входа.

- **Машина Тьюринга:** классический пошаговый автомат, который двигает головку по ленте и меняет символы шаг за шагом.
  - **Архитектура фон Неймана:** модель, по которой работают наши ПК. Процессор последовательно берет команды из памяти по счетчику (PC) и выполняет их одну за другой.
- 

## 56. Что такое машина Тьюринга и почему она важна для теории вычислений?

**Машина Тьюринга** — это абстрактная математическая модель компьютера, у которой есть бесконечная лента и управляющая головка. Головка умеет ходить влево-вправо, читать символ, заменять его на другой и менять внутреннее состояние машины.

В железе её построить нельзя из-за бесконечной ленты, но для теории она фундаментально **важна потому что**:

- **Задает эталон (Полнота по Тьюрингу):** считается, что любой алгоритм в мире, который вообще в принципе можно вычислить, можно реализовать на машине Тьюринга. Если язык программирования может то же, что и она, он называется Тьюринг-полным.
  - **Показывает границы возможного (Проблема остановки):** именно на ней Тьюринг доказал, что нельзя написать универсальную программу, которая бы со 100% гарантией заранее сказала, зависнет другая программа или завершится.
  - **Разделяет логику и данные:** данные лежат на ленте, а программа (таблица переходов) сидит в самой головке. Это заложило основу для разделения софта и харда.
- 

## 57. Что такое Random Access Machine? Каково её устройство и место сегодня? Какова её связь с машиной Тьюринга?

**Random Access Machine** — это модель, где вместо бесконечной ленты используется **память произвольного доступа** (набор адресуемых регистров). можно записать/прочитать данные в любую ячейку по её адресу.

**Связь с машиной Тьюринга:**

- **По мощности они равны:** RAM-машина не умеет вычислять то, что не может машина Тьюринга.
- **Но она гораздо быстрее и удобнее для анализа из-за прямого доступа:** в машине Тьюринга, чтобы прочесть элемент в конце ленты, нужно сделать миллион шагов. В RAM-машине это делается за 1 шаг ( $O(1)$ ). Поэтому сложность реальных алгоритмов оценивают именно на модели RAM.

идейный фундамент современных процессоров и ассемблера.

---

## 58. Что такое функциональные модели вычислений? Приведите примеры. Как в них представляется вычислительный процесс?

**Функциональные модели вычислений** — это модели, которые представляют вычисление не как пошаговые команды для процессора, а как **набор символов и правил их преобразования с помощью функций**. Здесь нет жесткого порядка шагов и понятия глобального состояния или переменной.

Вычислительный процесс описывается **декларативно**: мы пишем, *какой результат* хотим получить, а система преобразует выражения (через подстановку и сокращение), пока не дойдет до финального ответа.

**Примеры:**

- **λ-исчисление:** базис функционального программирования. Всё состоит из лямбда-выражений (переменные, константы, функции). Вычисление происходит через замену аргументов внутри функций.
  - **Язык Рефал:** советский язык программирования, основанный на распознавании образов и подстановке строк.
- 
- 

## 59. Что такое параллельные модели вычислений? Приведите примеры. Как в них представляется вычислительный процесс?

**Параллельные модели вычислений** — это модели, которые описывают систему, где одновременно существуют и взаимодействуют **несколько независимых процессов**.

Вычислительный процесс здесь — это **не линейная цепочка**, а дерево или граф из параллельных ветвей. Самое важное в них — не сами вычисления, а то, как потоки синхронизируются.

#### Примеры:

- **Модель Акторов (Actor Model):** каждый актор — это изолированный объект со своей памятью. Они не могут залезть в чужие данные и общаются только через отправку писем в «почтовые ящики» друг друга (как в Erlang).
  - **Многопоточная модель (Multi-Thread):** потоки выполняются одновременно, но у них есть **общая память**. Из-за этого приходится использовать семафоры-счетчики и ключи-мьютексы, атомарные опера<sup>4</sup> чтобы избежать гонок и конфликтов.
- 

## 60. Что такое Model-Driven Engineering (MDE)? Чем цепочка трансформации в MDE отличается от языков высокого уровня? Приведите примеры.

**Model-Driven Engineering (MDE)** — это подход к разработке, где главным объектом разработки является **абстрактная модель системы**, а не тонны написанного вручную кода. Код для конкретной платформы генерируется из этой модели автоматически.

#### Отличие от обычных языков (C++, Java):

В обычных языках программирования мы сразу пишем код, завязанный на платформу и архитектуру. В MDE появляется **дополнительный уровень абстракции**: мы сначала описываем что система делает на независимом от платформы языке, а транслятор сам решает, *как и где* это запустить (например, превращает модель в код для микроконтроллера или в Java-приложение).

#### Примеры:

- **Executable UML** : логика рисуется в виде диаграмм классов, которые можно запустить, протестировать, а потом скомпилировать в рабочий код на Си.
  - **DSL (Domain Specific Language):** специализированные языки под узкую задачу. Яркий пример — **SQL**: мы пишем `SELECT * FROM users` , описывая, *что* нам нужно, а СУБД сама строит план вычислений и лезет в файлы на диске.
-

## 61. Что такое универсальный процессор? Каковы его особенности и свойства?

**Универсальный процессор** — это вычислитель, способный решать огромный круг самых разных задач, причем его конкретная функция задается уже **после производства** с помощью программирования.

Здесь идея **двухэтапного производства**: сначала завод штампует миллионы одинаковых «пустых» чипов, а затем программист вшивает софт, превращая процессор в то что нам необходимо.

Такой процессор состоит из **4 базовых частей**: ввод, процессор, память и вывод.

**Главные особенности универсального процессора:**

- **Гибкая память:** у него изменяемое ПО, а размер программы ограничен только объемом внешнего диска или флешки, но не самим процессором.
- **Универсальность не Тьюринг-полнота:** теоретически процессор может быть Тьюринг-полным, но на практике не уметь эффективно работать с какими-то специфическими вещами (например, с графикой).
- **Плата за гибкость:** чем универсальнее платформа, тем она менее эффективна по сравнению с чипами, созданными строго под одну задачу.

**свойства:**

- Идеален **для двухэтапного производства**: сначала выпускаются процессоры а потом уже пишем по
- **Практически всегда Тьюринг-полный:** это дает процессору *теоретическую универсальность* — на нём можно запустить абсолютно любой алгоритм. но то что теоретически он универсален, не значит что на практике будет удобный.
- **Свобода от жестких лимитов:** размера программ, тк не ограничен «защитой» логикой процессора, а зависит только от емкости внешнего хранилища.

---

## 62. В чём заключается противоречие между универсальностью и эффективностью в разных видах процессоров (СБИС, FPGA, CGRA, GPU, DSP, CPU)?

Главный конфликт в архитектуре ЭВМ: **чем процессор универсальнее, тем он хуже и медленнее решает конкретную узкую задачу. И наоборот — чем сильнее чип заточен под задачу, тем он в ней эффективнее, но ничего другого делать не умеет.**

- **ASIC / СБИС** : чипы под одну задачу (например, чип в крипто-кошельке).  
**Эффективность максимальная**, но универсальность нулевая.
- **FPGA** : Внутри логические блоки, соединения между которыми можно перенастраивать софтом после производства. Очень эффективен ценой огромной площади кристалла и сложного проектирования.
- **CGRA**: в нем перестраиваются не транзисторы и вентили, а крупные блоки (АЛУ). Проще, чем FPGA, и идеально подходит для регулярных потоковых вычислений.
- **DSP** : жестко заточен под потоковую обработку сигналов (аудио, видео, радиоволны). Мастерски решает математику, но как универсальный процессор слаб.
- **GPU**: лучший для массово-параллельных вычислений. Состоит из тысяч простых ядер. Супер-эффективен, если нужно одновременно умножить миллион одинаковых чисел (что нужно для графики, нейросетей), но жестко проседает на сложном коде с кучей ветвлений ( `if/else` ).
- **CPU** : король **универсальности**. Умеет делать вообще всё, но из-за этого дико **неэффективен** по транзисторам и энергии. Большая часть его кристалла занята не вычислениями, а сложными схемами предсказания переходов, кэшами и декодерами команд, которые просто управляют этой универсальностью.

## 63. Как соотносятся вычислительные платформы и языки описания вычислительного процесса? Как это связано с эффективностью и областью применения?

прямая: язык, на котором мы пишем, всегда привязан к абстракциям той платформы, где код будет выполняться.

Если представить это как спектр от универсального (гибкого) к специализированному (эффективному):

- **Слева (C - RISC )**: пишем на обычном Си под риск процессор. Область применения — огромная, гибкость — максимальная. **эффективность ниже**, потому что код оторван от реального железа.
- **Посередине (HDL/C - FPGA/ASIP, C/ASM - DSP)**: - для FPGA и ASIP используются языки описания аппаратуры или специализированный Си. Для сигнальных процессоров (DSP) пишется код на Си или чистом Ассемблере (ASM). область применения сужается, но эффективность заметно растет.
- **Справа (HDL - ASIC)**: пишем на HDL строго под конкретную микросхему, которую отольют на заводе. код не запустится больше нигде, зато **аппаратная эффективность — максимальная**.

чем дальше язык от железа, тем проще переносить программу, но хуже оптимизация. Чем ближе язык к аппаратуре, тем выше скорость и меньше жор батареек, но код становится «одноразовым» — только под эту задачу.

---

## 64. Что такое архитектура и микроархитектура процессора? В чём различие между ними?

- **Архитектура (ISA)** — это то, как процессор выглядит для программиста. Это набор доступных команд (инструкций), видимые регистры, правила работы с памятью и прерываниями. «**ЧТО процессор умеет делать?**»
- **Микроархитектура** — это конкретная **внутренняя реализация** архитектуры. То, как физически соединены транзисторы, АЛУ, конвейеры и кэши на кристалле, чтобы эти команды выполнялись. «**КАК процессор делает внутри?**».

Для одной архитектуры (например, **x86** или **ARM**) - сотни разных микроархитектур. Архитектура у них одна, а вот микроархитектура (кэши, частоты, количество ядер) — разная.

---

## 65. Что такое система команд и какова её роль в архитектуре процессоров? Что определяет система команд?

**Система команд (ISA)** — это контракт между софтом и хардом. объясняет компилятору и программисту, как управлять этим процессором.

**ISA определяет:**

- **Набор инструкций:** которые процессор вообще понимает (сложить, умножить, прыгнуть по адресу).
- **Типы данных:** с которыми он умеет работать (целые, с плавающей точкой, разрядность регистров).
- **механизмы работы** с памятью, прерываниями, исключениями и вводом-выводом.

**Что ISA НЕ определяет:**

Она вообще ничего не говорит про физику: какое будет энергопотребление, сколько тактов займет команда `ADD` и какая будет частота чипа. Это уже забота микроархитектуры.

---

## 66. Что такое машина фон Неймана? Какова её связь с машиной Тьюринга? Каковы её ключевые принципы?

**Машина фон Неймана** — это идейное продолжение машины Тьюринга, но перенесенное на реальное железо. Вместо бесконечной ленты здесь появляется **Random Access Memory** а вычисления выполняются по шагам.

**Ключевые принципы:**

1. **Двоичное кодирование:** и данные, и команды кодируются нулями и единицами.
2. **Программное управление:** процессор выполняет программу автоматически, без участия человека.
3. **Общая память (Принцип однородности):** программы и данные лежат в одной и той же памяти. Процессор одинаково может прочитать число или команду.
4. **Адресуемость памяти:** память разбита на ячейки, у каждой есть свой уникальный адрес.
5. **Условные переходы:** программа может менять порядок выполнения команд в зависимости от условий (появляются циклы и `if/else`).

Также вводится понятие **машинного слова** — это базовый размер данных (например, 32 или 64 бита), с которым процессору и его регистрам удобнее всего работать за один раз.

---

## 67. Что такое микропроцессор и микроконтроллер? В чём их отличия?

- **Микропроцессор (CPU):** это **чисто вычислительное ядро**. Его задача — быстро обрабатывать информацию и гонять её в оперативку. Чтобы он заработал, к нему на плату нужно распаять кучу внешней периферии: оперативка, память, контроллеры дисков и видеокарту.
- **Микроконтроллер (MCU):** это **целый компьютер на одном чипе**. Внутри одного крошечного корпуса упакованы и процессорное ядро, и память, и таймеры, и порты ввода-вывода (GPIO).

**На пальцах:** Микропроцессор нужен, чтобы мощно считать (серверы, ПК), а микроконтроллер — чтобы управлять железками и датчиками напрямую (умный дом, робототехника, стиральные машины).

---

## 68. Что такое Control Unit и Data Path? Каковы их назначение и принципы взаимодействия?

Процессор логически можно разделить на **Control Unit** и **Data Path**.

- **Data Path (Тракт данных):** это **рабочие руки процессора**. Сюда входят регистры, АЛУ (арифметико-логическое устройство) и внутренние шины. Эта часть физически хранит, двигает, складывает и умножает байтики данных.
- **Control Unit (Устройство управления):** это **командир процессора**. Он берёт команду из памяти, расшифровывает её и начинает посылать управляющие сигналы в Data Path.

**Взаимодействие:** Data Path делает всю грязную работу с числами, но только как ему прикажет Control Unit.

---

## 69. Какие виды инструкций существуют в машине фон Неймана? Приведите примеры. Каковы принципы кодирования?

Все команды процессора можно разбить на **4 основные группы**:

- **Работа с памятью:** загрузить данные в регистр ( **LOAD** ) или сохранить их в память ( **STORE** ).
- **Арифметика и логика:** сложить ( **ADD** ), побитовое И ( **AND** ), сдвиги и т.д.
- **Управляющие:** прыжки по коду ( **JUMP** ), вызовы функций ( **CALL** ) и возвраты ( **RET** ). Они меняют счетчик команд (PC).
- **Команды для сопроцессоров:** например, передать тяжелую математику на графический или плавающий чип.

**Принципы кодирования:**

Каждая инструкция — это двоичное машинное слово, поделенное на поля.

1. **Orcode (Код операции):** уникальный код команды, который говорит, *что именно* нужно сделать ( прибавить число ).
  2. **Адреса операндов (Addr 1 / Addr 2):** номера регистров, с которыми работаем (куда читать и куда писать).
  3. **Immediate value (Непосредственное значение):** сама константа (число 350), зашитая прямо в тело команды.
-

## 70. Каковы особенности принстонской архитектуры? Каковы её достоинства и недостатки? Какова область применения?

Принстонская архитектура — это и есть классическая архитектура фон Неймана.

Её главная фишка: и данные, и инструкции лежат в одной общей памяти и передаются процессору по одному общему каналу (шине).

Особенности:

- **Кодирование:** База двоичная, но бывает троичное или двоично-десятичное (для точных финансовых расчетов).
  - **Адреса:** Адрес ячейки — это **не просто число**. Из-за виртуальной памяти он проходит через сложные таблицы трансляции. Плюс память давно **не пассивна** — она сама регенерирует данные и умеет делать простейшие операции внутри себя (PIM).
  - **Линейность выполнения с поправкой на переходы:** Программа прыгает по коду не только из-за `if/else`, но и по прерываниям от железа, системных вызовов или ошибок деления на ноль.
  - **Плюсы:** максимально простая и дешевая в реализации.
    - одна память, общие провода и единая адресация.
  - **Минусы:** Возникает **«УЗКОЕ горлышко»**. Поскольку из-за того что шина ИО одна, процессор не может одновременно читать следующую команду и загружать данные из памяти для текущей команды.
  - **Применение:** Используется везде, где важна гибкость, универсальность и простота — это основа архитектуры большинства современных универсальных процессоров (ПК, смартфоны, серверы), хотя на уровне кэшей там идут свои хитрости.
- 

## 71. Каковы особенности гарвардской архитектуры? Каковы её достоинства и недостатки? Какова область применения?

Гарвардская архитектура — архитектура, в которой память команд и память данных физически разделены и имеют свои собственные независимые шины (каналы).

Достоинства:

- Процессор может в один и тот же такт читать следующую инструкцию и загружать/сохранять данные. горлышко фон Неймана исчезает.

- **Разная ширина данных и команд:** Разрядность ячеек для команд и данных может быть разной (например, команды по 16 бит, а данные по 8 бит).
- Программа физически изолирована от данных. Она **не может сама себя случайно или намеренно изменить/переписать**. Защита от кучи уязвимостей (вроде переполнения буфера) на аппаратном уровне.

Недостатки:

- **Сложность и стоимость:** Нужно разводить на плате в два раза больше проводов и шин
- **Проблема с компилятором:** Поскольку память команд закрыта, возникают жесткие для него:
  - Компилятор — это обычная программа, которая генерирует код и пишет его в сектор *данных*. Но приходится городить сложные костыли, тк у него нет прямого доступа из данных в память инструкций
  - В Гарвардской архитектуре обычный указатель на данные не умеет ссылаться на код, что сильно усложняет жизнь разработчикам компиляторов.

---

## 72. Как организовано адресное пространство в гарвардской и принстонской архитектурах?

В **принстонской архитектуре** инструкции и данные находятся в **едином адресном пространстве**: один контроллер памяти, один канал доступа, одна общая память.

В **гарвардской архитектуре** память команд и данных логически разделена, поэтому существуют **два независимых пространства и два канала доступа**. Это позволяет одновременно читать инструкцию и данные и даже использовать разную разрядность памяти команд и памяти данных.

Главный эффект такого разделения — параллельный доступ и меньшая конкуренция за память. В принстонской архитектуре этот параллелизм отсутствует, поскольку и инструкции, и операнды используют один канал.

---

## 73. Какие существуют варианты обхода ограничений гарвардской архитектуры, включая "Модифицированную гарвардскую архитектуру"?

Чтобы решить проблему компилятора и позволить программам генерировать и запускать код на ходу, используют три основных подхода:

- **Память инструкций как данные (Instruction-memory-as-data):** Процессору добавляют специальную лазейку, позволяющую читать и писать напрямую в память программ, как если бы это были обычные данные.
    - *Минус:* Падает безопасность, программа снова может затереть саму себя.
  - **Память данных как инструкции (Data-memory-as-instruction):** Процессору разрешают прыгать по коду прямо в сектор данных и выполнять его оттуда.
    - *Минус:* Теряется преимущество в скорости. Если код выполняется из общей памяти данных, шина забивается, и мы возвращаемся к проблеме одного канала.
  - **Модифицированная Гарвардская архитектура (Mainstream):** Самый умный и полезный вариант.
    - **Как устроена:** Снаружи (на уровне ОЗУ и общей шины платы) память одна, всё дешево и просто. Но внутри процессора стоят **кэши инструкций и данных первого уровня (L1)**
    - **Профит:** Процессор одновременно и параллельно забирает команды из одного кэша, а данные из другого (Гарвард на максималках). При этом компилятору и ОС жить легко, ведь общая память едина.
- 

## 74. Что такое "аккумуляторная система команд"? Каковы основные элементы? Приведите примеры. Каковы достоинства и недостатки?

**Аккумуляторная система команд** — это архитектура, где вся арифметика крутится вокруг одного главного регистра — **аккумулятора**. ранние процессоры вроде **Intel 8008**

в командах указывается только *один* операнд, а вторым **неявно** всегда выступает аккумулятор. Туда же записывается и результат операции.

`LOAD A` (загрузить A в аккумулятор), `ADD B` (прибавить B к аккумулятору)

Плюсы:

- Супер-простая логика процессора и дешевое железо.
- Короткие команды (всего один адрес в инструкции), код занимает мало места.

Минусы:

- Аккумулятор — это **дикое бутылочное горлышко**. Постоянно приходится жонглировать данными: загружать что-то из памяти в Асс, считать один шаг,

выгружать обратно, чтобы освободить место для следующего вычисления. Программа раздувается от лишних пересылок.

---

## 75. Что такое Register-to-Memory системы команд? Каковы основные элементы? Приведите примеры. Каковы их достоинства и недостатки?

**Register-to-Memory** — это архитектура, где арифметические команды могут выполнять инструкции с данными, когда один операнд в **регистре**, а второй — в **памяти**. регистров теперь много в отличие от аккумулятора. **Классическая архитектура x86**, которая относится к классу **CISC / Motorola 68000 (m68k)**

- **Пример:** Команда `ADD R1, [1000]` берет значение из регистра R1, прибавляет к нему число из ячейки памяти по адресу 1000 и сохраняет результат в R1.

Плюсы:

- Код компактный и удобный для программиста (не нужно делать лишний `LOAD` перед каждым сложением).
- Гораздо меньше холостых обращений к памяти, чем в аккумуляторной схеме.

Минусы:

- Команды становятся **сложными и тяжелыми для декодера**. Если операнд может лежать в памяти, процессору нужно поддерживать режимы адресации. Из-за этого команды получаются разной длины, а время их выполнения становится непредсказуемым.
- 

## 76. Что такое Register-to-Register системы команд? Каковы основные элементы? Приведите примеры. Каковы их достоинства и недостатки?

**Register-to-Register** — это база для **RISC-процессоров**. Главное правило: прямой доступ к памяти для вычислений строго запрещен. современные **RISC-архитектуры** — **ARM** (в смартфонах и макбуках)

доступ к памяти реализуется через две отдельные команды: `Load` (скачать в регистр) и `Store` (загрузить на диск).

- Чтобы сложить данные из памяти, мы обязаны сначала скачать данные, только потом выполнить чистую регистровую команду `ADD R3, R1, R2` и, если надо, отправить обратно через `STORE [C], R3`. Примеры: **MIPS, ARM, RISC-V**.

Плюсы:

- Инструкции простые, фиксированной длины, декодеру проще обрабатывать.
- Идеально для **конвейера и параллелизма**, так как время выполнения всех регистровых команд одинаковое.

Минусы:

- Писать код вручную — то еще удовольствие. Для банального действия требуется выдать пачку из 3-4 отдельных команд, из-за чего текст программы кажется длиннее.

---

## 77. Что такое Memory-to-Memory системы команд? Каковы основные элементы? Приведите примеры. Каковы их достоинства и недостатки?

**Memory-to-Memory** — это архитектура, где операции выполняются **напрямую над ячейками памяти**, и не нужно думать о регистрах. нейропроцессоры, где нужно прокачивать терабайты потоковых данных мимо регистров.

Плюсы:

- Максимально красивый и понятный Ассемблер. Код пишется легко, ведь он отражает чистую математику алгоритма без возни с регистрами.

Минусы:

- **Катастрофически низкая производительность**. Память работает в сотни раз медленнее процессора. Чтобы выполнить всего *одну* такую команду, процессору нужно трижды сходить во внешнюю память (два раза прочесть, один раз записать). Железо получается дико сложным, а кэши постоянно страдают. В современном мире этот подход полностью вытеснен.

---

## 78. Что такое CISC процессоры? Каковы их ключевые особенности, достоинства и недостатки?

**CISC (Complex Instruction Set Computer)** — это архитектура процессоров со **сложным набором команд**. Главная идея: одна инструкция умеет делать много разнородных действий за раз.

Причины появления:

- программировали вручную на низкоуровневых языках. Сложные команды были банально **удобнее для человека**.
- **Слабые компиляторы**: переводить сложный код в простые команды компиляторы тогда еще толком не умели. давайте сделаем одну большую и сложную команду прямо в процессоре.
- **Дорогая и медленная память**: нужно было из всех сил **минимизировать объем программы** и накладные расходы на чтение кода из ОЗУ. Одна длинная сложная команда занимала меньше места, чем пачка простых.

Плюсы:

- компактный машинный код.
- Мощные инструкции, которые заменяют целые куски алгоритмов.

Проблемы:

- **Дико сложный Control Unit и декодер**: процессору приходится на ходу расшифровывать команды, которые имеют **переменную длину** (variable length) и могут делать всё что угодно.
- **Ад для компилятора**: компиляторам очень трудно автоматически генерировать эффективный код под такое разнообразие специфических команд.
- **Тяжело анализировать и оптимизировать**: из-за разного времени выполнения команд процессор сложнее разгонять по частоте.

---

## 79. Какие виды адресации аргументов инструкции в CISC процессорах?

В CISC-процессорах (например, в классическом **Motorola 68000 / m68k**) способов найти данные в памяти огромное количество.

**Основные режимы адресации «на пальцах»:**

- **Прямая регистровая**: данные лежат прямо в регистре (никуда ходить не надо).
- **Непосредственная (Immediate)**: число зашито прямо в тело команды (константа).
- **Косвенная**: в регистре лежит адрес, по которому надо сходить в память.

- **Косвенная с постинкрементом / преддекрементом:** процессор берет адрес из регистра, идет в память, а сам регистр автоматически увеличивает (или уменьшает) на единицу. для перебора массивов в цикле.
  - **Косвенная со смещением:** адрес в памяти равен: значение регистра + фиксированное число .
  - **Косвенная с индексом и смещением (Самая комплексная):** процессор на ходу складывает берем значение по адресу из регистра + индексный регистр + смещение и только потом идет по полученному адресу в ОЗУ.
- 

## 80. В каких случаях CISC инструкции эффективны? Приведите примеры.

CISC-инструкции эффективны за счет того, что они достаточно узкоспециализированы, каждая из них делает очень конкретную задачу, но с очень узкой областью применения.

- **Пример из m68k:** Команда может делать сразу кучу работы за один присест. брать адрес из регистра, прибавлять к нему индекс из другого рега, прибавлять смещение, лезть по результирующему адресу, забирает оттуда данные и копирует в регистр D0 .

### Сравнение:

- **В CISC** это одна сложная инструкция.
- **В RISC** эту же задачу пришлось бы решать последовательностью из 4–5 простых инструкций фиксированной длины (сначала умножить индекс, сложить с базой, прибавить константу, сделать LOAD из памяти).

**Цена такой эффективности:** узкая специализация. Если алгоритм программы чуть-чуть изменится, эта сложная инструкция станет бесполезной, а транзисторы на её поддержку в чипе всё равно будут тратить энергию.

---

## 81. В чём выражается комплексность систем команд CISC?

Комплексность в том, что сложность переносится из программного кода (software) внутрь самого железа (hardware).

Вместо того чтобы заставлять программиста или компилятор собирать сложную операцию из мелких кирпичиков, CISC делает эту операцию атомарной.

То есть вся логика по расчету сложно-результативных адресов, чтению промежуточных данных и их обработке зашивается прямо в **Control Unit, декодер и внутреннюю микропрограмму процессора**. Программа на диске становится короче и красивее, но сам процессор сложнее

---

## 82. Какие существуют подходы к реализации Control Unit? Достоинства и недостатки Hardwired и микропрограммного управления?

Control Unit можно сделать двумя принципиально разными способами: «в железе» или «на софте».

- **Hardwired Control Unit (Схемная логика):** Управляющие сигналы для дата паф выдает жестко спаянная комбинационная схема из логических вентилей.
    - **Плюсы:** Работает на **максимальной скорости**. Никаких лишних прослоек.
    - **Минусы:** Схема получается огромной и адски сложной, если у нас CISC-процессор с кучей команд. Изменить или исправить ошибку после производства чипа невозможно.
  - **Microcoded Control Unit (Микропрограммное управление):** Внутри процессора прячется еще один маленький «микрокомпьютер». Каждая сложная команда превращается в цепочку простейших внутренних команд — микрокода.
    - **Плюсы:** Намного проще проектировать сложные ISA (CISC). Поведение процессора можно менять и обновлять.
    - **Минусы:** Требуется места на кристалле под хранение микрокода. Работает медленнее, так как появляется дополнительный шаг трансляции.
- 

## 83. Что такое микропрограммное управление? Что такое микроинструкция? В чём отличия инструкций и микроинструкций?

**Микропрограммное управление** — подход, когда выполнение большой архитектурной команды разбивается на последовательность мелких инструкций.

**Микроинструкция** — это низкий уровень управления железом. Она открывает или закрывает конкретные вентили и шины в Data Path ( «открыть затвор регистра R1 и подать ток на левый вход АЛУ»).

**Разница «на пальцах»:**

- **Инструкция (уровень ISA):** Видима программисту. Говорит процессору, *ЧТО* сделать в рамках прикладной задачи (например, `ADD R1, R2` ).
  - **Микроинструкция (внутренний уровень):** Скрыта внутри чипа.
  - Показывает, *КАК* именно это физически сделать железу за один такт. Одна инструкция может состоять из десятка микроинструкций.
- 

## 84. Как можно оптимизировать инструкции при помощи микропрограммирования? Приведите пример.

Микропрограммирование дает гибкость: мы можем оптимизировать и менять логику процессора, просто **переписывая микрокод**, без переделки транзисторов.

Достоинства:

- **Простота реализации CISC:** Сложные команды собираются как конструктор из готовых микроинструкций.
- **«Программирование» системы команд:** Можно делать кастомную ISA под конкретную задачу, чтобы сократить объем кода и поднять эффективность. **Яркий пример — процессор «Самсон»**, разработанный в ЛИТМО
- **Доступ к микрокоду:** В некоторых машинах программист может сам дописать свои микрокоманды, если ему не хватает стандартных.
- **Ремонтопригодность:** Позволяет исправлять баги в железе и закрывать уязвимости (как Spectre/Meltdown) уже после выпуска процессора, просто обновив его микрокод.

Недостатки :

- Микрокод нужно где-то хранить в процессоре (тратится место на чипе).
  - Слишком много форматов, размеров и длительностей команд усложняют как сам процессор, так и разработку компиляторов.
  - Микрокод приносит с собой **все проблемы обычного программирования**: в нем тоже бывают баги, его тяжело отлаживать и тестировать.
- 

## 85. Какие существуют подходы к поиску первой микроинструкции для заданной инструкции?

- **Алгоритмический поиск:** Процессор не прыгает сразу по адресу, а запускает стартовую микропрограмму. Она пошагово анализирует код операции (opcode) через

условия и ветвления, пока не дойдет до нужной ветки. Такой принцип «программного» декодирования используется, например, в **БЭВМ** (Базовой ЭВМ).

- **Отображение (Mapping) opcode на адрес микрокода:** Тут адрес находится аппаратно и мгновенно. Бывает двух видов:
    - **Прямое отображение:** Берутся биты самого кода операции (opcode) и напрямую используются как адрес в памяти микрокода (или дополняются нулями).
    - **Непрямое отображение:** Используется встроенная аппаратная таблица соответствия (Look-up table). Процессор подставляет туда opcode, а на выходе сразу получает готовый адрес первой микрокоманды.
- 

## 86. Что такое архитектура NISC? Каковы область её применения, достоинства и недостатки?

**NISC (No Instruction Set Computer)** — это архитектура, в которой **вообще нет привычной системы команд (ISA)**. Компилятор не переводит код в команды вроде `ADD` или `MOV`, а сразу генерирует «длинные» управляющие слова, которые напрямую открывают и закрывают вентили конкретных функциональных блоков процессора.

По сути, это микропрограммное управление, где компилятор взял на себя роль декодера команд.

Достоинства :

- Процессору больше не нужны декодеры команд и сложные блоки управления — он просто выполняет готовые управляющие сигналы.
- Компилятор идеально знает железо и задействует блоки на 100%.
- **Нет ISA — нет проблем её проектирования:** Не нужно придумывать, стандартизировать и поддерживать систему команд.

Недостатки (из слайда):

- Код намертво привязан к конкретной схеме. Если на заводе изменят положение хотя бы одного блока или шины, старый скомпилированный код превратится в тыкву и процессор сломается.
- Управляющие слова получаются огромными по размеру (много бит), код занимает много памяти.

Применение:

- Специализированные ускорители

- **Проект NITTA** — это реальный пример CGRA-процессора. Транслятор сам раскидывает управляющие слова по блокам напрямую, минуя уровень ISA.
- 

## 87. Что такое стековые системы команд? Каковы основные элементы? Что такое неявная адресация аргументов? Каковы их достоинства и недостатки?

**Стековая система команд** — это сискоманд, где главным местом для хранения данных и вычислений является **стек**, а не куча регистров общего назначения.

**Неявная адресация аргументов** — в командах вообще не указываются адреса или номера регистров. Процессор по умолчанию знает: *все данные нужно брать с вершины стека*.

- **Пример:** Команда `ADD` не имеет параметров. Она просто молча снимает два верхних числа со стека, складывает их и кидает результат обратно на вершину стека.

Плюсы:

- Команды получаются супер-компактными (в них нет адресов, только код операции — `opcode`).
- Идеально и естественно ложится на вызовы процедур (функций) и математические выражения.

Минусы:

- Неудобно работать с данными, которые закопаны глубоко в стеке. Приходится постоянно делать холостые операции, чтобы их оттуда достать.
  - **Ужасно для параллелизма (ILP):** Поскольку все команды завязаны на одну точку — вершину стека, процессору крайне тяжело выполнять команды параллельно или менять их местами.
- 

## 88. Какие виды стеков есть в стековых процессорах? Их количества?

может быть как один стек, так и **несколько независимых стеков** (обычно два), чтобы разделять задачи. поддерживать параллелизм и поддерживать процедуры

**По количеству и назначению (классическая схема):**

- **Стек данных (Data Stack):** Здесь лежат операнды и крутится вся арифметика.
- **Стек возвратов (Return Stack):** Хранит адреса возврата из функций и локальные переменные, чтобы они не мешались под руками у математических команд.

По аппаратной реализации выделяют 2:

- **Стек в памяти:** Самый простой и дешевый вариант. Стек лежит в обычной ОЗУ, а в процессоре есть Stack Pointer .
- **Аппаратный стек (на регистрах):** Верхние элементы стека (например, первые 4 или 8 ячеек) сделаны в виде сверхбыстрых связанных регистров прямо внутри ядра. Это позволяет процессору щелкать операции со стеком мгновенно, за 1 такт.

---

## 89. Каковы основные принципы описания алгоритмов для стековых процессоров? Что такое Forth и High-level language computer?

Для стекового процессора алгоритм описывается через операции над стеком: операнды сначала помещаются на стек, затем команда берёт нужное количество верхних значений и кладёт результат обратно.

**Forth** — императивный язык программирования, естественно построенный вокруг стека. Он хорошо компилируется под стековые архитектуры и удобно поддерживает процедуры и рекурсию. В нём всё пишется через постфиксную нотацию и операции со стеком ( DUP , DROP , SWAP ). Он идеально компилируется в машинный код стековых процессоров практически без потерь эффективности.

**High-level language computer** — процессоры, чья архитектура непосредственно поддерживает конструкции языка высокого уровня. Стековый процессор подходит под это определение, потому что вызовы функций реализуются очень естественно: аргументы кладутся на стек, функция их снимает, выполняет работу и возвращает результат через стек.

---

## 90. Каковы источники роста производительности процессоров? (и алгоритмов, в целом эффективности решения задач)

Основные источники:

- Самый очевидный путь — делать больше тактов в секунду (выполнять команды быстрее).
  - Замена по костылей аппаратными блоками.
    - Гораздо быстрее поставить на чип выделенный *аппаратный умножитель*. чем считать умножение через сдвиги и сложения на обычном АЛУ
  - Создание сложных CISC-команд, которые за один присест делают тяжелую работу, экономя время на чтении кода из памяти.
  - **Параллелизм на всех 3 уровнях:**
    - *Уровень бит*: Переход от 8-битных процессоров к 32- и 64-битным (обрабатываем больше данных за один такт).
    - *Уровень инструкций (ILP)*: Использование конвейеров (pipelines) и суперскалярности.
    - *Уровень задач (TLP)*: Многоядерность и многопоточность.
  - Отказ от универсальности ради эффективности (как в GPU, DSP или ASIC).
  - Перепрошивка управляющей логики чипа под конкретный алгоритм как в плис
- 

## 91. Какие законы и ограничения влияют на производительность современных процессоров?

- **Закон Мура**: Количество транзисторов удваивалось  $\times 2$  каждые 1.5–2 года. и соответственно производительность - конец в 2010 годах
  - **Закон Деннарда**: при уменьшении транзисторов (и соответственно увеличении производительности) падает их энергопотребление, что позволяло бесплатно наращивать частоту. работает до до некоторого предела. уже упираемся в физические пределы и возможности производства
  - **Закон Амдала**: увеличивая параллелизм вдвое надо увеличить площадь  $\times 2$  - накладно. с каждым шагом увеличения параллелизма результат времени будет меньше (100-50-25)
  - **Power Wall / Тёмный кремний**: если мы в процессоре повышаем плотность транзисторов, то в какой-то момент мы не сможем больше отводить от него тепло, его будет выделяться слишком много.
  - **Memory Wall**: Процессоры стали слишком быстрыми, а оперативка осталась медленной. Процессор постоянно простаивает в ожидании данных. в помощь кэши.
-

## 92. Что представляют собой закон Мура и закон Деннарда?

**Закон Мура** — эмпирическое наблюдение: количество транзисторов в процессорах примерно удваивается за полтора года, вместе с чем исторически росла и производительность. перестал работать примерно к 2010 году.

**Закон Деннарда** связывает уменьшение размера транзисторов с ростом эффективности: меньший транзистор требует меньше энергии, сигналы проходят меньшие расстояния, поэтому можно повышать частоту и плотность вычислений.

Проблема в том, что масштабирование имеет физический предел. Когда транзисторы становятся очень маленькими, дальнейшее уменьшение уже не даёт прежнего выигрыша по энергопотреблению и частоте и упирается в технологические ограничения.

---

## 93. Что представляет собой закон Амдала? На каком уровне параллелизма он работает?

**Закон Амдала** показывает, что ускорение программы ограничено её непараллелизуемой частью.

Даже если параллельную часть выполнять на всё большем числе вычислителей, выигрыш быстро уменьшается. Например, если только половина программы может быть распараллелена, то даже при бесконечном количестве ресурсов общее ускорение не превысит двух раз. При 95% параллелизуемой части предел — около 20 раз.

Кроме того, увеличение параллелизма требует дополнительных аппаратных ресурсов: чтобы выполнять больше работы одновременно, нужны дополнительные вычислительные блоки.

В исходном материале этот закон связывается прежде всего с **параллелизмом уровня инструкций** — насколько поток инструкций одной прикладной задачи удаётся исполнять параллельно.

---

## 94. Что такое Power Wall и тёмный кремний?

**Power Wall** — ограничение роста производительности из-за энергопотребления и тепловыделения. При повышении частоты тактов потребление и выделение тепла быстро

растут, поэтому наступает предел, после которого чип уже невозможно нормально охлаждать.

**Тёмный кремний** — следствие похожего ограничения. Даже если технологически можно разместить на кристалле больше транзисторов, то суммарное тепловыделение станет слишком большим при одновременном включении.

Поэтому часть физически существующих вычислительных блоков должна оставаться выключенной или использоваться не одновременно. Ограничение - не только количество транзисторов, но и допустимая мощность на единицу площади.

---

## 95. Какие подходы существуют к решению проблемы **Memory Wall** в рамках процессоров семейства фон Неймана?

**Memory Wall** — ситуация, когда скорость процессора растёт значительно быстрее скорости памяти. Процессор способен вычислять быстрее, чем память успевает поставлять ему данные, и начинает простаивать.

В рамках фон-неймановских процессоров проблему можно только смягчать:

- использовать **кэши**, чтобы часто нужные данные находились ближе к ядру;
- увеличивать число каналов доступа к памяти;
- применять гарвардские или модифицированные гарвардские элементы, чтобы чтение инструкций и данных меньше конфликтовало между собой.

Главная идея — уменьшить число обращений к медленной основной памяти и повысить пропускную способность пути между памятью и вычислительными блоками.

---

## 96. Что такое RISC? Каковы особенности и предпосылки появления (ПО и аппаратура)?

**RISC — Reduced Instruction Set Computer.** Это подход, противопоставленный CISC: вместо большого количества сложных команд используется небольшой набор **простых базовых инструкций**.

Особенности:

- Можно ускорить и оптимизировать каждую инструкцию

- Появляется параллелизм уровня инструкций
- Место под микрокоманды и декодер можно пустить на более полезные задачи в духе кэша или регистров

Предпосылки:

- сложно компилировать код для CISC процессоров. Раньше люди писали на ассемблере и CISC для них было просто удобнее, но сейчас ассемблер генерируется, и проще генерировать под RISC
  - простые инструкции дают простой декодер и единый формат.
- 

## 97. Как сравниваются CISC и RISC с точки зрения архитектуры?

С точки зрения **архитектуры**, CISC предоставляет программисту более мощные и сложные инструкции, а RISC — небольшой набор более простых операций.

Для CISC характерно:

- сложные инструкции, которые могут выполнять несколько действий;
- операции типа **memory-to-memory** или работа арифметики напрямую с памятью;
- много режимов адресации;
- переменная длина инструкций;
- более сложный и менее единообразный формат.

Для RISC характерно:

- строгий принцип **Load/Store**: память обслуживают отдельные команды, вычисления идут между регистрами;
- фиксированная длина и более единый формат инструкций;
- обычно более простая модель для компилятора.

То есть CISC переносит больше функциональности в отдельную инструкцию, RISC — разбивает её на простые шаги.

---

## 98. Как сравниваются CISC и RISC с точки зрения микроархитектуры?

С точки зрения **микроархитектуры** RISC обычно проще CISC.

В RISC:

- декодер проще, потому что инструкции имеют регулярный формат;
- **Control Unit** проще оптимизируется;
- Control Unit проще реализовать как hardwired-комбинационную логику вместо сложного микрокодного управления;
- проще организовать **конвейеризацию**, потому что операции более однотипны по структуре и времени исполнения.

В CISC сложность - внутри процессора: сложный декодер, большое число режимов адресации и часто микропрограммная реализация сложных инструкций.

---

## 99. Каковы особенности кодирования инструкций при сравнении CISC и RISC?

В **CISC** инструкции могут иметь **разную длину** и множество вариантов формата в зависимости от конкретной операции и режима адресации; в одну инструкцию можно встроить сложный способ вычисления адреса, например индекс и смещение.

В **RISC** формат более строгий и единый:

- длина инструкций обычно фиксированная;
- меньше режимов адресации;
- поля расположены более регулярно.

На схеме исходника это показано буквально: CISC-команды имеют **variable length**, а RISC-команды — **fixed length**.

Плюс RISC — проще определить границы следующей инструкции, декодировать её и подавать в конвейер. CISC даёт более плотный и выразительный код, но усложняет аппаратное декодирование.

---

## 100. Каковы принципы конвейерного исполнения инструкций? Как это влияет на загрузку и производительность?

**Конвейеризация** делит выполнение инструкции на несколько стадий, которые используют разные части процессора.

**Fetch → Decode → Execute → Write.**

Без конвейера вторая инструкция Fetchится только после полного завершения первой.  
С конвейером - пока первая инструкция декодируется, вторая уже загружается;

конечно важно, чтобы одна инструкция не зависела от результата другой иначе все ломается

Рост производительности получается за счёт лучшей загрузки аппаратных блоков и параллельной обработки разных стадий разных инструкций.

---

## 101. Каковы типовые стадии RISC процессора?

У RISC-процессора выделяют пять стадий выполнения инструкции:

1. **Instruction Fetch (IF)** — процессор по PC считывает инструкцию из памяти, идет по этому адресу во внутреннюю память (кэш инструкций L1i) и скачивает оттуда 32-битную команду, pc +4 байта чтоб смотреть на след инстр.
  2. **Instruction Decode (ID)** — декодирует инструкцию и читает нужные регистры.
  3. **Execute (EX)** — выполняет операцию в АЛУ: арифметику, сравнение, вычисление адреса и т.д. если прыжок или условие то чекаем флаг и проверка адреса куда.
  4. **Memory Access (MEM)** — если инструкции нужна память, происходит чтение или запись.
  5. **Write Back (WB)** — результат записывается обратно в регистр.
- 

## 102. Какие проблемы и архитектурные ограничения связаны с конвейеризацией?

стадии должны быть максимально **независимы друг от друга**. На практике это нарушается, потому что инструкции используют общие ресурсы и зависят друг от друга.

из-за этого:

1. **Структурный конфликт** — две стадии одновременно хотят один и тот же аппаратный ресурс. Например, одну память с одним портом.
2. **Конфликт по данным** — следующей инструкции уже нужен результат предыдущей, а та ещё не успела его получить или записать.
3. **Конфликт по управлению** — при условном переходе заранее неизвестно, какую инструкцию загружать следующей надо ждать.

Все эти конфликты приводят к остановкам, пузырькам, сбросам конвейера и в итоге уменьшают его реальную производительность.

---

## 103. Как разрешаются структурные конфликты в конвейере?

**Структурный конфликт**, когда две стадии конвейера в один момент требуют один и тот же аппаратный ресурс.

Есть два основных способа решения:

1. **Увеличить количество ресурсов.** Например, юзать двухпортовую память, чтобы два обращения могли выполняться одновременно.
2. **Разделить ресурсы.** инструкции и данные лежат в разных памятьях. Тогда `Instruction Fetch` не конфликтует с чтением или записью данных.

Если конфликт аппаратно убрать нельзя, одну из операций временно задерживают: в конвейер вставляется **ПУЗЫРЕК**, то есть пустой такт. Это проще, но снижает производительность, поэтому архитектурно лучше по возможности сделать конфликтующие стадии независимыми.

---

## 104. Как разрешаются конфликты по данным в конвейере?

**Конфликт по данным**, когда одна инструкция зависит от результата другой, но из-за параллельного выполнения этот результат ещё не готов.

Основной конфликт — инструкция хочет прочитать значение, которое предыдущая ещё не записала.

Способы решения:

- **ПУЗЫРЕК** — задержать зависимую инструкцию на несколько тактов.
- **Проброс операндов** — результата напрямую с выхода АЛУ, не дожидаясь `Write Back`.
- **Исполнение не по порядку** — пока зависимая инструкция ждёт, выполнить другую независимую инструкцию.
- **Переименование регистров** — применяется для ложных зависимостей `WAR` и `WAW`.

Write after read - могут быть проблемы с кешами, не оч проблема

Write after Write - когда важна последовательность записи, ex: в управляющие регистры

## 105. Как происходит разрешение конфликтов при помощи пузыря в конвейере?

**Bubble** — пустое место в конвейере, на несколько тактов заставляя нужные стадии простаивать и тем самым разносит конфликтующие операции во времени.

Например, второй инструкции нужен неготовый результат первой, зависимую инструкцию задерживают - перед ней вставляют bubble. Через несколько тактов результат появляется, и выполнение продолжается нормально.

**bubble** — не то же самое, что **NOP**. **NOP** является реальной инструкцией: её нужно считать из памяти и декодировать. **Bubble** ничего не считывает и не выполняет, это именно искусственный простой конвейера.

---

## 106. Как происходит разрешение конфликтов при помощи проброса операндов в конвейере?

Проброс операндов нужен, когда следующей инструкции требуется результат предыдущей, но тот ещё не **Write Back**-нулся.

Вместо ожидания процессор берёт результат прямо там, где он уже появился — типа, с **выхода АЛУ после стадии EX** — и напрямую подаёт его на вход АЛУ следующей инструкции.

путь становится короче: **EX первой инструкции → EX следующей инструкции**, а не: **EX → MEM → WB → регистр → EX**.

**forwarding** помогает не всегда. может всё равно понадобится один или несколько **bubble**, чтобы дождаться готовности данных.

---

## 107. Какие существуют способы разрешения конфликтов по управлению во время компиляции?

**Конфликт по управлению** - из-за переходов: процессор ещё не знает, какая инструкция будет следующей, но конвейер нужно чем-то заполнять - возможно неправильная ветка.

На этапе компиляции можно уменьшить проблему несколькими способами:

1. **Сокращать число переходов.** Например, `loop unrolling` разворачивает цикл: инструкций становится больше, но переход выполняется реже - просто последовательность однотипных действий а не цикл.
  2. **Использовать условное перемещение данных.** Инструкции вроде `mov - conditional move if less or equal` позволяют в некоторых случаях заменить `if + jump` на одну условную операцию без перехода.
  3. **Помогать предсказателю переходов.** за счет того что на этапе компиляции передавать информацию о наиболее вероятном исходе.
- 

## 108. Какие существуют способы разрешения конфликтов по управлению во время исполнения?

**Конфликт по управлению** - из-за переходов: процессор ещё не знает, какая инструкция будет следующей, но конвейер нужно чем-то заполнять - возможно неправильная ветка.

Во время исполнения конфликт по управлению решают уже средствами процессора:

1. **ПУЗЫРЕК.** Как только обнаружен переход на декодировании, загрузка новых инструкций останавливается - пузырь - до обработки перехода. Надёжно, медленно.
2. **Сброс конвейера** если ветка инструкций неверная, их отменяют и начинают загружать правильную ветку.
3. **Предсказание переходов (branch prediction).** Процессор заранее угадывает исход ветвления и выполняет предполагаемую ветку.

Последний вариант производительный: если предсказание верное, простоя почти нет. Если неверное — ошибочно загруженные инструкции приходится сбросить, поэтому возникает штраф за неверное предсказание.

---

## 109. Какие существуют способы предотвращения конфликтов по управлению через branch prediction? Что такое статические предсказатели переходов?

**Конфликт по управлению** - из-за переходов: процессор ещё не знает, какая инструкция будет следующей, но конвейер нужно чем-то заполнять - возможно неправильная ветка.

**Branch prediction** — попытка заранее угадать, произойдёт переход или нет, чтобы не останавливать конвейер и сразу начать загружать предполагаемую ветку.

**Статический предсказатель** не изучает историю исполнения программы, а использует заранее заданные правила.

Примеры:

- безусловный `jump` всегда считается выполняющимся;
- условный переход **назад** обычно считается выполняющимся, потому что так часто устроены циклы - такой переход назад статистически срабатывает примерно в 90% случаев;
- для перехода вперёд простой статический прогноз обычно заметно менее полезен.

Если предсказание верное — конвейер продолжает работать без простоя. Если нет — загруженные по неверной ветке инструкции сбрасываются.

ЕСТЬ ДИНАМИЧЕСКИЙ ПРЕДСКАЗАТЕЛЬ ПЕРЕХОДОВ

---

## 110. Что представляют собой динамические предсказатели переходов в конвейере?

**Динамический предсказатель переходов** принимает решение во время выполнения программы на основе того, как этот переход вел себя раньше.

у процессора есть таблица, где хранится:

- адрес инструкции перехода ( `Branch PC` );
- предполагаемый следующий адрес ( `Predicted PC` );
- несколько бит состояния предсказателя.

Часто используется **двухбитный счётчик с насыщением**. Если переход ПРОИЗОШЕЛ, уверенность в ПЕРЕХОДЕ УВЕЛИЧИВАЕТСЯ; если не произошёл — уменьшается. Поэтому единичное необычное выполнение не обязательно сразу переворачивает прогноз.

Более сложные предсказатели учитывают историю нескольких предыдущих переходов и умеют находить повторяющиеся паттерны поведения программы.

---

## 111. Как влияют различные виды конфликтов на производительность конвейера?

Все конфликты уменьшают преимущество конвейера, потому что мешают выдавать новую завершённую инструкцию каждый такт.

- **Структурные конфликты** уменьшают пропускную способность: две стадии конкурируют за один физический ресурс, поэтому одна вынуждена ждать.
- **Конфликты по данным** добавляют задержки между зависимыми инструкциями. Без ПРОБРОСА и других механизмов параллельное выполнение частично превращается обратно в последовательное.
- **Конфликты по управлению** особенно дороги: при неверной ветке уже частично выполненные инструкции приходится выбрасывать из конвейера.

Чем **глубже конвейер**, тем потенциально больше инструкций приходится остановить или сбросить, поэтому цена конфликта и особенно ошибки branch prediction возрастает.

---

## 112. Как связаны конвейерное исполнение и спекулятивное исполнение инструкций?

**Спекулятивное исполнение** — это выполнение работы заранее, когда процессор ещё не знает, понадобится её результат или нет.

В конвейере оно связано с **условными переходами**. Вместо того чтобы ждать результата ветвления, процессор с помощью branch prediction выбирает наиболее вероятную ветку и начинает загружать и выполнять её инструкции.

- Если прогноз оказался **верным**, уже выполненная работа сохраняется, и процессор выигрывает время.
- Если прогноз **неверный**, спекулятивные инструкции сбрасываются, состояние возвращается к корректному, после чего загружается другая ветка.

То есть спекуляция позволяет не простаивать конвейеру, но требует механизмов отмены ошибочно выполненной работы.

---

## 113. Каковы шаги типового взаимодействия с устройством ввода-вывода? Каков интерфейс устройства для процессора?

Типовое взаимодействие процессора с устройством ввода-вывода выглядит так:

1. **Конфигурация устройства.** Процессор задаёт режим работы и параметры.
2. **Проверка состояния.** Готово ли устройство принять или выдать данные.
3. **Отправка данных или команды.**
4. **Проверка статуса выполнения.** Закончилось ли действие, появился ли ответ.
5. **Чтение результата.**

Для процессора интерфейс устройства обычно выглядит как набор ячеек памяти или регистров, с которыми идет взаимодействие:

- **регистры данных** — полезная нагрузка;
- **регистры статуса** — готовность, ошибки, завершение операции;
- часто есть и **управляющие регистры** для настройки устройства.

Поверх этих регистров определяется протокол: в каком порядке что читать и записывать.

---

## 114. Как реализуется ввод-вывод с точки зрения системы команд в Port Mapped IO?

**Port Mapped IO (PMIO)** — ввод-вывод, в котором устройства имеют **отдельное адресное пространство**, не совпадающее с адресами оперативной памяти.

в системе команд появляются специальные инструкции для ввода-вывода, например:

```
in 15  
out 20
```

То есть обычный `load/store` работает с памятью, а специальные `in/out` — с портами внешних устройств.

**Главная идея:** память и ввод-вывод архитектурно разделены.

Плюс —

обычное адресное пространство памяти не тратится на устройства.

Минус —

процессор и ISA усложняются:

нужны отдельные инструкции, отдельная адресация и логика работы с ИО

---

## 115. Как реализуется ввод-вывод с точки зрения системы команд в Memory Mapped IO?

**Memory Mapped IO (MMIO)** — подход, при котором регистры внешних устройств отображаются прямо в **адресное пространство памяти**.

Например, программа делает обычную запись по адресу `0x1000`, но физически по этому адресу находится не RAM, а управляющий регистр устройства.

С точки зрения системы команд всё максимально просто: используются **те же инструкции чтения и записи**, что и для обычной памяти.

Дополнительные `in/out` не нужны.

То есть процессор выполняет обычный `load/store`, а аппаратная логика уже определяет, должен запрос уйти в оперативную память или во внешнее устройство.

Плюс — простая ISA и возможность использовать обычные механизмы адресации.

Минус — часть адресного пространства занимается устройствами.

---

## 116. Как устроен ввод-вывод в Memory Mapped IO с точки зрения устройства процессора?

Memory Mapped IO (MMIO) - способ организации ввода-вывода, когда наше устройство "подключается" по некоторым адресам в RAM, в связи с чем, читая или записывая по этим адресам на самом деле мы работаем не с памятью, а с внешним устройством. Для процессора выглядит как работа с памятью

В **MMIO** процессор формирует запрос к адресу, как будто работает с памятью.

Дальше **Address Decoder** смотрит на адрес:

- если адрес относится к RAM — активируется микросхема памяти
- если адрес относится к диапазону ввода-вывода — RAM отключается, а через сигнал выбора контроллера активируется нужный интерфейс который выставит на шину нужное значение, а процессор его защелкнет как значение из памяти.

При чтении выбранное устройство выставляет значение своего регистра на шину данных, а процессор принимает его так, будто это значение

пришло из памяти.

При записи устройство получает данные по той же шине.

Таким образом, **для программы это память**, а на уровне микроархитектуры адресный декодер маршрутизирует запрос либо в RAM, либо в конкретный интерфейс ввода-вывода.

---

## 117. Как работает программно-управляемый ввод-вывод без прерываний? Каковы его ограничения и типовой сценарий? Как связан с теоремой Котельникова?

Самый простой вариант — **polling**:

процессор периодически читает регистр состояния и проверяет, не произошло ли событие.

Если произошло — выполняет обработку, затем возвращается к опросу.

### Недостатки:

- процессор тратит время и энергию на постоянные проверки;
- тяжело совмещать ввод-вывод с другими задачами;
- сложнее программно выдерживать сложные протоколы.

Но при **очень плотном потоке событий** polling иногда выгоднее прерываний, потому что нет расходов на постоянное переключение контекста.

теорема про то что аналоговый сигнал можно восстановить его дискретным представлением, если дискретизировать его с частотой, в два раза бОльшей максимальной частоты его спектра. И на самом деле теорема здесь неприменима.

Для цифровой линии нам нужно опрашивать её достаточно часто, как минимум примерно вдвое выше частоты сигнала.

---

## 118. Что такое дребезг контактов? В чём разница между уровнем и сигналом применительно к кнопке? Как с дребезгом бороться программно?

**Дребезг контактов** — физическое явление когда:

при нажатии механической кнопки контакт не переключается

мгновенно из 0 в 1, а несколько раз быстро замыкается и размыкается.

- **уровень** — конкретное значение на входе: 0 или 1;
- **сигнал** — осмысленное событие, например «кнопка действительно нажата».

Во время дребезга уровни скачут, но это не означает множество реальных нажатий.

#### **Программный антидребезг:**

после первого изменения можно подождать  
небольшой интервал и проверить значение снова;  
считать состояние подтверждённым только после  
нескольких одинаковых чтений подряд;

---

## **119. Как реализовать псевдо-параллельный программно-управляемый ввод-вывод в одном потоке исполнения (state machine)?**

Параллельный программно-управляемый ввод-вывод - это процесс ввода-вывода без прерываний, когда обработчику одного устройства не отдается весь поток управления, он делит этот поток с другими обработчиками.

Идея — не отдавать одному устройству весь поток управления, а представить каждый обработчик как **конечный автомат (state machine)**.

Например, для кнопки: автомат хранит состояния `нажата`, `не нажата`.

Функция обработки делает только небольшой шаг и сразу отдает управление:

1. Вместо ожидания в коде, автомат **хранит свое текущее состояние в переменной**
2. функция обработчика вызывается бесконечно и при необходимости переводит автомат в новое состояние;
3. сразу возвращает управление.

В основном цикле последовательно вызываются обработчики многих устройств. Каждый работает очень недолго, поэтому процессор быстро переключается между ними, и внешне кажется, что устройства обслуживаются одновременно.

Физически поток один, поэтому это не настоящий параллелизм,  
а **кооперативная многозадачность / псевдопараллельное выполнение**.

---

## 120. Какие существуют уровни параллелизма: бит, инструкций и задач? Приведите примеры.

Можно выделить три основных уровня параллелизма:

### 1. Уровень битов.

За одну операцию обрабатывается сразу много битов.  
процессор складывает два числа не по одному биту программно,  
а одновременно все б

### 2. Уровень инструкций.

Несколько инструкций находятся в выполнении одновременно.  
**конвейер и суперскалярный процессор**, который может запускать несколько инструкций за такт.

### 3. Уровень задач.

Одновременно выполняются несколько независимых  
или частично независимых алгоритмов/процессов.  
браузер играет видео, а система параллельно скачивает файл.

Чем выше уровень, тем крупнее единица работы, которую мы пытаемся выполнять параллельно.

---

## 121. В чём разница между параллелизмом и конкурентностью (concurrency)? Приведите примеры.

**Конкурентность (concurrency)** —

несколько задач находятся в работе одновременно логически,  
но исполнитель просто быстро переключается между ними.

Например, один процессор выполняет немного задачи А, затем немного В, потом снова.  
В каждый конкретный момент он исполняет только одну задачу.

**Параллелизм** — две задачи физически выполняются одновременно разными исполнителями, например на двух ядрах процессора.

То есть:

- один исполнитель + быстрое переключение = **конкурентность**;
- несколько исполнителей + одновременная работа = **реальный параллелизм**.

Конкурентность нужна для организации нескольких независимых задач,  
а параллелизм дополнительно может дать ускорение за счёт одновременного

## **122. В чём заключается проблема реализации параллелизма уровня задач в архитектуре фон Неймана?**

Классическая архитектура фон Неймана ориентирована на **один последовательный поток инструкций**: процессор выполняет команду за командой, пока программа не завершится или не встретит переход.

Поэтому параллелизм уровня задач приходится добавлять :

- прерываниями;
- переключением контекста;
- планировщиком;
- несколькими ядрами или процессорами.

Как только вычислителей становится несколько, возникают новые проблемы: нужно синхронизировать доступ к общей памяти и устройствам, согласовывать результаты, решать конфликты и изолировать задачи друг от друга.

То есть последовательная модель фон Неймана очень проста для одной задачи, но настоящий ПАРАЛ УР ЗАДАЧ требует большого количества дополнительных архитектурных механизмов.

---

## **123. Сравните кооперативную и вытесняющую многозадачность: достоинства, недостатки и область применения каждой.**

### **Кооперативная многозадачность**

Задача сама добровольно отдаёт управление другим задачам.

#### **Плюсы:**

простая реализация,  
мало накладных расходов,  
задачу не прерывают в случайном месте,  
поэтому проще сохранять атомарность операций.

**Минусы:**

если одна задача зависла или долго не отдаёт управление, остальные тоже перестают выполняться.  
Сбой одной задачи может повлиять на всю систему.

Часто используется во **встроенных системах** и в асинхронных средах

**Вытесняющая многозадачность**

Задачу может остановить **ОС/планировщик независимо от её желания**.

**Плюсы:**

хорошая интерактивность,  
ОС контролирует ресурсы и не даёт одной задаче захватить процессор.

**Минусы:**

переключение контекста стоит времени,  
а непредсказуемое прерывание порождает гонки и проблемы синхронизации.

Это основной подход в ОС общего назначения.

---

## **124. Какие механизмы необходимы для реализации кооперативной многозадачности?**

Для кооперативной многозадачности нужны :

1. **Добровольная остановка / yield.**  
задача сама передаёт управление диспетчеру.
2. **Сохранение состояния.**  
Нужно запомнить регистры, стек и другое состояние, чтобы потом продолжить с того же места.
3. **Планирование.**  
Диспетчер выбирает, какую задачу выполнять следующей.
4. **Возобновление.**  
Состояние выбранной задачи восстанавливается, после чего ей снова передаётся процессор.

Дополнительно могут понадобиться:

- **изоляция задач**, чтобы они не портили память друг друга;
- **механизмы взаимодействия**, если задачам нужно обмениваться данными.

Главное отличие от вытесняющей схемы: переключение начинается по инициативе самой выполняющейся задачи.

---

## 125. Как взаимодействуют кооперативная многозадачность и система прерываний?

Для **кооп многозадач** аппаратные прерывания не обязательны: задача сама вызывает системную функцию и добровольно отдаёт управление диспетчеру.

Теоретически добровольное переключение можно оформить и через **программное прерывание**, но это не является сутью подхода.

При этом кооперативная многозадачность отлично может существовать **вместе с аппаратными прерываниями**. Например:

- задачи переключаются между собой добровольно;
- таймер, сеть или др устр используют аппаратные прерывания для быстрого ввода-вывода.

То есть прерывания могут обслуживать внешние события, не превращая сам механизм планирования задач в вытесняющий. Вытеснение появляется, когда прерывание используется именно для принудительной остановки текущей задачи.

---

## 126. Как реализуется кооперативная многозадачность на уровне приложения через Event-loop?

**Event-loop** — цикл событий, который позволяет одному потоку обслуживать много задач без блокировки на долгих операциях.

1. В очереди появляется задача или событие.
2. Event-loop запускает соответствующий обработчик.
3. Обработчик работает пока ему не требуется внешняя операция: диск, сеть, база данных и т.д.
4. Вместо ожидания он отправляет запрос ОС, регистрирует **callback** и отдаёт управление.
5. Event-loop начинает выполнять другую задачу.

6. Когда внешняя операция завершилась, событие попадает в очередь, и выполнение первой задачи продолжается.

один поток не простаивает на ожидании I/O.

задача передаёт управление в определённых безопасных точках.

---

## 127. Каковы достоинства и недостатки вытесняющей многозадачности?

При **вытесняющей многозадачности** планировщик может остановить задачу практически между любыми двумя инструкциями и передать процессор другой.

### Достоинства:

- ОС сама распределяет процессорное время;
- зависшая задача не обязана блокировать остальные;
- получается хорошая интерактивность и быстрый отклик системы;
- ресурсы контролирует ОС, а процессы можно изолировать друг от друга.

### Недостатки:

- переключение контекста создаёт накладные расходы;
  - процесс хранит большой контекст: регистры, стек, состояние памяти;
  - задачу могут прервать в непредсказуемый момент;
  - нарушается естественная атомарность операций, поэтому появляются гонки и необходимость синхронизации;
  - время выполнения становится менее предсказуемым.
- 

## 128. Какие механизмы необходимы для реализации вытесняющей многозадачности?

В вытесняющей многозадачности нужны примерно те же механизмы, что и в кооперативной, но принципиально меняется способ остановки задачи:

1. **Прерывание.** Система должна уметь забрать управление у текущей задачи независимо от её желания. Обычно для этого используется таймер.
2. **Сохранение контекста.** Сохраняются PC, регистры, стек и т.д. состояние задачи.
3. **Планировщик.** Решает, какая задача должна выполняться следующей.

4. **Восстановление контекста.** Загружается состояние новой задачи.
5. **Возобновление выполнения.** Процессор продолжает её с сохранённого места.

также нужны **изоляция памяти** и механизмы **межпроцессного взаимодействия**, потому что задачи выполняются независимо и не должны портить состояние друг друга.

---

## 129. Что даёт переход от программно-управляемого ввода-вывода (polling) к вводу-выводу через систему прерываний с точки зрения эффективности и параллелизма уровня задач?

При polling процессор постоянно спрашивает устройство: «данные готовы?». Это занимает процессорное время даже тогда, когда ничего не происходит.

При **вводе-выводе через прерывания** процессор запускает операцию и продолжает выполнять основную программу.

Когда устройство действительно требует внимания, оно генерирует прерывание и запускается обработчик.

За счёт этого:

- процессор не тратит время на пустой опрос;
- быстрее реагирует на редкие внешние события;
- может параллельно по времени выполнять полезную задачу;
- появляется естественное разделение на
  - **основную задачу и обработчик прерывания**

При очень частых событиях polling иногда всё же выгоднее из-за отсутствия накладных расходов на множество прерываний.

---

## 130. Как по шагам происходит обработка прерывания?

1. Внешнее / внутреннее событие регаётся системой прерываний.
2. Процессор завершает текущую операцию и **сохраняет контекст**:  
как минимум Program Counter, а обычно и необходимые регистры.
3. определяется адрес обработчика (ISR).
4. Процессор переходит на этот адрес и выполняет обработчик.

5. Если система допускает вложенные прерывания, более приоритетное событие может прервать текущий обработчик.
6. После завершения сохранённый контекст восстанавливается.
7. Специальная команда возврата из прерывания возвращает выполнение откуда была остановлена основная программа.

Идея —

временно нарушить обычный последовательный поток инструкций, обработать событие и затем полностью восстановить исходное выполнение.

---

## **131. Почему появление системы прерываний приводит к автоматическому появлению параллелизма уровня задач? Охарактеризуйте его.**

Как только появляется система прерываний, фактически существуют как минимум **две независимые задачи**:

- основная программа;
- обработчик прерывания.

Основная программа выполняется своим потоком, а внешнее событие может в любой момент переключить процессор на обработчик. После обработки управление возвращается.

На одном ядре это не физический параллелизм в один момент времени, а **конкурентное / псевдопараллельное выполнение**: задачи чередуются во времени.

Но с точки зрения архитектуры это параллелизм уровня задач, потому что существуют независимые вычислительные процессы со своим состоянием и моментами запуска.

Особенно важно, что обработчик и основная программа могут обращаться к общим данным, поэтому сразу возникают вопросы атомарности и синхронизации.

---

## **132. Как классифицируются прерывания по источнику (синхронные/асинхронные, аппаратные/программные)? Их отличительные особенности.**

По источнику прерывания можно разделить ап и прог:

## Аппаратные

Возникают из-за событий в железе.

- **Внешние / асинхронные** — прерывания не связаны с текущей инструкцией процессора  
таймер, клавиатура, сетевой пакет.
- **Внутренние / синхронные** — прерывания-следствия конкретной инструкции.  
деление на ноль, ошибка доступа к памяти.

## Программные

Вызываются самой программой спец инструкцией.

Например, так можно передать управление ОС или вызвать системный сервис.

Главная разница:

**синхронное** событие возникает в определённой точке программы,  
а **асинхронное** может прийти независимо от того, какую инструкцию процессор выполняет сейчас.

---

## 133. Каковы задачи контроллера прерываний? Что такое маскИруемые и немаскИруемые прерывания? Относительный и абсолютный приоритеты?

**Контроллер прерываний** аппаратно принимает запросы от разных источников, фиксирует их, определяет приоритет, передаёт процессору нужное прерывание и помогает выбрать соответствующий обработчик.

**МаскИруемое прерывание** можно временно запретить, тип сбросив соответствующий бит разрешения.

**НемаскИруемое** отключить обычным способом нельзя;  
его оставляют для критических событий вроде серьёзного аппаратного сбоя.

По приоритетам:

- **Относительный приоритет** — когда после входа в обработчик новые прерывания не вытесняют его, даже если сами имеют более приоритетны.
- **Абсолютный приоритет** — когда обработчик может быть прерван другим, более приоритетным прерыванием.

То есть контроллер решает не только *что произошло*, но и *какое событие обслуживать первым*.

---

## 134. Какие виды событий существуют в системе прерываний (по фронту, по уровню, по сообщению, doorbell)? В чём особенности каждого?

Основные способы сигнализировать о событии:

1. **По фронту** Прерывание возникает в момент изменения уровня, например `0 -> 1`. Событием является сам переход.
2. **По уровню** Пока на линии держится активный уровень, запрос прерывания считается активным. Устройство должно убрать уровень после обслуживания.
3. **По сообщению**. Источник передаёт процессору/контроллеру специальное сообщение, из которого можно определить, какое устройство инициировало событие.
4. **Doorbell**. Отдельное уведомление в стиле «данные уже готовы, заberi их из известного места». Само прерывание не несёт весь блок данных, а только сообщает о его готовности.

Разница в том, **как именно кодируется факт события и информация о его источнике**.

---

## 135. Что такое сторожевой таймер и каков принцип его работы?

**Сторожевой таймер** — аппаратный механизм, который позволяет автоматически восстановить систему, если процессор или программа зависли.

1. сторож запускает независимый таймер.
2. Нормально работающая программа периодически посылает ему сигнал — «я жива».
3. После такого сигнала отсчёт начинается заново.
4. Если программа зависла, сигнал больше не приходит.
5. Таймер достигает заданного порога и аппаратно инициирует `Reset` процессора или всей системы.

Главное преимущество — не зависит от зависшей программы. Даже если обычный код уже ничего не может сделать, отдельная аппаратная схема всё равно способна

перезапустить устройство.

---

## 136. Какие подходы существуют к решению проблемы изоляции регистров и инструкций в памяти при многозадачности?

При многозадачности каждая задача должна видеть **своё состояние**, а не случайно изменять состояние другой.

### Изоляция регистров

Есть два основных подхода:

1. **Программно сохранять контекст** при переключении задачи :  
регистры сохраняются в память, а регистры следующей задачи восстанавливаются.
2. **Несколько физических банков регистров.**  
При переключении меняется активный набор регистров.  
быстрее, но требует больше аппаратуры.

### Изоляция инструкций в памяти

Код разных задач размещают в защищённых областях памяти.

Для этого могут использоваться

**банки памяти, сегментация или виртуалка**

Процесс получает доступ только к своему адресному пространству, а система памяти не позволяет ему читать или изменять чужой код.

Идея одна: не давать задачам бесконтрольно видеть состояние друг друга.

---

## 137. Какие подходы существуют к решению проблемы изоляции данных в памяти (статика, куча, стек) при многозадачности?

У программы есть несколько типов памяти:

- **статика** — глоб и стат данные, обычно секция `.data` ;
- **куча** — динамически выделяемая память;
- **стек** — автомат память вызовов функций и лок переменных.

При многозадачности данные разных процессов нужно изолировать, иначе один процесс сможет повредить другой.

Основные аппаратные подходы:

1. **Банки памяти** — разным задачам физически выдаются разные банки.
2. **Сегментация** — каждой задаче выдаются свои сегменты с границами и правами.
3. **Виртуальная память** — каждый процесс получает собственное виртуальное адресное пространство, которое отображается на разрешённые ему физические страницы.

Сегодня основным механизмом изоляции является именно виртуальная память.

---

## 138. Как банки памяти используются для расширения адресного пространства?

**Банки памяти** позволяют подключить к процессору больше физической памяти, чем он может адресовать напрямую одним обычным адресом.

Идея: **номер банка** выбирает, с какой именно микросхемой сейчас работает процессор. несколько микросхем памяти используют один и тот же диапазон адресов.

Например, если процессор видит только 64 КБ адресов, можно подключить несколько банков по 64 КБ. Адрес внутри банка остаётся тем же, но отдельный регистр или дополнительные управляющие биты выбирают активный банк.

Мы ставим внешний переключатель, и номер выбранного банка работает как **дополнительные провода для адреса**.

Сам процессор думает, что работает, например, со своими стандартными 64 Кб, но за счёт этих внешних битов мы можем по очереди подставлять ему разные чипы памяти, увеличивая общий объём памяти в разы

Минус — перед обращением к другой области памяти приходится явно переключать банк.

---

## 139. Как банки памяти используются для расширения машинного слова?

Банки памяти можно использовать не только для увеличения объёма, но и для получения более широкого машинного слова.

Например, процессору нужно читать слово шириной 16 бит, а доступные микросхемы памяти имеют ширину только 8 бит.

Тогда ставят **два банка параллельно**:

- первый хранит младшие 8 бит;
- второй — старшие 8 бит.

При чтении оба банка одновременно получают один адрес и выставляют свои части на разные линии шины данных. Процессор объединяет их и за один доступ получает полноценное 16-битное слово.

---

## 140. Что такое сегментная память? Как происходит трансляция адресов? Каковы достоинства и недостатки?

**Сегментная память** делит адресное пространство на логические сегменты. Адрес состоит из двух частей:

номер сегмента + смещение внутри сегмента .

В **таблице сегментов** для каждого сегмента хранятся его физический базовый адрес, размер и права доступа.

Трансляция:

1. по номеру сегмента находится его начало;
2. проверяется, что смещение не выходит за границу;
3. физический адрес вычисляется как `base + offset` .

**Плюсы:**

изоляция процессов, контроль границ, права `read/write/execute` , возможность подключить один сегмент нескольким процессам.

**Минусы:**

сложное управление сегментами и компиляция - размер рандом фрагментов границы, **внешняя фрагментация** — между сегментами переменного размера со временем остаются маленькие свободные куски памяти.

---

## 141. Что такое виртуальная память? Как происходит трансляция адресов? Каковы достоинства и недостатки?

**Виртуальная память** создаёт для каждого процесса иллюзию, что у него есть собственное непрерывное адресное пространство.

Память делится на одинаковые **страницы**, а физическая память — на кадры такого же размера. Виртуальный адрес состоит из:

номер виртуальной страницы + смещение .

По **Page Table** номер виртуальной стр переводится в номер физ кадра, после чего к его адресу добавляется то же смещение.

#### **Плюсы:**

изоляция процессов, удобное линейное адресное пространство, страницы можно физически размещать где угодно, нет внешней фрагментации, часть страниц можно временно на диске.

**Минусы:** таблицы страниц могут быть огромными, трансляция требует дополнительных кэшей TLB, а `page fault` (когда нет в оперативке идем в диск) делает время доступа к памяти непредсказуемым.

---

## **142. Что такое внутренняя и внешняя фрагментация в контексте сегментной памяти?**

**Внутренняя фрагментация** — неиспользуемая память **внутри уже выделенной области**. Например, сегменту фактически нужно 950 байт, а система по некоторой причине выделила 1 КБ — оставшаяся часть не используется.

**Внешняя фрагментация** — свободная память есть, но она разбита на множество маленьких несмежных участков **между сегментами**.

Для сегментной памяти особенно характерна именно **внешняя фрагментация**, потому что сегменты имеют разный размер и должны занимать непрерывный диапазон физической памяти.

После множества операций выделения и освобождения может оказаться, что суммарно свободной памяти достаточно, но одного большого непрерывного участка для нового сегмента нет.

---

## 143. Что такое внутренняя и внешняя фрагментация в контексте виртуальной памяти?

В страничной виртуальной памяти ситуация отличается от сегментации.

**Внутренняя фрагментация** остаётся: память выделяется целыми страницами, поэтому последняя страница области может быть заполнена не полностью. Например, если странице 4 КБ, а программе нужно ещё только 1 КБ, оставшиеся 3 КБ этой страницы простаивают.

**Внешней фрагментации практически нет**, потому что физические страницы имеют одинаковый размер и не обязаны лежать рядом. Любой свободный кадр можно отдать любой виртуальной странице.

Именно поэтому виртуальная память позволяет плотнее использовать физическую RAM: процесс видит непрерывное адресное пространство, хотя его страницы на самом деле могут быть разбросаны по всей памяти.

---

## 144. Как появление сегментной памяти улучшило пользовательский опыт?

Сегментная память дала программам гораздо более безопасную и независимую среду выполнения.

Основные улучшения:

- программы могут работать со своими областями памяти, не оглядываясь на размещение других программ;
- ошибка одной программы с меньшей вероятностью повреждает данные другой, потому что доступ ограничен границами сегмента;
- для сегментов можно задавать разные права: например, код сделать только исполняемым/читаемым, а данные — записываемыми;
- один сегмент можно подключить нескольким программам, например для **общей библиотеки или shared data**.

То есть пользователь получает возможность безопаснее запускать несколько программ одновременно, а разработчик — более понятную модель памяти и аппаратную защиту от выхода за разрешённые области.

---

## 145. Как появление виртуальной памяти улучшило пользовательский опыт?

Виртуальная память делает работу нескольких программ намного удобнее и надёжнее.

1. **Каждый процесс получает своё адресное пространство.** Программа может писать код так, будто она одна использует память компьютера.
2. **Изоляция.** Ошибка в управлении памятью одного процесса обычно не повреждает память другого.
3. **Можно использовать больше памяти, чем физически есть в RAM.** Неактивные страницы можно временно хранить на диске.
4. **Физическую память проще распределять.** Страницы процесса не обязаны лежать рядом, поэтому уменьшается проблема внешней фрагментации.

Для пользователя это означает, что одновременно можно запускать много независимых программ, а ОС сама решает, где физически лежат их данные и как распределить доступную память.

---

## 146. Какие уровни (виды) задач существуют: основной поток, прерывание, процессы, потоки, зелёные потоки?

Условно можно выделить несколько уровней задач:

1. **Основной поток** — исходный последовательный поток инструкций системы или программы. управляет **аппаратура процессора** (счётчик команд PC ).
  2. **Обработчики прерываний** — задачи, которые временно прерывают основной поток при внешнем или внутреннем событии. управляет **аппарат проца совместно с ОС**
  3. **Процессы** — тяжёлые изолированные задачи со своим адресным пространством. управляет ос
  4. **Потоки** — задачи внутри процесса. Они имеют собственный стек и контекст исполнения, но **разделяют адресное пространство и данные процесса.** управляет ос
  5. **Зелёные потоки** — потоки, которыми управляет не ОС, а runtime или виртуальная машина. Обычно они реализуют лёгкую коопера многозадачность, для асинхронного исполнения.
-

## 147. Как могут взаимодействовать процессы через основной поток? Какие бывают разделяемые ресурсы?

Процессы обычно имеют **раздельные адресные пространства**, поэтому один процесс не может просто взять указатель и прочитать память другого. Взаимодействие организует **ОС / ядро**.

Основные варианты:

1. **общая Memory**. Одни и те же физические страницы отображаются в адресные пространства нескольких процессов.
2. **Signals**. Один процесс через ОС посылает другому короткое управляющее событие, например просьбу завершиться.
3. **Ввод-вывод**. через файлы, сокеты, сеть могут обмениваться данными
4. **каналы**. Ядро предоставляет канал, куда один процесс пишет данные, а другой читает.

То есть процессы остаются изолированными, но есть общ ресурс

---

## 148. Что такое процессоры ввода-вывода? Каковы их назначение, интерфейс, достоинства и недостатки?

**Процессор ввода-вывода** — отдельный процессор рядом с CPU, который берёт на себя работу с внешними устройствами.

Основной процессор передаёт ему **не отдельную операцию, а небольшую программу или последовательность команд ввода-вывода**. После этого I/O processor самостоятельно выполняет её, а CPU продолжает считать свою задачу.

**Плюсы:**

- разгружает основной CPU;
- ввод-вывод идёт параллельно с вычислениями;
- эффективное обслуживание сложных устройств и протоколов.

**Минусы:**

- отдельный процессор сильно усложняет и удорожает систему;
- архитектура получается специализированной;
- нужно синхронизировать два вычислителя.

Поэтому сегодня такой подход используется редко, а во многих задачах его заменяет более простой **DMA**.

---

## **149. Что такое контроллер прямого доступа к памяти (DMA)? Каковы его назначение, интерфейс, достоинства и недостатки?**

**DMA (Direct Memory Access)** — контроллер, который умеет переносить блоки данных между памятью и внешним устройством **без постоянного участия CPU**.

Процессор обычно задаёт DMA параметры: откуда читать, куда писать, размер и направление передачи. После этого контроллер сам выполняет перенос и сообщает процессору о завершении.

### **Плюсы:**

- CPU освобождается от рутинного копирования каждого байта;
- ввод-вывод идёт параллельно с выполнением программы;
- DMA может обслуживать несколько устройств;
- особенно эффективен для больших последовательных блоков данных.

### **Минусы:**

- DMA и CPU конкурируют за память и системную шину;
  - сложнее синхронизировать их работу;
  - непоследовательный доступ менее удобен, потому что DMA лучше работает с непрерывными блоками;
  - процессор перестаёт полностью контролировать все обращения к памяти.
- 

## **150. Какие существуют способы интеграции и взаимодействия контроллера прямого доступа к памяти (DMA) с процессором?**

DMA и процессор делят доступ к памяти, поэтому главное — определить, кто и когда получает системную шину.

### **Основные режимы:**

- **Пакетный режим:** приоритет получает DMA. Пока он передаёт целый блок данных, процессор ждёт.
- **Циклический режим:** доступ делится по тактам или циклам: часть времени работает процессор, часть — DMA.
- **Прозрачный режим:** приоритет у процессора. DMA использует шину только тогда, когда процессору она не нужна.

То есть различие между режимами — в **арбитраже доступа к памяти**. Чем больше приоритет DMA, тем быстрее крупные передачи, но тем сильнее он может тормозить процессор.

---

## 151. Что такое гонка состояний (race condition)? Почему она возникает при вытесняющей многозадачности?

**Race condition** — ситуация, когда итог работы программы зависит от того, в каком порядке выполнялись несколько задач или потоков.

Например, два потока читают одну переменную, изменяют её и записывают обратно. Если их инструкции перемешаются неудачно, одно из изменений может потеряться.

При **вытесняющей многозадачности** это особенно опасно, потому что планировщик может остановить задачу практически между любыми двумя инструкциями и передать процессор другой задаче. Поэтому порядок доступа к общим данным заранее неизвестен.

Проблема возникает именно там, где несколько задач работают с **общим изменяемым состоянием** без достаточной синхронизации.

---

## 152. Как синхронизируются потоки? Что такое атомарная операция, мьютекс и семафор?

Синхронизация нужна, чтобы несколько потоков безопасно работали с общими ресурсами и не создавали гонки состояний.

- **Атомарная операция** выполняется как единое неделимое действие: если она началась, другой поток не может увидеть её «наполовину выполненной».

- **Мьютекс** — объект, который одновременно может захватить только один поток. Остальные ждут освобождения.
- **Семафор** — более общий механизм: одновременно ресурс могут захватить не более  $N$  потоков.

Идея одна: ограничить одновременный доступ к критическому участку и сделать порядок работы с общими данными контролируемым.

---

## 153. Почему необходима иерархия памяти в современных компьютерных системах?

Иерархия памяти возникает из-за:

**чем память быстрее, тем она дороже и меньше по объёму.**

Нельзя сделать всю память такой же быстрой, как регистры процессора, и одновременно дешёвой и огромной. Поэтому используют несколько уровней.

Ближе к процессору находятся маленькие и очень быстрые регистры и кеши. Дальше — более объёмная, но медленная DRAM, затем SSD, HDD и другие накопители - перечислить из других вопросов.

Так система получает почти скорость маленькой быстрой памяти, но при этом сохраняет большой общий объём хранения.

---

## 154. Какие основные виды памяти входят в иерархию памяти и каковы их характеристики?

Типичная иерархия идёт от самой быстрой и маленькой памяти к самой медленной и большой:

1. **Регистры процессора** — минимальная задержка, очень маленький объём.
2. **Кеши L1/L2/L3** — очень быстрые, но дорогие по площади.
3. **Оперативная память DRAM** — заметно медленнее кеша, зато гораздо больше.
4. **SSD** — энергонезависимая память с большей задержкой - пояснить что эт.
5. **HDD** — ещё медленнее, зато дешёвый большой объём - пояснить.

6. **Сетевые хранилища и магнитные ленты** — огромный объём, но очень высокая задержка.

При движении вниз обычно **растёт объём и падает цена за бит**, но увеличивается время доступа.

---

## 155. В чём различие между явной иерархией памяти и скрытой? Приведите примеры.

**Явная иерархия** — программист сам понимает, где именно лежат данные, и явно выбирает нужный тип памяти. Например, отдельно работает с регистрами устройства, SRAM, DRAM или диском.

**Скрытая иерархия** — программа видит более единое адресное пространство, а система сама перемещает данные между уровнями.

Пример — современные системы с кешами и виртуальной памятью: процесс обращается к адресу, а процессор и ОС сами решают, находится страница в кеше, RAM или временно выгружена на диск.

Плюс скрытого подхода — проще программировать. Минус — сложнее устройство системы и меньше контроля над точным временем доступа.

---

## 156. Какие типы доступа к памяти существуют (произвольный, последовательный, гибридный)? Приведите примеры.

Выделяют три основных способа доступа.

- **Произвольный доступ:** задержка почти не зависит от того, к какому адресу обращались раньше. Главный пример — RAM.
- **Последовательный доступ:** чтобы добраться до нужного места, приходится проходить данные в определённом порядке. Примеры — магнитные ленты, частично механические накопители.
- **Гибридный доступ:** система сочетает оба подхода. До крупного блока добираемся одним способом, а внутри него работаем более свободно. линии кеша

Последовательный доступ может быть очень быстрым на длинном непрерывном потоке, но дорогим при частой смене позиции.

---

## 157. Каково устройство памяти с произвольным доступом (decoder, wordline, bitline)? Как устроена ROM-ячейка и как происходит чтение?

Память с произвольным доступом обычно строится как **матрица битовых ячеек**.

Адрес поступает в **decoder**, который выбирает нужную строку и активирует её **wordline**. Ячейки выбранной строки воздействуют на вертикальные **bitline**, по которым формируется читаемое машинное слово.

В простой ROM-ячейке :

данные «зашиваются» ещё на заводе с помощью физического наличия или отсутствия транзисторов в сетке.

Транзистор здесь работает как обычный выключатель, который умеет замыкать вертикальный провод (*bitline*) на «землю» (логический ноль).

- для **1** - транзистора нет, и *bitline* остаётся в предварительно установленной единице; Провод *wordline* проходит мимо пустого места
  - для **0** - транзистор есть и при активации *wordline* стягивает *bitline* к нулю.
  - Перед началом чтения абсолютно все вертикальные линии (*bitlines*) принудительно «заряжают» током — устанавливают в состояние **логической 1** .
  - Декодер адреса подаёт ток на нужный *wordline*
  - где закодирована **единица** (транзистора нет), линия *bitline* так и остаётся висеть в состоянии **1**
  - где закодирован **ноль** (транзистор есть), ток с *wordline* открывает транзистор. Тот мгновенно **стягивает (замыкает) свой *bitline* на ноль**.
  -
1. считывает уровни напряжений с вертикальных линий. Где остался заряд — там **1** , где он ушёл в землю — там **0** . Это и есть наше прочитанное слово.
-

## 158. Каково устройство многопортовой памяти? Как связана площадь с количеством портов?

**Порт памяти** — независимый интерфейс, через который можно читать или записывать данные. Обычная память (однопортовая) может обслужить только один запрос за такт: либо одно чтение, либо одну запись.

Если нам нужно, чтобы процессор мог параллельно читать два операнда и одновременно записывать третий результат (как это постоянно происходит в АЛУ), обычной памяти не хватит — нужна **многопортовая память**.

**приходится полностью дублировать всю инфраструктуру доступа к ячейкам.** Для каждого нового порта с нуля создаются свои:

- **Decoder** — индивидуальный декодер адреса для этого порта.
- **Wordline** — своя независимая сетка горизонтальных линий выбора строки.
- **Bitline** — свои вертикальные провода для чтения/записи данных.
- **Транзисторы доступа** — к каждой базовой битовой ячейке приходится подпаивать дополнительные управляющие транзисторы, чтобы к ней можно было подойти с «разных сторон».

Поэтому многопортовая память позволяет обслуживать несколько обращений параллельно, но резко увеличивает площадь схемы.

Поэтому большое число портов очень дорого и используется только там, где параллельный доступ действительно необходим.

---

## 159. Сравните технологии реализации битовых ячеек (Flip-flop, ROM, SRAM, DRAM) по числу транзисторов, скорости, плотности и стоимости. В какой области иерархии памяти каждая из них применяется?

- **Flip-flop:** около 20 транзисторов на бит. Очень быстрый и очень дорогой по площади. Используется в регистрах и небольшой внутренней памяти процессора.
- это логическая схема, которая мгновенно выдает состояние, но жрёт безумное количество транзисторов. Поэтому регистров у процессора всего несколько десятков.  
Он собирается из нескольких логических вентилей (И-НЕ / ИЛИ-НЕ), соединенных в кольцо, плюс обвязка для управления.

Ему не нужны никакие внешние декодеры, шины или такты, чтобы просто удерживать значение. Из-за этого он выдает данные **\*\*мгновенно (0 тактов)\*\***

- **ROM:** примерно 0–1 транзистор на бит. Чтение быстрое, а запись ограничена, сложна или вообще невозможна. Используется для постоянных данных.
- данные в нее зашиваются раз и навсегда, и нам вообще **\*\*не нужно** тратить элементы на схему записи**\*\***.
- **SRAM:** классически 6 транзисторов. Быстрая, не требует регенерации, но дорогая. Основное применение — кеши.
- **\*\*4 транзистора\*\*** образуют два перекрестных инвертора. Они непрерывно держат друг друга: если на одном `1`, он удерживает на втором `0`, и наоборот  
**\*\*2 транзистора\*\*** стоят по бокам как «ворота» (управляющие ключи). Они открываются только тогда, когда активируется линия строки (`_wordline_`), чтобы впустить или выпустить данные на вертикальные шины
- **DRAM:** примерно один транзистор и конденсатор на бит. Очень плотная и дешёвая, но медленнее SRAM и требует регенерации. Используется как основная оперативная память.
- **\*\*Конденсатор\*\*** — это крошечное ведерко с зарядом. Есть заряд — `1`, нет заряда — `0`.  
**\*\*Транзистор\*\*** — это просто кран, который открывает доступ к этому ведерку для чтения или записи.

Итого: сверху иерархии — быстрые дорогие ячейки, ниже — более плотные и дешёвые.

---

## 160. Каковы технологии реализации SRAM ячеек? Как происходит чтение и запись?

**SRAM-ячейка** состоит из

4 транзистора = 2 инвертора. Инверторы образуют два устойчивых состояния и тем самым хранят один бит.  
2 транзистора доступа.

**Чтение:**

- Decoder включает строку (*wordline*).

- Защёлка выдаёт ток на вертикальные провода (*bitlines*).
- усилитель ловит разницу потенциалов.
- Выдаёт чистый 0 или 1

#### Запись:

1. На провода *bitlines* принудительно подают 1 и 0.
2. Включают строку *wordline*.
3. Мощные внешние драйверы **пересиливают** защёлку.
4. Защёлка перекидывается в новое состояние.

SRAM быстрая и не требует refresh, но шесть транзисторов на бит делают её дорогой и неплотной.

## 161. Каковы технологии реализации DRAM ячеек? Как происходит чтение и запись? Что такое регенерация и зачем она нужна?

- 1 конденсатор — хранит заряд (1 или 0).
- 1 транзистор — ключ для доступа.

В DRAM бит хранится как **заряд конденсатора**: заряжен — условная 1, разряжен — 0. Для доступа используется транзистор, управляемый *wordline*.

#### Запись:

1. Включают строку *wordline* (открывают ключ).
2. На провод *bitline* подают ток или замыкают в ноль.
3. Конденсатор заряжается или разряжается.

#### Чтение:

- Вертикальный провод заряжают ровно наполовину.
- Открывают ключ транзистора.
- Конденсатор делится зарядом с проводом.
- Напряжение на проводе чуть прыгает вверх (1) или падает (0).
- Усилитель распознаёт этот микро-сдвиг.

Чтение разрушает исходный заряд, а заряд ещё и постепенно утекает сам. Поэтому контроллер периодически выполняет **регенерацию (refresh)** — считывает и заново восстанавливает содержимое ячеек.

---

## 162. Что такое кеш? Каково его назначение, место в иерархии памяти и основные метрики?

**Кеш** — небольшая быстрая СРАМ память между процессором и более медленной памятью. В нём хранят данные и инструкции, которые с высокой вероятностью понадобятся снова.

Главная задача — уменьшить среднее время доступа к памяти. Обычно кеш строится на SRAM и для программы работает почти прозрачно.

### Основные характеристики:

- объём;
- задержка доступа;
- размер кеш-линии;
- ассоциативность - правила разметки кэша;
  - прямое отображение с фиксированными местами и риском конфликтов,
  - полностью ассоциативный кэш с гибким размещением, но сложным и дорогим оборудованием,
  - компромиссный множественно-ассоциативный кэш, используемый в современных процессорах.
- **hit rate** — доля обращений, найденных в кеше;
- соответственно **miss rate** — доля промахов.

Чем выше hit rate, тем реже процессору приходится обращаться к медленным уровням памяти.

---

## 163. Что такое принцип локальности и каковы его виды (временная, пространственная) применительно к механизму кеширования?

**Принцип локальности** говорит, что обращения к памяти обычно распределены не случайно. Недавно использованные данные и соседние адреса часто понадобятся снова.

Есть два основных вида:

- **Временная локальность:** если программа недавно обращалась к некоторому адресу, вероятно, скоро обратится к нему ещё раз.
- **Пространственная локальность:** если использован некоторый адрес, велика вероятность обращения к соседним адресам.

Именно поэтому кеш хранит не только один запрошенный байт, а целую **кеш-линию**. Это позволяет одним обращением подтянуть соседние данные и использовать пространственную локальность.

---

## 164. Как пошагово происходит процесс чтения данных через кеш-память процессора?

Пусть процессор хочет прочесть данные по некоторому адресу.

1. Процессор выдаёт адрес нужной переменной.
2. Запрос падает в контроллер кэша.
3. Кэш проверяет **теги** сохранённых линий по этому адресу.
4. **Вариант А: Cache Hit (Попадание)**
  - Линия найдена. Данные мгновенно улетают в процессор.
5. **Вариант Б: Cache Miss (Промех)**
  - Процессор встаёт на паузу.
  - Кэш отправляет запрос на нижний уровень (L2/L3 или ОЗУ).
  - Скачивается целая новая кэш-строка.
  - Если кэш забит, старая ненужная строка выселяется.
  - Новая строка пишется в кэш вместе со своим тегом.
  - Данные наконец отдаются ядру. Процессор продолжает работу.

Промех увеличивает задержку, причём точное время уже зависит от более медленных уровней памяти.

---

## 165. Как пошагово происходит процесс записи данных через кеш-память процессора? Каковы политики записи при попадании (write-through, write-back) и при промехе (write-allocate, no-write-allocate)?

Сначала кеш проверяет, есть ли нужный блок.

#### При cache hit:

- **write-through:** меняем данные в кеше и сразу отправляем изменение на следующий уровень памяти. Просто и согласованно, но медленнее.
- **write-back:** меняем только кеш и помечаем линию изменённой. В нижнюю память данные уходят позже, например при вытеснении.

#### При cache miss:

- **write-allocate:** сначала загружаем блок в кеш, затем выполняем запись как при hit.
- **no-write-allocate:** записываем сразу на нижний уровень, не размещая блок в текущем кеше.

Политики выбирают компромисс между задержкой записи, трафиком памяти и вероятностью повторного обращения к данным.

---

## 166. Что такое кеш-промах? Как промах по-разному влияет на чтение данных, запись данных и чтение инструкций?

**Cache miss** — это когда процессор спросил адрес, а нужной строки в кэше нет. Приходится идти в медленную ОЗУ.

Влияние зависит от типа операции:

- **Чтение данных:** процессор должен получить данные с более медленного уровня. Задержку иногда можно частично скрыть параллельным выполнением других инструкций.
- **Запись данных:** задержка часто скрывается лучше, потому что запись можно поместить в буфер или использовать подходящую политику записи.
- **Чтение инструкции:** особенно критично. Пока следующая инструкция не пришла, процессору может быть нечего декодировать и выполнять.

Если промахи происходят часто, выигрыш от кеша резко уменьшается.

---

## 167. Что такое ассоциативность кеша? Какова "структура" адреса с точки зрения кеша?

- **Ассоциативность кеша** определяет, в какие кеш-линии может быть помещён блок памяти с конкретным адресом.
  - прямое отображение с фиксированными местами и риском конфликтов,
  - полностью ассоциативный кэш с гибким размещением, но сложным и дорогим оборудованием,
  - компромиссный множественно-ассоциативный кэш, используемый в современных процессорах.

С точки зрения кеша адрес условно делится на части:

- **offset** — младшие биты, выбирают байт внутри кеш-линии;
- **index** — выбирает конкретную линию;
- **tag** — старшие биты, позволяют проверить, тот ли блок сейчас лежит в выбранном месте.

Если кеш-линия содержит 64 байта, младшие 6 бит — это offset.

Чем выше ассоциативность, тем больше возможных мест для блока и меньше конфликтных промахов, но тем больше сравнений тегов и сложнее логика.

---

## 168. Каковы особенности реализации полностью ассоциативного кеша?

В **полностью ассоциативном кеше** любой блок памяти можно положить в любую кеш-линию.

При поиске адреса кеш должен сравнить его tag с тегами всех линий. Чтобы делать это быстро, используют много компараторов параллельно.

### Плюсы:

- максимальная свобода размещения;
- меньше конфликтных промахов;
- можно выбирать наиболее выгодную линию для вытеснения.

### Минусы:

- много компараторов и управляющей логики;

- большая площадь и энергопотребление;
- сложный механизм поиска и замещения.

Поэтому полностью ассоциативный подход выгоден только для сравнительно небольших структур.

---

## 169. Каковы особенности реализации кеша с прямым отображением?

В кэше с прямым отображением каждому блоку памяти соответствует ровно одна возможная кеш-линия.

Из адреса берётся **index**, который сразу указывает нужную линию. Затем сравнивается только один **tag**.

**Плюсы:** простая схема, один компаратор, быстрый поиск и мало логики.

**Минус:** два разных блока с одинаковым **index** постоянно вытесняют друг друга, даже если в других линиях кеша есть свободное место. Это создаёт конфликтные промахи.

Механизм выбора жертвы здесь фактически не нужен: если линия занята, новый блок может попасть только туда и автоматически вытесняет старый.

---

## 170. Что такое множественно-ассоциативный кеш и что определяет уровень множественности?

**Множественно-ассоциативный кеш** — компромисс между прямым и полностью ассоциативным кешем.

Кеш делится на наборы (**sets**). По **index** выбирается один набор, а внутри него блок может находиться в любой из нескольких линий — **ways**.

Например, **4-way** означает, что для каждого **index** существует четыре возможных места.

При поиске кеш параллельно сравнивает tag во всех ways выбранного набора. Если все места заняты, механизм замещения выбирает одну линию для вытеснения.

Уровень множественности — это и есть **число ways в одном наборе**.

---

## 171. Как связана ассоциативность кеша с механизмом вытеснения и замещения?

Замещение требуется только тогда, когда у блока есть **несколько допустимых мест** в кеше и нужно выбрать, какое из них освободить.

В direct-mapped cache выбора нет: адрес однозначно задаёт линию, поэтому полноценная политика замещения не нужна.

В множественно-ассоциативном кеше выбираем жертву среди ways одного набора.

В полностью ассоциативном — потенциально среди всех линий кеша.

То есть чем выше ассоциативность, тем больше свобода размещения и тем важнее алгоритм, который решает, **какую кеш-линию выгоднее вытеснить**.

---

## 172. Как работает механизм вытеснения и замещения LRU в кеше?

**LRU — Least Recently Used**: вытесняется линия, которой не пользовались дольше остальных.

Концептуально кеш должен помнить «время последнего доступа» к каждой линии. При чтении или записи метка линии обновляется. Когда место заканчивается, выбирается линия с самым старым значением.

Идея хорошо соответствует временной локальности: если к данным давно не обращались, вероятность скорого повторного обращения считается ниже.

Проблема — точный LRU дорог в железе: нужно постоянно обновлять порядок использования, сравнивать значения и учитывать переполнение

счётчиков. Поэтому в реальных кешах часто используют приближённые варианты.

---

## 173. Как работает механизм вытеснения и замещения PLRU в кеше?

**PLRU — Pseudo-LRU** приближённо определяет давно неиспользуемую линию, но требует гораздо меньше логики, чем точный LRU.

Один из вариантов строится как бинарное дерево. Листья соответствуют кеш-линиям, а внутренние узлы хранят по одному биту.

Биты показывают, в какую сторону дерева сейчас следует идти при выборе кандидата на вытеснение. После обращения к линии биты на пути обновляются так, чтобы в будущем предпочтение отдавалось другой ветви.

Для  $N$  линий требуется примерно  $N - 1$  бит состояния.

PLRU не гарантирует абсолютно точный LRU, но обычно даёт близкий результат значительно дешевле.

---

## 174. Почему необходимо использование многоуровневой иерархии кеш-памяти в современных процессорах?

Один кеш не может быть одновременно **очень большим, очень быстрым и дешёвым**. Чем больше структура, тем сложнее сделать её с минимальной задержкой.

Поэтому используют несколько уровней:

- маленький и максимально быстрый кеш рядом с ядром;
- более крупные, но более медленные уровни дальше;
- затем основную память.

Если данных нет в L1, процессор ищет их в L2, затем L3 и только потом в RAM.

Многоуровневая организация позволяет получить небольшой средний latency, не расходуя огромную площадь на один гигантский быстрый

кеш. Разные уровни также могут отличаться по разделённости, включению и общности между ядрами.

---

## 175. В чём разница между разделённым и унифицированным кешем и где они применяются?

**Разделённый кеш** физически разделяет кеш инструкций и кеш данных.

Это позволяет одновременно читать следующую инструкцию и работать с данными, уменьшая структурные конфликты. Кроме того, каждый кеш можно оптимизировать под свой тип обращений.

Минус — объём разделён жёстко: свободное место в кеше инструкций нельзя отдать данным и наоборот.

**Унифицированный кеш** хранит инструкции и данные вместе. Его объём используется гибче, но разные типы обращений конкурируют за общий ресурс.

На практике нижние уровни, особенно L1, часто разделены, а более крупные уровни могут быть унифицированными.

---

## 176. В чём разница между включающим и исключаящим кешем и где они применяются?

Во **включающей (inclusive)** иерархии содержимое верхнего уровня дублируется на нижнем: если линия есть в L1, она также есть в L2/L3.

**Intel (Core, Xeon)**. сильно упрощает жизнь многоядерным системам: чтобы проверить, нет ли у соседа нужных данных, процессору достаточно заглянуть в общий L3, не дёргая мелкие частные кэши L1.

Плюс — проще проверять наличие данных и поддерживать когерентность.

Минус — часть общей ёмкости тратится на дубликаты.

В **исключающей (exclusive)** иерархии одна линия старается находиться только на одном уровне.

**AMD (Zen / Ryzen)**. Так как у AMD исторически кэши L2 были меньше, им было критично не тратить драгоценное место на дубликаты, а суммировать объёмы

Плюс — полезная ёмкость примерно складывается:  $L1 + L2 + L3$ .

Минус

— промахи требуют перемещения линий между уровнями, а поиск и поддержание когерентности становятся сложнее.

Выбор зависит от архитектуры и приоритетов конкретного процессора.

---

## 177. В чём разница между частным и общим кешем и как они применяются в многоядерных процессорах?

**Частный кеш** доступен только одному ядру. Он расположен близко к ядру, имеет маленькую задержку и не создаёт очередь из запросов других ядер.

На **верхних уровнях — L1 и L2**

Минус — одинаковые данные могут дублироваться в частных кешах, а изменения требуют протокола когерентности.

**Общий кеш** доступен нескольким ядрам. Он позволяет динамически использовать весь объём и не обязательно хранить несколько копий одной линии.

На **самом нижнем уровне — кэш L3**

Минус — ядра конкурируют за доступ, поэтому нужна арбитражная логика и задержка обычно выше.

Типичная система комбинирует подходы: быстрые верхние уровни делают частными, а крупный нижний уровень — общим.

---

## 178. Что представляет собой типовой многоуровневый кеш: L1, L2, L3, L4? Каковы их назначение, тип ячеек, классификация?

Кеши процессора обычно строятся на **SRAM**, но уровни отличаются объёмом, скоростью и способом совместного использования.

- **L1** — самый маленький и быстрый. Обычно частный для ядра и часто разделён на инструкции и данные.
- **L2** — больше и медленнее L1. Может быть частным или разделяться между несколькими ядрами.

- **L3** — ещё больше и обычно общий для группы или всех ядер. Он медленнее, но уменьшает обращения в RAM.
- **L4** — встречается редко, чаще в крупных серверных или специализированных системах.

Чем дальше уровень от ядра, тем обычно больше объём и выше задержка доступа.

---

## 179. Что такое когерентность кеш-памяти и каково её значение для многоядерных систем? Какие существуют возможные состояния кеш-линий?

**Когерентность кешей** означает, что разные ядра видят согласованные и актуальные значения общих данных.

Проблема возникает потому, что одна и та же строка RAM может одновременно находиться в кешах нескольких ядер. Если одно ядро её изменит, остальные копии могут устареть.

В протоколе **MOESI** используются состояния:

- **Modified:** линия изменена, свежая копия только здесь, RAM устарела.
- **Owned:** это ядро владеет актуальной изменённой копией и может отдавать её другим.
- **Exclusive:** копия только здесь, но совпадает с RAM.
- **Shared:** актуальная копия есть у нескольких ядер, обычно доступна для чтения.
- **Invalid:** линия недействительна.

При записи в Shared остальные копии должны быть инвалидированы, после чего пишущее ядро получает право менять данные.

---

## 180. Как происходит обмен информацией о кеш-линиях через справочник? Как этот подход масштабируется?

При СПРАВОЧНОМ подходе существует центральный или распределённый справочник, который знает состояние кеш-линий.

Для каждой линии он может хранить:

- признак, изменена ли она относительно RAM;
- **presence bits** — в кешах каких ядер сейчас есть копия.

Если ядро хочет читать или изменять линию, оно обращается к справочнику. Справочник определяет, где лежит актуальная копия, и отправляет сообщения только нужным ядрам.

Плюс — подход хорошо масштабируется по сравнению с общей ширококестательной шиной.

Минус — сам справочник требует памяти и логики и при неудачной организации может стать **центральным узким местом**.

---

## 181. Как происходит обмен информацией о кеш-линиях через отслеживание? Как этот подход масштабируется?

**Отслеживание (Snooping / Прослушивание шины)** — это децентрализованный подход к поддержке когерентности кэшей.

Контроллер кэша каждого ядра непрерывно **«слушает» общую системную шину**, по которой другие ядра общаются с памятью.

- Когда Ядро №1 хочет **записать** данные в свою кэш-строку, оно выбрасывает в общую шину ширококестательный сигнал: **«Я меняю адрес X»**.
- Все остальные ядра, «услышав» этот адрес на шине, сверяют его со своими частными кэшами.
- Если Ядро №2 находит у себя копию этой строки, оно мгновенно помечает её как **недействительную (Invalid)**.
- Теперь, если Ядру №2 снова понадобятся эти данные, оно будет вынуждено скачать их обновленную версию.

Плюс — механизм концептуально простой и не требует отдельного справочника.

Минус — все кеши должны постоянно слушать общую шину. При росте числа ядер увеличиваются трафик и объём логики.

Поэтому Отслеживание хорошо работает на относительно небольшом числе ядер, но **масштабируется ужасно. Трафик на шине растёт пропорционально квадрату числа ядер ( $O(N^2)$ )**

---

## 182. Как происходит обмен информацией о кеш-линиях через перехват? Как этот подход масштабируется?

**Перехват** развивает идею snooping: кеши наблюдают не только за адресами, но и за **данными**, передаваемыми по общей шине.

Если одно ядро отправляет актуальную кеш-линию, другое ядро может перехватить данные напрямую и сразу обновить свою копию, не дожидаясь отдельного чтения из RAM.

Это удобно для протоколов вроде MOESI, где владелец Modified/Owned линии может передавать свежие данные другим кешам.

Минус тот же, но сильнее: кроме адресной инфраструктуры нужна ещё общая передача данных. трафик на шине растёт пропорционально квадрату числа ядер\*\*  
(( $O(N^2)$ )))

---

## 183. Какие существуют типы кеш-промахов (3C + 1: cold, capacity, conflict, coherence)? Какие способы смягчения характерны для каждого?

Классическая классификация — **3C + 1**.

1. **Cold**: первое обращение к блоку, его ещё никогда не было в кеше.

Смягчается предвыборкой (**prefetch**).

- Процессор пытается предугадать шаги алгоритма и заранее скачивает данные из ОЗУ в кэш наперёд, пока ядро занято другими командами.

2. **Capacity**: рабочий набор больше кеша, поэтому нужные данные вытесняются.

*По-умному (софт)*: Оптимизировать код алгоритма, чтобы программа пережёвывала данные маленькими порциями, влезаящими в кэш.

*По-богатому (хард)*: Физически увеличивать объём кэш-памяти на кристалле (например, наращивать L3-кэш).

3. **Conflict**: несколько блоков конкурируют за одни и те же линии, хотя свободное место в кеше есть.

**Повышать ассоциативность кэша**

(переходить к множественно-ассоциативному кэшу). Чем больше каналов (*Ways*) в наборе, тем больше подстраховочных мест для адресов с одинаковым индексом.

4. **Coherence:** линия инвалидирована из-за записи другого ядра или DMA.

Возникают только в многоядерных системах. Ядро №1 хотело прочесть строку у себя в кэше, но Ядро №2 только что изменило эти данные и через протокол когерентности принудительно сделало строку соседа **недействительной (Invalid)**

У разных типов разная причина, поэтому одного универсального способа убрать все промахи нет.

---

## 184. Что такое CAP-теорема? Какова область её применения и как она применяется к когерентности кешей?

**CAP-теорема** относится к распределённым системам и говорит о компромиссе между тремя свойствами: Consistency, Availability и Partition tolerance.

В общем случае при сетевом разделении нельзя одновременно гарантировать все три свойства: приходится выбирать, что сохранить при конфликте требований.

В курсе эту идею связывают с когерентностью кешей как с частным случаем согласования нескольких копий данных.

Для кешей внутри одного компьютера обычно не рассматривают полноценное сетевое **partition tolerance**, поэтому основной практический вопрос сводится к тому, как одновременно обеспечить актуальность данных и доступ к ним при параллельной работе ядер.

---

## 185. Что означают термины "Consistency", "Availability" и "Partition tolerance" в CAP-теореме? Опишите их.

В CAP используются три свойства.

- **Consistency:** каждое успешное чтение получает последнюю актуальную запись либо ошибку. Все участники системы согласованы относительно значения.
- **Availability:** каждый запрос к работающему узлу получает ответ, хотя это не гарантирует, что ответ содержит самую свежую запись.

- **Partition tolerance:** система продолжает работать даже при потере связи между частью узлов.

Смысл CAP в том, что при реальном разделении сети приходится выбирать между строгой согласованностью и постоянной доступностью: одновременно полностью гарантировать все три свойства нельзя.

---

## 186. Как кеш влияет на производительность памяти? Какие существуют способы оптимизации доступа, включая AoS и SoA?

Кеш уменьшает среднее время доступа только тогда, когда программа хорошо использует **локальность**. Поэтому производительность зависит не только от алгоритма, но и от расположения данных в памяти.

Два типичных варианта:

- **AoS — Array of Structures:** рядом лежат все поля одного объекта.  $x, y, z$ . Удобно, когда обычно читается объект целиком или одна запись.
- **SoA — Structure of Arrays:** отдельно лежат массивы каждого поля.  $x, x, x, x$ . Удобно, когда обрабатывается одно поле сразу для множества объектов.

Например, если нужно вычислять среднее значение  $x$  для миллионов точек, SoA плотнее размещает нужные  $x$  и лучше использует кеш-линии. Если часто нужна вся точка целиком, AoS может быть выгоднее.

---

## 187. Как кеш работает с виртуальными и физическими адресами?

Процессы обычно работают с **виртуальными адресами**, которые затем транслируются в физические. Возникает выбор, по какому адресу индексировать кеш.

**Виртуальный адрес** можно использовать раньше — без ожидания трансляции, поэтому доступ быстрее. Но одинаковые виртуальные адреса

разных процессов могут означать разные физические данные, что создаёт коллизии при переключении контекста.

**Физический адрес** однозначен для всей системы и проще для согласования кешей, но сначала требуется трансляция адреса.

самые быстрые уровни кэша используют виртуальную адресацию полностью или частично, а более низкие уровни чаще опираются на физический адрес.

---

## 188. Что такое HyperThreading? Как два логических ядра делят физический кеш и как это влияет на производительность?

**HyperThreading** механизм позволяющий одному физическому ядру поддерживать два логических потока за счёт нескольких наборов архитектурных регистров.

Когда один поток простаивает, например ждёт данные после cache miss, ядро может использовать исполнительные ресурсы для другого потока.

При этом логические ядра не являются двумя полноценными физическими ядрами: они делят исполнительные блоки и **одни и те же кеши**.

Поэтому HyperThreading повышает загрузку ресурсов и обычно увеличивает суммарную производительность, но не в два раза. Если оба потока одновременно упрутся в кеш или память, они начинают конкурировать и выигрыш уменьшается.

---

## 189. В чём суть уязвимостей Meltdown и Spectre? Как кеш и спекулятивное исполнение создают канал утечки данных?

Обе уязвимости используют то, что процессор **спекулятивно выполняет инструкции**, результат которых потом может быть отменён архитектурно, но изменения состояния кеша уже остаются.

**Meltdown:** процессор успевает спекулятивно обратиться к секретным данным, к которым процесс не должен иметь доступа.

- Обычная вредоносная программа пишет команду: *«прочитать секретный байт из памяти ядра операционной системы»*. Архитектурно это делать строго запрещено.
- **Спекулятивный прыжок:** до проверки прав доступа.
- Следующая спекулятивная команда берёт этот украденный секретный байт (допустим, его значение равно 4) и использует его как индекс для чтения массива данных: *«прочитать 4-ю строку нашего массива»*. Эта строка автоматически **загружается из медленной ОЗУ в быстрый кэш**.
- **Архитектурный откат:** Блок защиты наконец просыпается, видит нарушение прав, кричит *«Сюда нельзя!»*, отменяет все спекулятивные команды в регистрах и выдаёт программе ошибку (Segmentation Fault). Секретного байта в регистрах приложения больше нет.
- **Чтение через тайминг:** Вредоносная программа перехватывает ошибку, не давая себе закрыться, и начинает в цикле замерять скорость чтения всех строк своего массива с помощью высокоточного таймера:
  - 1-я строка читается медленно (200 тактов из ОЗУ).
  - 2-я строка читается медленно (200 тактов).
  - 3-я строка читается медленно (200 тактов).
  - **4-я строка считывается мгновенно (за 2 такта)!**
- **Финал:** Программа понимает: раз 4-я строка оказалась в кэше, значит спекулятивный код успел прочесть секрет, и этот секрет был равен четырём. Атака повторяется миллионы раз, позволяя выкачать вообще всю память ядра компьютера, включая пароли и ключи шифрования.

**Spectre:** злоумышленник обучает branch predictor ошибочному переходу. Процессор спекулятивно выполняет запрещённый по логике программы и снова кодирует секрет через состояние кеша.

То есть кеш становится **побочным каналом**, потому что cache hit и cache miss имеют разное время доступа.

---

## 190. Что представляет собой концепция уровневой организации компьютерных систем и каковы её преимущества?

**Уровневая организация** делит сложную систему на относительно независимые уровни. Каждый уровень решает свою задачу, предоставляет понятный интерфейс верхнему уровню и скрывает детали внутренней реализации.

Преимущества:

- уменьшается сложность разработки;
- разные команды могут работать независимо;
- реализацию одного уровня можно менять, сохраняя интерфейс;
- появляются стандартные границы и возможность переиспользования;
- систему проще развивать модульно.

Главная идея — верхнему уровню не нужно постоянно знать, **как именно** нижний уровень реализует свою функцию, пока соблюдается контракт между ними.

---

## **191. Какие примеры уровневой организации вычислительных систем вы знаете? Чем они отличаются друг от друга?**

Примеры уровневой организации зависят от того, что именно мы рассматриваем.

**Для компьютера в целом:**

прикладные программы →

ОС →

ISA/архитектура →

микроархитектура →

логика →

цифровые схемы →

аналоговые схемы →

физические устройства.

**Для сетей:**

модель OSI разделяет передачу данных по функциям разных уровней.

прикладной

представления

сеансовый

транспортный

сетевой

канальный

физический

между уровнями нет их "горизонтальной" связанности. Т.е. когда у нас идет пересылка HTTP пакета, он всегда идет на всех уровнях сразу.

**Для программного проекта:** библиотеки, framework, API, язык, ОС, isa и виртуализированные ресурсы тоже образуют уровни.

Lava Flow - антипаттерн проектирования (как делать не надо), когда проект можно представить как извергающийся вулкан. Т.е. во время разработки проекта вулкан периодически извергается (приходит новый разработчик, появляется новое требование, меняется подход), далее из него изливается лава (пишется новый код там и так далее) вот только вся эта лава (и вся работа над проектом) накладывается поверх предыдущих, уже окаменевших слоев. Никто не подвергает сомнению старый код, никто его не рефакторит, просто фигачат сверху больше и больше.

Различие — в объекте декомпозиции. Но принцип одинаковый: каждый уровень скрывает внутренние детали и общается с соседними через определённый интерфейс.

---

## 192. Что такое явление разделения на уровни (disaggregation) и каково его современное положение? Что было до него?

Раньше компьютерные компании часто строили **вертикально интегрированные системы**: сами делали чипы, компьютер, ОС и прикладное ПО.

**Disaggregation** — метод при котором разработка системы разделяется между специализированными компаниями.

Теперь один производитель может делать процессоры, другой — компьютеры, третий — ОС, четвёртый — приложения. За счёт стандартных интерфейсов один и тот же софт может работать на разных платформах.

Плюс — каждая компания специализируется на своём уровне, растут конкуренция и скорость развития.

Минус — теряется часть возможностей сквозной оптимизации, которую имеет разработчик, контролирующий всю систему от железа до

## 193. Почему необходимо нарушение уровневой иерархии в современных системах? Приведите примеры.

Абстракции удобны, пока нас устраивает обычная производительность. Но при жёстких требованиях по скорости, энергии или реальному времени верхний уровень уже не может полностью игнорировать устройство нижнего.

Поэтому иерархию иногда сознательно «пробивают».

Примеры:

- программист меняет структуру данных под размер кеш-линии;
- выбирает AoS или SoA с учётом кеша;
- использует SIMD или специальные инструкции;
- пишет ассемблерные вставки;
- отключает или обходит механизмы, создающие непредсказуемую задержку.

То есть оптимизация требует знания того, **что происходит под абстракцией**, хотя формально верхний уровень не должен об этом заботиться.

---

## 194. Какие проблемы возникают при обеспечении реального времени в современных процессорах?

Реальное время требует не просто высокой средней скорости, а **предсказуемой верхней границы задержки**.

Современные процессоры устроены наоборот: многие оптимизации делают среднее выполнение быстрее, но вносят непредсказуемость.

Источники вариативности:

- кеши и cache miss;
- виртуальная память и page fault;
- branch prediction и спекулятивное выполнение;

- сложные конвейеры;
- внеочередное исполнение;
- конкуренция за общие ресурсы.

Поэтому одна и та же инструкция или операция может занимать разное время. Для систем жёсткого реального времени часть механизмов приходится ограничивать, отключать или учитывать специальными методами.

---

## 195. Что такое низкоуровневый параллелизм? Приведите примеры.

**Низкоуровневый параллелизм** — параллельное выполнение, которое организует сама микроархитектура процессора и которое обычно почти не видно прикладной программе.

Примеры:

- **конвейеризация:** разные стадии нескольких инструкций работают одновременно;
- **несколько функциональных блоков:** например, отдельные ALU и FPU могут выполнять независимые операции параллельно;
- **суперскалярность:** процессор выдаёт несколько независимых инструкций на исполнение за такт;
- **VLIW:** несколько операций заранее объединены в одну широкую команду.

Главная цель — повысить количество полезной работы за такт, не заставляя программиста вручную управлять каждым исполнительным блоком.

---

## 196. Что такое суперскалярные процессоры и каковы принципы их работы, достоинства и недостатки?

**Суперскалярный процессор** имеет несколько исполнительных блоков и пытается выполнять несколько независимых инструкций параллельно.

Инструкции читаются и декодируются, помещаются в очередь, после чего аппаратный диспетчер анализирует зависимости и отправляет готовые

команды на свободные ALU/FPU. Результаты затем приводятся к правильному программному порядку.

**Достоинства:** высокая производительность, хорошая загрузка блоков, совместимость с обычным последовательным машинным кодом.

**Недостатки:** сложная логика зависимостей и переупорядочивания, большое энергопотребление, влияние cache miss и ветвлений, менее предсказуемое время выполнения.

То есть «умность» перенесена в железо: процессор сам ищет параллелизм во время работы.

---

## 197. Как пошагово выполняются инструкции в суперскалярном процессоре?

Типовая схема делится на три части.

1. **Вход — in-order.** Fetch читает инструкции по порядку, Decode декодирует, Dispatch распределяет их по станциям резервирования.
2. **Исполнение — out-of-order.** Инструкция ждёт, пока готовы её операнды и нужный функциональный блок. Готовые независимые команды могут обогнать более раннюю зависшую команду.
3. **Завершение — снова in-order.** Результаты временно попадают в буфер переупорядочивания. Архитектурное состояние обновляется строго в исходном порядке.

Так процессор получает параллелизм внутри, но снаружи сохраняет видимость обычного последовательного выполнения программы.

---

## 198. Что такое VLIW-процессоры? Какими достоинствами и недостатками они обладают?

**VLIW — Very Long Instruction Word:** одна широкая машинная команда содержит сразу несколько операций для разных функциональных блоков.

В отличие от суперскаляра, параллелизм в основном ищет **компилятор заранее**. Он определяет независимые операции, назначает их на блоки

и формирует широкую команду.

**Плюсы:** процессор проще, меньше аппаратной диспетчеризации и энергопотребления, декодер проще, выполнение более предсказуемо.

**Минусы:** очень сложный компилятор, широкие регистры и каналы данных, простой сразу нескольких блоков при задержке одной операции, пустые слоты/NOP, слабая переносимость бинарного кода между разными ширинами микроархитектуры.

VLIW особенно хорошо подходит для регулярных вычислений с заранее понятным параллелизмом.

---

## 199. Как соотносятся суперскалярные и VLIW процессоры применительно к разным классам задач?

Это две крайности распределения работы между компилятором и процессором.

**Суперскаляр** лучше для универсального кода: ветвлений много, зависимости проявляются во время работы, поведение памяти непредсказуемо. Железо само динамически переставляет инструкции и сохраняет совместимость с ISA.

**VLIW** лучше для регулярных специализированных вычислений, где компилятор заранее хорошо видит параллелизм: DSP, численные вычисления, некоторые ускорители.

VLIW проще и энергоэффективнее, но сильнее зависит от качества компилятора и структуры задачи.

Суперскаляр тратит больше аппаратных ресурсов, зато лучше адаптируется к непредсказуемому потоку инструкций.

---

## 200. Какие существуют промежуточные варианты между суперскалярным и VLIW процессором? Какие проблемы они решают?

Суперскаляр и VLIW — две крайности того, **кто именно ищет параллелизм**: процессор во время исполнения или компилятор заранее.

Для параллельного выполнения можно выделить четыре шага:

1. **Code generation** — сгенерировать машинный код.
2. **Instruction grouping** — найти независимые инструкции и объединить их в группы.
3. **Functional unit assignment** — назначить инструкции конкретным исполнительным блокам.
4. **Initiation timing** — определить момент запуска каждой операции.

**Суперскаляр:** компилятор в основном делает только первый шаг, остальное динамически выполняет железо.

**EPIC:** компилятор дополнительно группирует независимые команды, а процессор сам назначает их блокам и выбирает момент запуска.

**Dynamic VLIW:** компилятор ещё и назначает функциональные блоки, а железо в основном управляет временем запуска и может реагировать на непредсказуемые задержки.

**VLIW:** почти всё решает компилятор заранее.

Промежуточные варианты пытаются совместить простоту и энергоэффективность VLIW с динамической адаптацией суперскаляра.

---

## 201. Что такое барьеры памяти и в каких ситуациях они применяются?

**Барьер памяти (memory fence)** — специальная инструкция, которая ограничивает переупорядочивание обращений к памяти.

Современный процессор и компилятор могут менять порядок чтений и записей, если считают, что это ускорит выполнение. Для одного потока результат часто остаётся тем же, но другой поток или устройство может увидеть операции в неожиданном порядке.

Барьер задаёт границу: обращения к памяти **до барьера** должны быть выполнены раньше обращений **после барьера** в том объёме, который задаёт конкретный тип fence.

Типичный пример:

1. поток записывает новые данные;
2. затем выставляет флаг `ready = 1`.

Без барьера другой процессор теоретически может увидеть флаг раньше самих данных. Fence запрещает такое опасное переупорядочивание.

Барьеры особенно важны в многопоточном коде, драйверах, низкоуровневой синхронизации и работе с Memory Mapped IO.

---

## 202. Как суперскалярное исполнение и store buffers влияют на порядок обращений к памяти в многоядерных системах? Что такое барьер памяти (fence) и зачем он нужен?

В суперскалярном процессоре инструкции могут выполняться **не по исходному порядку**: независимые команды уходят на разные исполнительные блоки и заканчиваются в разное время.

Особенно это заметно для операций памяти. Чтения могут фактически происходить в другом порядке, чем записано в программе.

**Store buffer** — буфер между исполнением инструкции записи и кешем/памятью. Процессор может положить результат записи в буфер и продолжить работу, не ожидая медленную память. Это ускоряет код, но для других ядер запись может стать видимой позже.

В итоге разные ядра могут наблюдать обращения к памяти не в том порядке, который виден в исходном коде.

**Fence** задаёт точку упорядочивания: определённые чтения и записи после него нельзя сделать видимыми раньше операций до него.

Он нужен, когда корректность программы зависит именно от порядка наблюдаемых операций памяти.

---

## 203. Какие ключевые элементы составляют структурное программирование? В чём отличие от кода на ассемблере?

**Структурное программирование** описывает программу через понятные управляющие блоки, а не через произвольные переходы между машинными инструкциями.

Три базовые конструкции:

1. **Последовательность** — операции выполняются одна за другой.
2. **Ветвление** — в зависимости от условия выполняется один из нескольких блоков ( `if/else` ).
3. **Цикл** — блок повторяется, пока выполняется условие.

Дополнительно код разбивается на именованные процедуры и функции.

На ассемблере в основе лежат инструкции процессора, метки и переходы `jump`. Те же конструкции можно реализовать вручную, но структура программы не задаётся самим языком настолько явно.

Язык высокого уровня поднимает разработчика от модели «какую инструкцию и куда прыгнуть» к модели «какой логический блок должен выполняться». Затем компилятор уже переводит эти конструкции в переходы и машинные инструкции.

---

## 204. Как реализуются процедуры на уровне ассемблера? Что такое `call/return` и зачем нужен стек вызовов?

Процедура нужна, чтобы один блок машинного кода можно было вызывать из разных мест, не копируя его инструкции каждый раз.

Есть несколько вариантов реализации:

- **inline** — инструкции процедуры просто вставляются в место вызова; вызова как такового нет, зато код дублируется;
- **goto/jump** — вручную переходим к процедуре и отдельно решаем, куда потом возвращаться;
- **call/return** — специальная аппаратно поддерживаемая схема.

При `call` процессор сохраняет **адрес возврата** и переходит к процедуре. При `return` этот адрес восстанавливается, и выполнение продолжается после места вызова.

Если процедуры могут быть вложенными, одного регистра адреса возврата недостаточно: каждый новый вызов затрёт предыдущий. Поэтому

адреса возврата сохраняют в **стеке вызовов**.

Стек также обычно хранит аргументы, локальные переменные и сохранённые регистры — всё это образует **stack frame** конкретного вызова.

---

## 205. Что такое реентерабельность и почему она важна? Приведите примеры.

**Реентерабельная процедура** может быть безопасно вызвана ещё раз до завершения предыдущего вызова.

Для этого разные вызовы не должны портить состояние друг друга. Обычно каждый вызов получает собственные локальные данные и собственный **stack frame**, а разделяемое изменяемое состояние либо не используется, либо синхронизируется.

Главные примеры:

- **рекурсия**: функция вызывает сама себя, поэтому одновременно существуют несколько её незавершённых вызовов;
- **обработчики прерываний**: обработчик может быть прерван новым событием и потенциально вызван повторно;
- функции, используемые несколькими потоками.

Нереентерабельный код часто хранит временные данные в одной общей глобальной области. Второй вызов тогда может перезаписать данные, которые ещё нужны первому.

То есть реентерабельность — свойство, позволяющее безопасно иметь несколько активных экземпляров одной и той же процедуры.

---

## 206. Как реализуется механизм рекурсии?

Классический способ реализации рекурсии — **стек вызовов**.

При каждом рекурсивном `call` создаётся новый **stack frame**. В нём хранятся данные именно этого вызова:

- аргументы;

- локальные переменные;
- сохранённые регистры;
- адрес возврата.

Поэтому вызовы одной функции не мешают друг другу. Когда достигается базовый случай, начинаются `return`: верхний `frame` удаляется, восстанавливается адрес возврата, затем снимается следующий и так далее.

В исходном материале также приведён второй вариант — **статически заранее выделить память** под каждый возможный уровень вложенности. Тогда аппаратный стек не нужен, но максимальная глубина рекурсии должна быть известна заранее и жёстко ограничена.

Стековый вариант универсальнее: память под глубину рекурсии выделяется динамически по мере реальных вызовов.

---

## 207. В чём заключается проблема распределения регистров и как работает алгоритм раскраски?

При компиляции хочется держать как можно больше промежуточных значений в регистрах, потому что обращения к памяти значительно дороже. Но физических регистров мало.

Компилятор сначала может представить, что виртуальных регистров сколько угодно. Затем строится **граф конфликтов**:

- вершина — виртуальный регистр или переменная;
- ребро между двумя вершинами есть, если их значения одновременно **живы**, то есть оба ещё понадобятся программе.

Дальше граф раскрашивается.

**Цвет = физический регистр.** Связанные вершины не могут иметь один цвет, потому что их значения нужны одновременно.

Если граф нельзя раскрасить доступным числом цветов, часть значений приходится **spill** — временно выгружать в память.

Таким образом задача распределения регистров превращается в задачу раскраски графа с ограниченным количеством цветов.

---

## 208. Как условный оператор может быть реализован через полиморфизм и замыкания? Пример на Smalltalk.

В Smalltalk условие можно реализовать **без специальной инструкции `if` в синтаксисе**, используя объекты, полиморфизм и замыкания.

`True` и `False` рассматриваются как разные объекты/классы, реализующие один и тот же метод, например `ifTrue:ifFalse:`.

Ему передаются **два блока кода**:

- ветка «тогда»;
- ветка «иначе».

Эти блоки оформлены как замыкания, поэтому при передаче они **не выполняются сразу**.

Дальше работает полиморфизм:

- объект `True` вызывает только первый блок;
- объект `False` вызывает только второй.

Замыкания здесь критичны. Если передать обычные заранее вычисляемые выражения, обе ветки выполнялись бы ещё до вызова метода и смысл условия исчез бы.

Итого: выбор ветки превращается из специальной управляющей конструкции в обычный полиморфный вызов метода.

---

## 209. Что такое ленивые вычисления и как они связаны с реализацией условного оператора?

**Ленивые вычисления (Lazy Evaluation)** — стратегия, при которой выражение вычисляется не в момент его создания, а только тогда, когда результат действительно понадобился.

Например, аргумент функции можно не вычислять до первого реального обращения к нему внутри функции.

Условный оператор по смыслу тоже требует ленивости.

Если есть `if condition: A else: B`, должна выполняться **только одна** ветка. Если заранее вычислить и `A`, и `B`, то побочные эффекты обеих веток уже произойдут, даже если результат одной потом выбросить.

Поэтому ветки должны быть представлены как отложенные вычисления: замыкания, блоки, `thunk` или другой механизм.

В примере со `Smalltalk` обе ветки передаются объекту `True/False` как замыкания, а уже выбранное замыкание вычисляется позже.

---

## 210. Как `async` / `await` связан с кооперативной многозадачностью на уровне языка программирования?

`async/await` — языковой механизм для удобной реализации кооперативной многозадачности.

Главный принцип кооперативной модели: задача сама добровольно отдаёт управление, а не прерывается планировщиком в произвольный момент.

Функция `async` может выполняться до точки `await`.

Когда она встречает операцию, которую надо долго ждать, например сеть, диск или таймер, она:

1. иницирует операцию;
2. сохраняет своё состояние;
3. добровольно отдаёт управление циклу событий;
4. позволяет выполнять другую готовую задачу;
5. после завершения ожидания продолжает работу после `await`.

Поэтому множество задач может продвигаться в одном физическом потоке без обязательного создания отдельного системного потока для каждой.

`await` — это фактически **явная безопасная точка переключения** между кооперативными задачами.

---

## 211. Что такое **FPGA** и каковы его преимущества и недостатки?

**FPGA (Field-Programmable Gate Array, ПЛИС)** — готовая микросхема, внутреннюю цифровую схему которой можно переконфигурировать под конкретную задачу.

По идее она находится между CPU и ASIC:

- **CPU** универсален и выполняет последовательность команд;
- **ASIC** физически спроектирован под одну задачу и очень эффективен;
- **FPGA** уже произведён как универсальная заготовка, но его логика настраивается пользователем.

#### Преимущества FPGA:

- высокая степень параллелизма;
- можно строить специализированный datapath;
- перепрограммируемость;
- для небольших партий намного дешевле разработки собственного ASIC;
- обычно выше эффективность конкретной задачи, чем на CPU.

#### Недостатки:

- ниже плотность и эффективность, чем у ASIC;
- ограничены ресурсами готового кристалла;
- проектирование сложнее обычного программирования;
- при огромном серийном производстве ASIC может оказаться дешевле в пересчёте на одно устройство.

---

## 212. Какие этапы включает в себя синтез для FPGA?

Разработчик FPGA описывает не последовательность обычных команд, а **цифровую схему**.

Упрощённая цепочка из исходного материала такая:

1. Схема описывается на **HDL** на уровне RTL: регистры, комбинационная логика и связи между ними.
2. Синтезатор преобразует RTL в более низкоуровневое логическое представление — по сути сеть логических функций (**Generic Boolean / netlist**).

3. Полученная логика отображается на реальные ресурсы конкретной FPGA: LUT, триггеры, блоки памяти, специальные вычислительные блоки.
4. Затем выбирается физическое размещение блоков и маршрутизация соединений между ними.
5. В результате формируется конфигурация, которая загружается в FPGA и задаёт её конкретную аппаратную структуру.

То есть здесь «компиляция» в итоге создаёт **конфигурацию железа**, а не поток машинных инструкций для CPU.

---

## 213. Как FPGA устроены внутри?

FPGA состоит из большого регулярного массива **конфигурируемых логических блоков** и программируемых соединений между ними.

Основные части:

- **LUT** — таблицы истинности, которые могут реализовать небольшую логическую функцию;
- **триггеры/регистры** — хранят состояние между тактами;
- **switch boxes / routing** — программируемые соединения, связывающие блоки в нужную схему;
- **BRAM** — встроенные блоки памяти;
- специализированные блоки, например умножители, DSP-блоки, сумматоры и другие часто используемые элементы.

Конфигурация задаёт:

1. какую функцию выполняет каждая LUT;
2. какие блоки соединены между собой;
3. где используются регистры и память.

Поэтому одна и та же физическая FPGA после другой конфигурации может стать совершенно другим цифровым устройством.

---

## 214. Как соотносятся CPU и FPGA с точки зрения модели программирования, модели вычислений, модели

# исполнения и элементов микроархитектуры?

## 1. Модель программирования

- **CPU:** императивная — разработчик задаёт последовательность действий на C/C++ и других языках.
- **FPGA:** декларативная — на HDL описывается сама цифровая схема.

## 2. Модель вычислений

- **CPU:** в основном временная (**temporal**) — одни и те же исполнительные блоки последовательно выполняют разные операции.
- **FPGA:** пространственная (**spatial**) — разным операциям можно выделить разные физические участки схемы, работающие параллельно.

## 3. Модель исполнения

- **CPU:** поток инструкций, динамическое управление, конвейер, branch prediction, возможно out-of-order.
- **FPGA:** структура заранее фиксируется конфигурацией; данные идут через аппаратный datapath, часто потоково.

## 4. Микроархитектура

CPU содержит ALU, register file, кешы, предсказатель переходов. FPGA — массив LUT/CLB, регистров, BRAM и программируемой коммутации.

То есть CPU переиспользует универсальное железо во времени, а FPGA настраивает железо в пространстве под конкретный вычислительный поток.

---

## 215. Что такое высокоуровневый синтез (HLS) и какова его роль в современной разработке?

В исходном файле для этого билета ответа нет. Ниже — стандартное объяснение HLS, чтобы билет можно было рассказать.

**HLS (High-Level Synthesis)** — автоматическое преобразование алгоритма, описанного на более высоком уровне, обычно на C/C++ или специализированном языке, в аппаратную RTL-структуру для FPGA/ASIC.

То есть разработчик описывает **что вычислять**, а инструмент HLS пытается построить **как это реализовать железом**:

- какие нужны функциональные блоки;
- где поставить регистры;
- что выполнять параллельно;
- как построить конвейер;
- как организовать память и интерфейсы.

Роль HLS — поднять уровень абстракции и сократить объём ручного HDL.

Плюс — быстрее разработка и проще экспериментировать с архитектурой.

Минус — результат зависит от качества инструмента и ограничений исходного кода; вручную написанный RTL иногда получается эффективнее.

---

## 216. Как высокоуровневый синтез (HLS) автоматизирует процесс выбора микроархитектуры и ускоряет тестирование?

**В исходном файле ответа на этот билет нет. Ниже — стандартное объяснение механизма HLS.**

Один алгоритм можно реализовать множеством микроархитектур.

Например, одну операцию умножения можно выполнять одним умножителем много тактов, поставить несколько умножителей и считать параллельно или глубоко конвейеризовать вычисления.

HLS автоматически исследует такие решения по ограничениям разработчика: частота, latency, throughput, площадь, количество ресурсов.

С помощью директив можно задавать, например:

- `pipeline`;
- `unroll`;
- распределение массивов;
- число экземпляров функциональных блоков.

После изменения директив HLS быстро пересинтезирует вариант и даёт оценки ресурсов и производительности.

Тестирование тоже ускоряется: алгоритм сначала можно проверять как обычную высокоуровневую функцию, а затем сравнивать её поведение с синтезированной RTL-реализацией.

---

## 217. Что представляет собой классификация Флинна и какие классы архитектур она выделяет? Примеры.

Классификация Флинна делит вычислительные архитектуры по двум параметрам:

- сколько существует потоков инструкций;
- сколько существует потоков данных.

Получаются четыре класса:

1. **SISD — Single Instruction, Single Data.** Один поток инструкций работает с одним потоком данных. Классический последовательный процессор.
2. **SIMD — Single Instruction, Multiple Data.** Одна операция одновременно применяется к нескольким данным. Примеры: векторные блоки, многие операции GPU.
3. **MISD — Multiple Instruction, Single Data.** Несколько потоков инструкций работают с одним потоком данных. Редкий класс; может встречаться в специальных отказоустойчивых системах.
4. **MIMD — Multiple Instruction, Multiple Data.** Несколько независимых вычислителей выполняют разные инструкции над разными данными. Типичный современный многоядерный процессор.

Минус классификации — она очень грубая: огромное количество разных современных систем просто попадает в MIMD.

---

## 218. Что такое SIMD и SIMT архитектуры? В чём их различия? Что такое lockstep execution?

**SIMD — Single Instruction, Multiple Data:** одна инструкция одновременно применяется к нескольким элементам данных.

Например, одна векторная команда складывает сразу несколько пар чисел.

**SIMT — Single Instruction, Multiple Threads:** программно мы видим множество отдельных потоков, но аппаратно группы потоков часто исполняют одну инструкцию совместно.

**Lockstep execution** означает, что группа вычислителей движется по программе синхронно: в конкретный момент они выполняют одну и ту же инструкцию над своими данными.

SIMT удобнее при ветвлениях, потому что можно маскировать часть потоков: одна группа временно выполняет ветку `if`, другая выключена, затем наоборот.

Но если потоки одной группы сильно расходятся по ветвям, возникает **divergence** и эффективность падает: разные ветки приходится фактически выполнять по очереди.

---

## 219. Что представляет собой классификация Дункана и чем она отличается от классификации Флинна? Какие проблемы она решает? Что включает верхний уровень классификации?

Классификация Флинна слишком грубая: она смотрит только на число потоков инструкций и данных, поэтому очень разные современные процессоры оказываются в одном классе MIMD.

**Классификация Дункана** развивает эту идею и сильнее учитывает реальную организацию параллельных вычислителей.

В исходном материале её верхний уровень разбивается на три крупные группы:

1. **Синхронные архитектуры** — есть единое или централизованное управление параллельным исполнением.
2. **MIMD-архитектуры** — различаются прежде всего организацией памяти: разделяемая или распределённая.
3. **MIMD-парадигмы** — различаются уже по модели вычислений: MIMD/SIMD, dataflow, reduction, wavefront.

При этом низкоуровневый параллелизм внутри обычного последовательного исполнения Дункан не делает главным критерием.

Смысл классификации — не просто сказать «это SIMD», а показать, **каким именно способом организован параллелизм.**

---

## 220. Какие основные типы синхронных архитектур выделяются по классификации Дункана?

**Синхронные архитектуры** в классификации Дункана — системы, где поведение множества вычислительных элементов координируется единым или централизованным механизмом управления.

В исходном материале выделены:

1. **Векторные архитектуры** — одна операция применяется к целому вектору элементов.
2. **SIMD-архитектуры:**
  - процессорные массивы;
  - ассоциативные массивы.
3. **Систолические массивы** — данные ритмично проходят через сеть вычислительных узлов по общему такту.
4. **Совмещение памяти и вычислений** — часть простых вычислений переносится ближе к месту хранения данных.

Общая идея этих архитектур — получить высокую производительность, когда задача допускает регулярный массовый параллелизм.

Они особенно выгодны для матриц, сигналов, графики и других задач, где много одинаковых или хорошо структурированных операций.

---

## 221. Какие принципы лежат в основе работы векторных архитектур и где они применяются?

**Векторная архитектура** умеет одной командой работать не с одним скаляром, а сразу с **вектором элементов**.

Вместо последовательности  $c[0]=a[0]+b[0]$ ,  $c[1]=a[1]+b[1]$  и т.д. процессор получает одну векторную операцию «сложить два вектора».

Аппаратно операция разбивается по нескольким вычислительным элементам или конвейерно обрабатывает элементы потока.

Плюсы:

- меньше инструкций и накладных расходов на декодирование;
- высокая загрузка вычислительных блоков;
- хорошо используется однотипный параллелизм данных.

Применение: линейная алгебра, обработка сигналов, графика, спектральный анализ, научные вычисления и задачи машинного обучения.

Главное условие эффективности — данные должны допускать массовую однотипную обработку.

---

## 222. Как работают ассоциативные массивы и где они применяются?

**Ассоциативная память** ищет информацию не только по обычному адресу, а **по содержимому или признаку**.

В обычной RAM мы говорим: «прочитай ячейку номер 100». В ассоциативной памяти запрос больше похож на: «найди все записи, у которых поле соответствует этому шаблону».

Сравнение может выполняться сразу со множеством ячеек параллельно, поэтому поиск получается очень быстрым, но аппаратно такая память дороже обычной.

**Ассоциативный массив** объединяет подобный поиск с параллельной обработкой найденных элементов.

Типичный пример применения из исходного материала — **сетевое оборудование**, где нужно очень быстро искать маршруты, правила или другие записи в таблицах.

Главный выигрыш — скорость поиска по ключу/содержимому. Главный недостаток — высокая стоимость и сложность аппаратной реализации.

---

## 223. Как работают систолические архитектуры и где они применяются?

**Систолическая архитектура** — регулярная сеть вычислительных элементов, между которыми данные проходят по цепочке в общем ритме.

На каждом такте узел:

1. получает данные от соседних узлов;
2. выполняет простую локальную операцию;
3. обновляет свой промежуточный результат;
4. передаёт данные дальше.

За счёт этого множество узлов одновременно обрабатывает разные части одного большого вычисления — как аппаратный конвейер по двум измерениям.

Особенно удобно так реализовывать **умножение матриц**: элементы строк и столбцов проходят через сетку, а каждый узел накапливает произведения.

Применения:

- машинное обучение;
- матричные ускорители;
- обработка изображений и видео;
- цифровая обработка сигналов.

Минус — архитектура очень эффективна только тогда, когда алгоритм хорошо отображается на регулярную сеть узлов.

---

## 224. Какие основные типы MIMD-архитектур выделяются по классификации Дункана?

В MIMD несколько вычислителей могут независимо выполнять **разные потоки инструкций над разными данными**.

В классификации Дункана ключевой вопрос для MIMD — **как организована память**.

Выделяются два основных типа:

1. **Архитектуры с разделяемой памятью**. Все процессоры работают с общим адресным пространством. Общаться удобно через общие данные,

но возникает конкуренция за память и проблема когерентности кешей.

2. **Архитектуры с распределённой памятью.** У каждого процессора или узла своя локальная память. Обмен происходит явно через сообщения.

Разделяемая память проще как модель программирования, но хуже масштабируется из-за общего ресурса.

Распределённая память сложнее программно, зато позволяет строить очень крупные системы, где каждый узел независимо работает со своей памятью.

---

## 225. Что из себя представляют архитектуры с разделяемой памятью? Каковы их особенности?

В архитектуре с **разделяемой памятью** несколько процессоров или ядер видят одно общее адресное пространство.

У каждого ядра могут быть свои локальные кеши, но логически память для программы общая.

### Плюсы:

- процессы и потоки легко обмениваются данными через общие переменные;
- модель удобна для программиста;
- не нужно явно посылать сообщение для каждого обмена.

### Проблемы:

- ядра конкурируют за каналы памяти;
- общий интерфейс памяти может стать bottleneck;
- копии данных в разных кешах нужно согласовывать;
- нужны протоколы когерентности и синхронизации.

По мере роста числа процессоров поддерживать единое быстрое пространство становится всё сложнее.

То есть главный плюс — простая коммуникация, главный минус — масштабирование общего ресурса памяти.

---

## 226. Что из себя представляют архитектуры с распределённой памятью? Каковы их особенности?

В архитектуре с **распределённой памятью** у каждого процессора или вычислительного узла есть собственная локальная память.

Другой процессор не может просто прочитать её обычной загрузкой по адресу. Для обмена используются **явные сообщения** через сеть между узлами.

Плюсы:

- нет постоянной конкуренции всех процессоров за одну память;
- локальная память хорошо масштабируется вместе с числом узлов;
- можно строить очень большие системы.

Минусы:

- программисту приходится явно организовывать коммуникацию;
- нужны протоколы передачи и синхронизация;
- производительность зависит от топологии и задержек сети.

Топология может быть кольцевой, ячеистой, древовидной, гиперкубом и т.д.

Идея похожа на распределённые сервисы: каждый узел автономен, а совместная работа строится через обмен сообщениями.

---

## 227. Какие основные типы MIMD парадигм выделяются по классификации Дункана?

Здесь классификация идёт уже не столько по физической памяти, сколько по **модели вычислений**, то есть по тому, как организовано параллельное выполнение задачи.

В исходном материале выделены четыре MIMD-парадигмы:

1. **MIMD/SIMD** — несколько SIMD-вычислителей вместе образуют более крупную MIMD-систему.

2. **Dataflow** — операция запускается, когда готовы её входные данные.
3. **Редукционная архитектура** — вычисление разворачивается от требуемого результата к зависимостям.
4. **Wavefront** — данные и готовность распространяются волной между связанными вычислителями.

Во всех случаях вычислители могут работать относительно независимо, но причина запуска операций и способ передачи данных различаются.

Это и есть более содержательное уточнение огромного класса MIMD, которого не хватает классификации Флинна.

---

## 228. Что представляет собой MIMD/SIMD парадигма и где она применяется?

**MIMD/SIMD** — иерархическая схема, где система содержит несколько независимых SIMD-вычислителей.

Внутри каждого такого блока:

- выполняется один поток инструкций;
- одна команда применяется сразу к множеству данных.

Но разные SIMD-блоки могут одновременно выполнять **разные** команды. Поэтому вся система целиком уже соответствует MIMD.

Идея хорошо подходит для крупных параллельных ускорителей: верхний уровень распределяет разные задачи между группами, а внутри каждой группы выполняется массовая однотипная обработка.

Это позволяет совместить два уровня параллелизма:

1. **task/instruction parallelism** между группами;
2. **data parallelism** внутри каждой SIMD-группы.

Такой принцип близок устройству многих GPU и других массивных параллельных ускорителей, хотя конкретная реализация может отличаться.

---

## 229. Как работает архитектура потоков данных (dataflow)? Подходы к реализации.

В **dataflow-архитектуре** порядок выполнения задаётся не Program Counter, а **готовностью данных**.

Программа представляется как граф:

- вершины — операции;
- рёбра — зависимости и передаваемые значения.

Операция может запуститься сразу, как только пришли все нужные входные данные. Поэтому независимые вершины графа автоматически выполняются параллельно.

Данные могут передаваться как токены, содержащие значение и информацию о том, для какой операции они предназначены.

По исходному материалу реализация может строиться как сеть полноценных вычислителей, которые работают асинхронно и передают результаты друг другу.

Главное отличие от фон-неймановского CPU: там выполнение определяется последовательностью инструкций, а здесь — **потоком зависимостей данных**.

Такой подход естественно извлекает параллелизм из самого графа вычислений.

---

## 230. Как работает редукционная архитектура? Каким образом она обеспечивает параллелизм?

Редукционная архитектура идейно движется **от требуемого результата к данным**, то есть в противоположную сторону относительно обычного dataflow.

Сначала появляется запрос на вычисление верхнего выражения. Чтобы его получить, система определяет, какие подвыражения нужны, затем рекурсивно раскрывает их зависимости, пока не доходит до исходных значений.

Независимые ветви дерева зависимостей могут вычисляться параллельно.

Это похоже на **ленивые вычисления**:

- если результат какой-то ветви не нужен;
- эта ветвь вообще не запускается.

Поэтому потенциально экономятся вычисления и энергия.

Когда результаты подветвей готовы, они сворачиваются обратно вверх до итогового значения.

Главная идея параллелизма: зависимости образуют дерево/граф, и все независимые поддеревья можно вычислять одновременно.

---

## 231. Как работает wavefront архитектура? В чём её отличия от систолической архитектуры?

**Wavefront-архитектура** состоит из связанных вычислительных узлов, по которым готовность данных распространяется как «волна».

Узел начинает работу, когда получил необходимые входы, а после завершения передаёт результат соседям. Поэтому разные участки сети могут работать в разные моменты.

Она похожа на систолическую архитектуру, но есть важное различие.

**Систолическая система:**

- жёстко синхронизирована общим тактом;
- данные сдвигаются по регулярной сети в заданном ритме.

**Wavefront:**

- нет обязательного общего такта для всех узлов;
- передача запускается по готовности данных;
- вычисление более асинхронно.

По сравнению с общим dataflow обычно используется более регулярная и фиксированная топология соседних узлов.

То есть systolic — «движемся по тактам», wavefront — «движемся, когда готовы данные».

---

## 232. Что такое CGRA-процессоры и какие возможности они предоставляют?

**CGRA — Coarse-Grained Reconfigurable Architecture —**

реконфигурируемая архитектура из сравнительно крупных вычислительных блоков.

По идее она находится между FPGA и обычными CPU/DSP-процессорами.

- FPGA содержит очень мелкие универсальные логические ресурсы и позволяет собрать почти произвольную цифровую схему.
- CGRA использует более крупные готовые элементы: ALU, умножители, регистры, локальную память и т.д.
- Эти блоки можно программируемо соединять в нужный datapath.

За счёт крупного зерна CGRA проще и эффективнее FPGA для арифметических задач, но менее универсальна.

Важная возможность — совмещать:

- **spatial computing**: часть алгоритма отображается прямо на сеть аппаратных блоков;
- **temporal computing**: один блок переиспользуется для разных операций во времени.

То есть CGRA пытается получить эффективность специализированного железа, сохраняя возможность реконфигурации.

---

## 233. В чём различие между пространственными (spatial) и временными (temporal) вычислениями на примере CGRA?

**Temporal computing** означает, что один физический вычислительный блок переиспользуется для разных операций во времени.

Например, одна ALU в первом такте считает сложение, в следующем — другую операцию. Это типичная идея CPU.

**Spatial computing** означает, что разные операции алгоритма одновременно отображаются на **разные физические блоки**.

Например: `A -> ALU1 -> ALU2 -> ALU3 -> result`.

После заполнения такого аппаратного конвейера разные элементы данных могут одновременно находиться на разных стадиях.

CGRA позволяет комбинировать оба подхода:

- критические или регулярные части закрепить пространственно;
- менее частые операции выполнять временно, переиспользуя блоки.

Spatial даёт высокий throughput и параллелизм, но требует больше аппаратных ресурсов. Temporal экономит железо, но одну и ту же операционную единицу приходится делить во времени.

---

## 234. Какие подходы существуют к решению проблемы Memory Wall в рамках ASIC и CGRA?

В исходном файле прямо указано, что этого материала не было в лекции. Ниже — стандартное объяснение, чтобы билет не оставался пустым.

**Memory Wall** — ситуация, когда вычислительные блоки становятся быстрее, чем система успевает доставлять им данные из памяти. Процессор простаивает не из-за нехватки ALU, а из-за задержки и пропускной способности памяти.

В ASIC и CGRA проблему уменьшают архитектурно:

- ставят **локальные SRAM/буферы** рядом с вычислителями;
- переиспользуют данные локально, не читая их заново из DRAM;
- строят потоковый datapath, где результат одного блока сразу идёт в следующий;
- организуют DMA и пакетную передачу больших блоков;
- используют несколько банков памяти и параллельные каналы;
- меняют размещение вычислений так, чтобы уменьшить общий data movement.

ASIC может максимально специализировать всю иерархию памяти. CGRA делает похожие оптимизации реконфигурируемо под конкретное отображение алгоритма.

---

## 235. Что такое HDL и RTL? Чем описание аппаратуры на HDL отличается от программирования на языках высокого уровня?

**HDL (Hardware Description Language)** — язык описания цифровой аппаратуры, например Verilog или VHDL.

**RTL (Register-Transfer Level)** — уровень описания, где схема задаётся через:

- регистры, хранящие состояние;
- комбинационную логику между ними;
- передачу данных между регистрами от такта к такту.

Главное отличие HDL от обычного языка программирования — **модель исполнения**.

В C/Python строки в обычной модели задают последовательность действий процессора во времени.

В HDL независимые блоки описывают **физически параллельно работающие части схемы**. Несколько `always` /процессов не означают «сначала первый, потом второй» — они описывают оборудование, существующее одновременно.

Отличается и результат компиляции:

- обычный язык -> машинные инструкции;
- HDL -> синтез -> netlist и физическая/конфигурационная схема.

Даже «переменная» в HDL часто означает не абстрактное значение в RAM, а конкретный провод, регистр или сигнал аппаратуры.

---